



Ain Shams University, Faculty of Engineering

CSE and ECE Departments

Safety Support System (ADAS)

By

Ahmed Yasser	1807401
Khaled Amr	1809517
Mahmoud Abdellatif	1803481
Mahmoud Ehab	1804667
Mahmoud Fathy	1805552
Mahmoud Khaled	1807350
Mohamed Khaled	1806113
Nour Atef	1804318
Rady Magdy	1805504

Supervised By

Prof. Dr. Wagdy Anis

Dr. Bassem Amin

Mentored By

Eng. Yara Mohsen (Valeo Mentorship Program)

Declaration

This is to certify that the project presented herewith, as a component of our partial fulfilment of the Bachelor of Science degree in Electronics and Communications Engineering for Ahmed Yasser, Mahmoud Ehab, Mahmoud Fathy, Nour Atef, and Rady Magdy, and as a component of our partial fulfilment of the Bachelor of Science degree in Computers and Systems Engineering for Khaled Amr, Mahmoud Abdellatif, Mahmoud Khaled, and Mohamed Khaled, is entirely the result of our independent work.

We affirm that we have taken great care to ensure the originality of this project and that it does not infringe upon any copyrighted materials to the best of our knowledge. Furthermore, we declare that no portions of this work have been borrowed from the efforts of others, except where duly cited and acknowledged within the text of our work.

This certification attests that the individuals above have dedicated their efforts and expertise to the completion of this project and that it reflects their genuine understanding and skills acquired during their academic pursuits.

Date: 4th of July, 2023

Signatures:

Name	Signature
Ahmed Yasser	
Khaled Amr	
Mahmoud Abdellatif	
Mahmoud Ehab	
Mahmoud Fathy	
Mahmoud Khaled	
Mohamed Khaled	
Nour Atef	
Rady Magdy	

ABBREVIATIONS AND SYMBOLS

Abbreviation	Meaning
TBEDS	Tire Burst Early Detection System
WCS	Water Cooling System
PAS	Parking Assistance System
ADAS	Advanced Driver-Assistance System
IoT	Internet of Things
SLS	Steering Lock System
TPMS	Tire Pressure Monitoring System
EDS	Early Detection System
PWM	Pulse Width Modulation
UART	Universal Asynchronous Receiver-Transmitter

ABSTRACT

This project presents an integrated system comprising multiple advanced features designed to enhance vehicle safety, performance, and convenience. The project incorporates a Tire Burst Early Detection System, a Water-Cooling System for Tires, a Steering Lock System, an Anti-Collision and Parking Assistance System, and an IoT System. These systems are interconnected through three nodes: the Moving Control Node, the Sensing Control Node, and the IoT Node.

The Tire Burst Early Detection System utilizes state-of-the-art sensors and intelligent algorithms to monitor tire conditions in real-time. By continuously analyzing factors such as tire pressure, and temperature, the system detects potential issues that may lead to tire bursts. Early detection allows for prompt intervention, reducing the risk of accidents and ensuring optimal tire performance.

To address the problem of tire overheating during prolonged use or demanding driving conditions, the project integrates a Water-Cooling System for Tires. This system incorporates a specialized cooling mechanism within the tires to improve heat dissipation, thereby preventing tire damage and enhancing overall performance and longevity.

The Steering Lock System is strategically deployed to bolster vehicle security. This system is designed to automatically engage the steering lock mechanism when a significant pressure threshold is detected, thereby preventing the possibility of uncontrolled vehicle movement upon coming to a halt.

The Anti-Collision and Parking Assistance System utilizes a combination of sensors to assist drivers in avoiding collisions and maneuvering in tight parking spaces. Real-time feedback is provided to the driver, alerting them to potential obstacles.

The systems are developed utilizing the Atmega32 microcontroller as the central processing unit, ensuring robust performance and reliable operation. Seamless data transfer and integration are facilitated through the utilization of communication protocols such as SPI, I2C, and UART (including Wi-Fi and Bluetooth). Furthermore, the project incorporates a diverse range of sensors, including the Ultrasonic Sensor and BMP180 Pressure and Temperature sensor. These sensors enable the collection of real-time data, empowering the system to make well-informed decisions based on accurate and precise information.

The code for each component is written in Embedded C. The microcontroller is programmed to take control of the vehicle in emergency situations, such as a tire burst, and take immediate actions to prevent harm to the vehicle and the driver. By combining the tire burst detection system, steering lock system, the project ensures the overall safety and stability of the vehicle.

The software solution is a comprehensive package that includes an embedded kit, a cloud server, and a mobile app. The embedded kit is equipped with sensors that monitor car location, tire pressure, and temperature. The cloud server integrates the embedded kit with the mobile app and manages user authentication and authorization. The mobile app provides users with real-time data and access to the ADAS system. The solution uses React Native for app development, Node.js for server and website development, and MongoDB for database management. The development process follows an AGILE Scrum model and includes testing.

Contents

INTRODUCTION	1
Tire Bursting Early Detection System (TBEDS)	1
Problem Definition.....	1
Problem Solution	1
Types Of Tire Bursting Handling Systems	1
Water Cooling System for Tires (WCS)	3
Problem Definition.....	3
Problem Solution	3
Car Moving Control System	5
Problem Definition.....	5
Problem Solution	5
Parking Assistance System (PAS)	6
Problem Definition.....	6
Problem Solution	6
Steering Lock System (SLS).....	8
Problem Definition.....	8
Problem Solution	8
Software Solution.....	10
Problem Definition.....	10
Problem Solution	10
PROTOTYPE	11
SYSTEM DESIGN AND ARCHITECTURE	13
Moving Control Node	14
Hardware Design (Circuit Diagram).....	14
Components Description.....	14
Layered Architecture.....	18
Application Code	18
Sensing Control Node	26
Hardware Design (Circuit Diagram).....	26
Components Description.....	26
Layered Architecture.....	33
Application Code	33
IoT Node	52
Hardware Design (Circuit Diagram).....	52
Components Description.....	52

Layered Architecture.....	57
Application Code	57
Mobile Application	62
App.js	62
Src folder:.....	62
Components folder.....	62
constants folder	70
navigation folder	71
screens folder	74
Amplify folder	101
Car information entry system (Web Application).....	102
Login Page	102
Dashboard Page	102
Add Car Page	102
Add User to Car Page	103
Server	104
MongoDB database.....	104
APIs that communicate with embedded kit	106
API that adds new car to the system	106
API that adds user to car	106
API getnewcars	107
Socket sellcar	107
Socket carData	107
Socket connection	107
Socket request tires temp and pressure	107
Socket running status	107
Socket siginconnect	107
Socket disconnect.....	107
distance.js.....	107
INTEGRATION AND TESTING	108
Unit Testing	108
BMP	108
GPS	112
Bluetooth.....	115
Integration Testing.....	119
Sensing node testing modes:	119

TOOLS AND PLATFORMS EMPLOYED.....	124
GCC compiler	124
Eclipse IDE	125
Visual studio Code with GitHub copilot extension and GitHub version control.....	126
CONCLUSION.....	128
APPENDIX.....	130
DIO Peripheral.....	130
Definition	130
Operation.....	130
Interrupts	130
Definition	130
Operation.....	131
Timer Peripheral	131
Definition	131
Operation.....	132
ICU.....	132
Definition	132
Operation.....	133
UART Communication Protocol	133
Definition	133
Operation.....	133
I2C Communication Protocol	134
Definition	134
Operation.....	135
SPI Communication Protocol.....	135
Definition	135
Operation.....	135
Wi-Fi.....	136
Definition	136
Operation.....	137
Bluetooth.....	137
Definition	137
Operation.....	137
Ultrasonic Sensor	138
HC-SR04 Pin Description.....	138
HC-SR04 Working Principle	139

Distance calculations.....	139
Steps of calculations	139
GPS Module.....	140
Working Principle of Ublox NEO-6M.....	140
REFERENCES	141

Figure 1: Prototype	11
Figure 2: Indications Layer (3D Printed)	12
Figure 3: Moving Control Node Proteus Simulation	14
Figure 4: HC-05 Bluetooth Module 1	14
Figure 5: HC-05 Bluetooth Module 2	14
Figure 6: Atmega32 Mcicrocontroller	15
Figure 7: Controlling RC Car Mobile App	16
Figure 8: HC-05 Bluetooth Module 3	16
Figure 9: DC Motor	18
Figure 10: Layered Architecture for Moving Control Node	18
Figure 11: QR Code for the Moving Control Node Folder.....	25
Figure 12: Sensing Control Node Proteus Simulation	26
Figure 13: Table describing the states of the tire	28
Figure 14: Flow chart showing the working sequence of SLS and EDS	29
Figure 15: Table shows the cases in which water cooling system is activated.....	30
Figure 16: Different distance ranges of the PAS	30
Figure 17: Flow Chart shows how the ultrasonic sensor works.....	31
Figure 18: Layered Architecture of Sensing Control Node	33
Figure 19: QR Code for the Sensing Control Node Folder.....	51
Figure 20: IoT Node Proteuos Simulation	52
Figure 21: GPS_sendingCoordinatesTask Flow Chart	54
Figure 22: BMP180_sendingDataTask Flow Chart.....	55
Figure 23: Layered Architecture for IoT Node	57
Figure 24: QR Code for the IoT Node Folder.....	61
Figure 25: Src Folder	62
Figure 26: Components Folder	63
Figure 27: ProgressBar	67
Figure 28: Sign in screen	68
Figure 29: CreateStore	69
Figure 30: Socket Folder 1.....	70
Figure 31: Constants Folder	70
Figure 32: Navigation Folder	71
Figure 33:BottomTabNav	72
Figure 34: Screens Folder	74
Figure 35: Test Case Diagram 1	109
Figure 36: Test Case Diagram 2	110
Figure 37: Test Case Diagram 3	112
Figure 38: HC-05 Testing Procedure	118
Figure 39: GCC Compiler.....	124
Figure 40: Eclipse IDE.....	125
Figure 41: VS.....	126
Figure 42: GitHub Copilot	126
Figure 43: GitHub	126
Figure 44: Ultrasonic Sensor.....	138
Figure 45: Ultrasonic Pinout	138
Figure 46: Ultrasonic Timing Diagram.....	139
Figure 47: GPS module.....	140
Figure 48: QR Code for the references	141

This Page is intentionally left blank

INTRODUCTION

Tire Bursting Early Detection System (TBEDS)

Problem Definition

A tire burst can occur for any reason, when the tire's internal structure fails due to overloading, underinflation, or aging, causing the tire to lose pressure rapidly. This can cause the tire to deflate, and the vehicle to lose stability and control, which can lead to an accident.

Flat tires can happen unexpectedly and can lead to a loss of control of the vehicle, especially at high speeds. This can result in accidents and injuries to the driver and passengers, as well as damage to the vehicle. Flat tires can also happen gradually, leading to decreased fuel efficiency and increased wear and tear on the tires, which can result in the need for frequent replacements.

Problem Solution

The system uses sensors that monitor the tire pressure and temperature, which are typically mounted on each wheel. These sensors can be either direct or indirect sensors. Direct sensors measure the tire pressure and temperature directly, while indirect sensors measure the air pressure and temperature inside the vehicle's air springs or air bags.

When a tire temperature burst is detected, the sensor data is sent to the electronic control unit (ECU) which compares the data to predetermined threshold values. If the sensor data indicates a tire burst, the ECU will activate a warning mechanism, such as a visual, to alert the driver of the problem.

The ECU will also take control of the vehicle's braking and stability systems to slow down and bring the vehicle to a safe stop. This can include reducing the engine power, applying the brakes, and adjusting the suspension to maintain stability.

It's worth noting that while these systems can be helpful in detecting and responding to a tire burst, it's important for the driver to check and maintain tire pressure regularly, and to replace tires when they become worn, in order to prevent tire burst.

Types Of Tire Bursting Handling Systems

- 1) Passive tire bursting handling system: This system only warns the driver if a tire burst has occurred.
- 2) Active tire bursting handling system: This system not only warns the driver but also takes control of the vehicle, slows it down, and brings it to a stop in a safe manner to avoid accidents.

Additionally, some tire bursting handling systems can also integrate with other vehicle systems, such as anti-lock brakes, traction control, and electronic stability control, to enhance the vehicle's stability and safety. This integration can allow for a more coordinated and efficient response to a tire burst, and can help prevent accidents or loss of control of the vehicle.

TBEDS is a system that uses sensors to monitor the pressure and temperature of tires in real-time, and alerts the driver if a tire is in danger of bursting. This can help to:

- prevent accidents caused by tire failure,
- help to extend the life of tires by alerting the user to issues before they become critical.

There are several different types of sensors that can be used in TBEDS:

- pressure sensors
- temperature sensors

These sensors are typically mounted on the outside of the tire, and communicate with a central control unit that analyses the data and generates alerts as needed.

Some of the key benefits of TBEDS include:

- Improved safety: By alerting the driver to potential tire failure before it occurs, preventing accidents and injuries.
- Extended tire life: By alerting the user to issues with tires before they become critical, extending the life of tires and reducing the need for costly replacements.
- Increased efficiency: By monitoring tire pressure and temperature in real-time, ensuring that tires are always properly inflated and operating at optimal temperatures, thus improving fuel efficiency and reducing wear and tear on the tires.

Overall, a Tire Bursting Early Detection System is an important tool for ensuring the safety and reliability of vehicles, and can help to reduce the costs associated with tire failure.

Some common scenarios where a Tire Bursting Early Detection System may be particularly useful include:

- Long distance driving: The prolonged use of a tire can cause it to overheat, leading to a tire burst. So, it can detect this and alert the driver to take a break and check the tires.
- High-speed driving: High-speed driving can put additional stress on tires and increase the risk of a tire burst. So, it can detect when a tire is overheating and alert the driver to slow down.
- Under-inflation: Tires that are under-inflated can overheat and burst. So, it can detect when a tire's pressure is low and alert the driver to inflate it to the proper level.
- Overloading: Carrying a heavy load in the vehicle can put additional stress on the tires and increase the risk of a tire burst. So, it can detect this and alert the driver to adjust the load.
- Punctures and damages: it can detect when a tire has been punctured or has suffered some damages, alerting the driver to take action before the tire fails.

It's important to note that a TBEDS is not a substitute for regular tire maintenance and inspection. Drivers should still check their tires regularly, including the pressure and the wear, to ensure they are in good condition.

Water Cooling System for Tires (WCS)

Problem Definition

The problem at hand is the overheating of tires during prolonged or intensive use, which can lead to reduced performance, increased wear and tear, and potential safety hazards. Traditional air-cooling methods are insufficient in effectively dissipating heat from the tires, especially in high-performance or heavy-duty applications. As a result, there is a need for an efficient water-cooling system for tires that can effectively regulate their temperature and enhance their overall performance and longevity.

Problem Solution

To address the issue of tire overheating, a water-cooling system can be implemented. This system would involve the integration of a network of small, lightweight tubes within the tire structure, capable of circulating coolant fluid. Here's a step-by-step solution:

- **Tire Design:** The tire design would be modified to accommodate the water-cooling system. This could involve incorporating channels or grooves within the tire tread or sidewalls to house the cooling tubes without compromising the tire's structural integrity.
- **Coolant Circulation:** The cooling system would require a pump to circulate the coolant fluid through the network of tubes within the tire. This pump could be electrically powered and controlled by a dedicated system, which monitors and regulates the tire's temperature.
- **Heat Exchange:** To facilitate efficient heat exchange, the cooling tubes within the tire would be strategically positioned to maximize contact with the tire's surface. This would allow the coolant to absorb heat generated during tire operation.
- **Coolant Reservoir:** A dedicated coolant reservoir would be integrated into the vehicle's system. This reservoir would store an adequate amount of coolant fluid, ensuring a continuous supply for the tire cooling system.
- **Temperature Monitoring:** Sensors placed within the tire would monitor the temperature and relay the information to a central control unit. The control unit would analyze the data and adjust the coolant flow rate accordingly to maintain optimal tire temperature.
- **Vehicle Integration:** The cooling system would be integrated into the vehicle's existing infrastructure, connecting the coolant reservoir, pump, and control unit to the tire cooling system. This integration would ensure seamless operation and compatibility with the overall vehicle system.

Benefits:

- **Temperature Regulation:** The water-cooling system would effectively regulate the tire temperature, preventing overheating and improving tire performance and longevity.

- **Increased Safety:** By maintaining optimal temperature, the system reduces the risk of tire blowouts, improving safety during operation.
- **Enhanced Performance:** Cooler tires would provide improved traction, handling, and braking, especially in demanding driving conditions or high-performance applications.
- **Extended Tire Life:** By minimizing heat-related wear and tear, the system would help prolong the life of the tires, reducing the frequency of replacements and associated costs.

Car Moving Control System

Problem Definition

In cases such as tire bursts, spontaneous reactions may lead to crucial accidents which may cause critical injuries and, in some cases, it may lead to death.

Problem Solution

The Car Moving Control System is an integral part of the vehicle's safety measures, designed to respond effectively in the event of a tire burst. When a tire burst occurs, causing an increase in temperature beyond a threshold and a sudden decrease in air pressure resulting in a flat tire, the Car Moving Control System takes immediate action to ensure the safety of the vehicle and its occupants. It assumes control from the driver, gradually reduces the speed of the car, locks the steering, and safely brings the vehicle to a stop.

- **Tire Burst Detection:** The Car Moving Control System continuously monitors tire temperature and pressure. Once it detects a tire burst, it activates the emergency response mechanism.
- **Gradual Speed Reduction:** The system takes control of the vehicle and initiates a controlled speed reduction process. This gradual decrease in speed ensures stability and minimizes the risk of sudden jolts or loss of control.
- **Steer Lock Activation:** To enhance safety during the tire burst event, the Car Moving Control System engages the steer lock mechanism. By locking the steering, it ensures that the vehicle maintains its heading, preventing potential accidents caused by veering off the road or colliding with other vehicles.
- **Controlled Stop:** The system continues to reduce the vehicle's speed gradually until it comes to a complete stop. This controlled stop allows for a safe and stable deceleration process, reducing the risk of accidents or further tire damage.
- **Driver Notification:** Throughout the tire burst response process, the Car Moving Control System provides clear and timely notifications to the driver. These notifications inform the driver about the tire burst event and the actions being taken by the system to ensure safety.

In summary, The Car Moving Control System plays a crucial role in responding to a tire burst event. By assuming control of the vehicle, gradually reducing speed, locking the steering, and safely bringing the vehicle to a stop, it ensures the safety of the vehicle and its occupants. This system provides a reliable and effective solution to mitigate the risks associated with tire bursts, maintaining stability and preventing accidents.

Parking Assistance System (PAS)

Problem Definition

The problem at hand is the difficulty faced by drivers in accurately and safely maneuvering their vehicles into parking spaces, particularly in tight or crowded areas. Limited visibility, blind spots, and spatial awareness challenges often lead to accidental collisions with other vehicles, curbs, or obstacles, causing property damage and potential safety risks. Therefore, there is a need for an effective parking assistance system that can aid drivers in parking their vehicles with precision and safety.

Problem Solution

To address the challenges associated with parking, a parking assistance system can be implemented. This system would leverage sensors, cameras, and advanced algorithms to provide drivers with real-time feedback and guidance during the parking process. Here's a step-by-step solution:

- **Sensor Installation:** The parking assistance system would involve the installation of various sensors on the vehicle, such as ultrasonic or radar sensors. These sensors would be strategically positioned around the vehicle to detect nearby obstacles, including other vehicles, curbs, walls, or pedestrians.
- **Camera Integration:** In addition to sensors, the system may also include cameras, such as rearview or surround-view cameras, to provide visual feedback and enhance the driver's situational awareness during parking maneuvers.
- **Data Processing and Analysis:** The sensor and camera data would be processed and analyzed by a dedicated onboard computer or control unit. Advanced algorithms would interpret the data and provide real-time information regarding the distance and position of obstacles in relation to the vehicle.
- **Visual and Audible Feedback:** The parking assistance system would communicate with the driver through visual and audible cues. This could involve the use of a display screen or dashboard indicators to show the proximity of obstacles, along with audio alerts or beeps to warn the driver of potential collisions.
- **Guidance and Path Prediction:** The system would provide guidance to the driver by calculating and displaying the optimal steering angle, indicating the direction and intensity of the steering input required for safe parking. Additionally, the system could predict the path of the vehicle during the parking maneuver, enabling the driver to visualize the trajectory and adjust accordingly.
- **Automated Parking (Optional):** In more advanced implementations, the parking assistance system could offer automated parking functionality. This feature would allow the vehicle to take over the steering inputs and maneuver into the parking space autonomously, with the driver only controlling the throttle and brake.

Benefits:

- **Improved Safety:** The parking assistance system would significantly reduce the risk of collisions and accidents during parking, enhancing overall safety for the driver, passengers, and pedestrians.
- **Increased Accuracy:** By providing real-time feedback and guidance, the system would enable drivers to park their vehicles with higher precision, minimizing the likelihood of scratches, dents, or other damages.
- **Enhanced Convenience:** Parking assistance systems simplify the parking process, making it more convenient and less time-consuming for drivers, especially in congested or challenging parking scenarios.
- **Reduced Stress:** The system's visual and audible cues alleviate the stress and anxiety associated with parking, particularly for inexperienced or nervous drivers.
- **Versatility:** Parking assistance systems can be integrated into various types of vehicles, ranging from compact cars to larger vehicles like SUVs or trucks, catering to a wide range of parking needs.

It's worth noting that drivers should remain attentive and actively monitor their surroundings, even when using parking assistance systems, as these systems are meant to augment driver capabilities, not replace them entirely. Regular maintenance and calibration of the parking assistance system would also be necessary to ensure accurate and reliable performance.

Steering Lock System (SLS)

Problem Definition

The problem is the potential risk of uncontrolled vehicle movement when a vehicle comes to a halt, which can pose safety hazards and lead to accidents or property damage. Without a reliable steering lock system, the vehicle's steering can be easily manipulated, allowing unauthorized individuals or external factors to initiate movement, even if the vehicle is intended to remain stationary. This lack of control over vehicle movement creates a safety concern that needs to be addressed.

Problem Solution

To mitigate the risk of uncontrolled vehicle movement, a steering lock system can be implemented. This system would automatically engage the steering lock mechanism when a significant pressure threshold is detected, ensuring that the vehicle remains stationary and cannot be easily maneuvered. Here's a step-by-step solution:

- **Pressure Sensing Technology:** The steering lock system would incorporate pressure sensors that monitor the force applied to the vehicle's wheels or suspension system. These sensors can detect when the vehicle comes to a halt and determine if there is a significant pressure threshold indicating a stationary position.
- **Automatic Lock Activation:** Once the pressure sensors detect that the vehicle has reached the desired threshold indicating a halt, the steering lock system automatically engages the lock mechanism. This mechanism can be designed to physically lock the steering column, preventing any movement of the steering wheel.
- **Integration with Ignition System:** The steering lock system can be integrated with the vehicle's ignition system. When the ignition is turned off or the engine is stopped, it triggers the activation of the steering lock system. This ensures that the steering is locked whenever the vehicle is not in motion, providing an additional layer of security.
- **Driver Override:** To accommodate driver control, the steering lock system should have a mechanism for easy disengagement by authorized individuals. This can involve using a specific key or inputting a unique code to deactivate the steering lock and enable normal steering operation.

Benefits:

- **Enhanced Safety:** The automatic engagement of the steering lock system prevents uncontrolled vehicle movement, reducing the risk of accidents, collisions, or unintended rollaways when the vehicle is stationary.
- **Theft Deterrence:** The steering lock system acts as a deterrent to vehicle theft, making it more challenging for unauthorized individuals to manipulate the steering and steal the vehicle.
- **Convenience:** The automatic activation of the steering lock eliminates the need for the driver to manually engage the lock, providing convenience and peace of mind when coming to a halt.

- **Compatibility:** The steering lock system can be designed to integrate with a wide range of vehicles, making it adaptable for various types of automobiles, including cars, trucks, and motorcycles.
- **Seamless Operation:** The integration of the steering lock system with the ignition system ensures seamless functionality, as it automatically activates when the engine is turned off or the vehicle is stopped.

Software Solution

Problem Definition

The problem that the system we are making tries to solve is the user-interface between the owner of the car and the detailed and comprehensive ADAS system that is installed in the car.

There is also the problem of taking this data from the embedded kit and trying to use it to do useful applications and get good insights into the car condition and status.

Problem Solution

For the solution of the first problem stated above we created a mobile application that speaks and interacts with a web server that takes the sensors readings from the embedded kit and show to the user data and statistics about his/her car.

The solution for the second problem is to show the user graphs about the car readings and store logs for past readings and show other statistics that might interest the user.

Mobile Application

The mobile application we built uses react native as the technology for building. It is used to authenticate the user and authorize him/her to be able to access the readings of the car and do various activities within the application.

Web Applications

The web application is used to make the employees of the company add new cars and to match cars bought with users' emails to add them to the database and send OTPs to the user for security when signing up.

Server

The server is written with Node.js with framework express.js. it is used as the mediator between the embedded kit and the mobile application and the database. It has APIs to connect with the embedded kit and then stores this data to the database and also APIs to initiate socket connection with the client (mobile application) to be able to fetch and send data from and to the server.

MongoDB and Socket.io

We use the MongoDB database as it is compatible with the application we created and because of the ability to add many documents without a defined schema so we have more freedom. Socket connection is used for the connection between the client and the server as it is a dual communication service so that the client can initiate the connection asking for data and the server can initiate the connection sending or requesting data.

PROTOTYPE

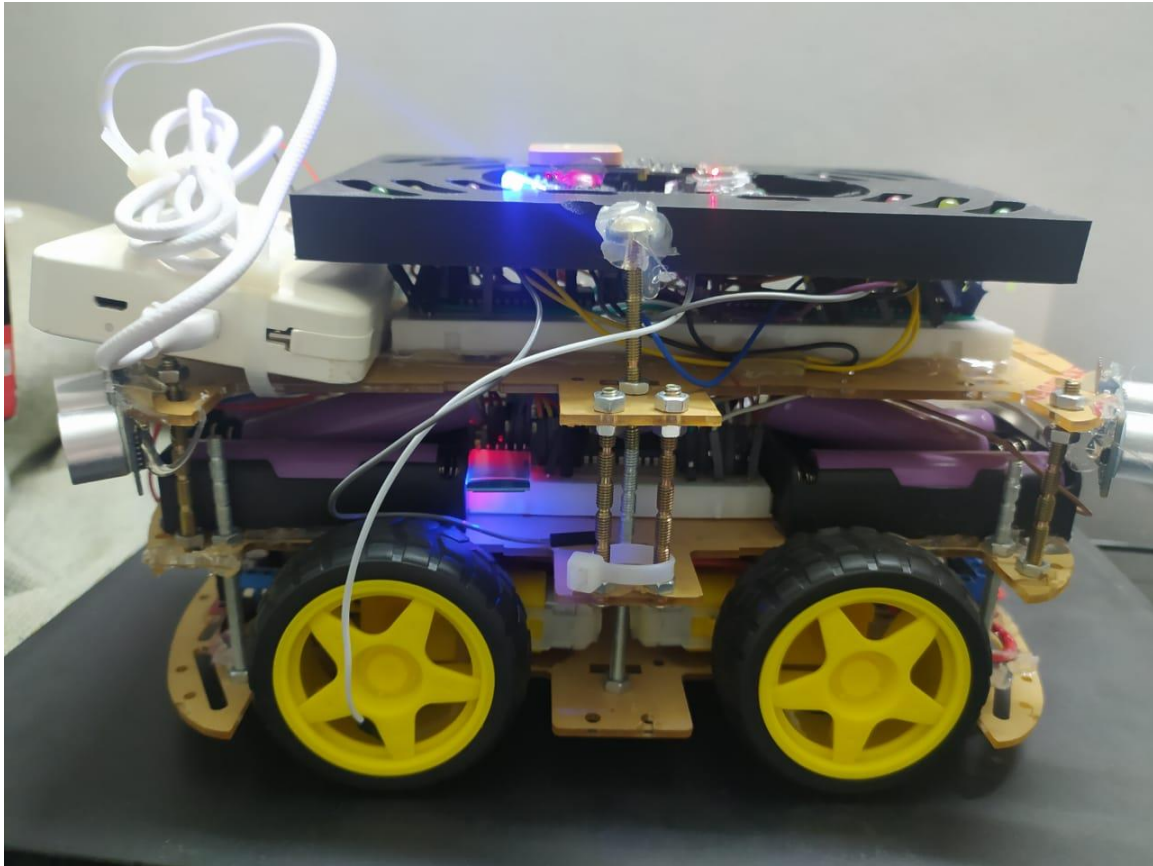


Figure 1: Prototype

Our innovative prototype introduces an RC car that revolutionizes the driving experience. This RC car is controlled via Bluetooth using UART signals, allowing precise and responsive control over the steering, brakes, and driving states. Through this wireless communication, users can effortlessly navigate the car with ease and precision.

To further enhance the functionality of the RC car, we have integrated a BMP180 sensor. This sensor accurately measures air pressure and temperature, simulating tire pressure and temperature. The data captured by the BMP180 sensor is then transmitted to the server using an ESP01 module. Along with the tire pressure and temperature information, the location of the car is also sent to the server.

The server plays a vital role in processing the received data. It analyzes the tire pressure and temperature readings, providing valuable insights into the condition of the tires. These tire states are made accessible to the user through a dedicated mobile application. Users can conveniently monitor the tire status, ensuring optimal performance and identifying any potential issues.

To further augment the safety features of our prototype, ultrasonic sensors have been incorporated. These sensors accurately calculate distances at the corners, enabling the car to assess its surroundings and detect potential collisions. This functionality ensures safe and reliable maneuvering, preventing accidents or damage to the car.

Moreover, we have added a 3D printed layer to the RC car, which features integrated LEDs. These LEDs serve multiple purposes. Firstly, they provide visual indicators during cornering and parking maneuvers, alerting the user to the proximity of obstacles. This helps the user make informed decisions and avoid collisions. Additionally, LEDs indicate the tire states, highlighting whether they are in good condition or require maintenance or repair. Furthermore, LEDs display the connectivity status of both the GPS module and the ESP01 module, providing the user with real-time information on the functionality of these modules.

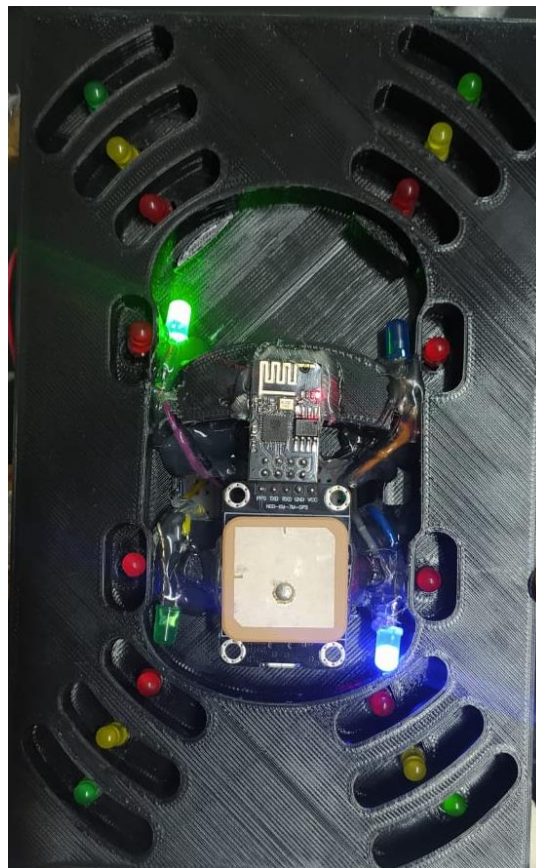


Figure 2: Indications Layer (3D Printed)

In summary, our prototype RC car showcases an array of advanced features. These include Bluetooth-controlled movements, simulated tire pressure and temperature monitoring, real-time data transmission to a server, tire state updates on a mobile app, ultrasonic sensors for distance calculation, and a 3D printed layer with LEDs for visual feedback. This comprehensive integration of technologies delivers an immersive and safe driving experience, ensuring optimal performance and enhancing user convenience.

SYSTEM DESIGN AND ARCHITECTURE

Our system comprises three essential nodes: the Moving Control Node, the Sensing Control Node, and the IoT Node. Additionally, we have developed a comprehensive software solution that encompasses a mobile app, a web application, a server, and a database utilizing mongoDB and Socket.io.

The Moving Control Node plays a pivotal role in accessing and controlling the movement of the car. It simulates the car's motion and enables the application of steering actions. By interfacing with the car's control mechanisms, this node allows for precise control over the car's movements.

The Sensing Control Node is responsible for monitoring the tire pressure and temperature. It utilizes specialized sensors and employs the SPI (Serial Peripheral Interface) communication protocol to gather data. This node then transmits the collected tire pressure and temperature information to the IoT Node, which acts as a centralized hub for data aggregation and transmission.

The IoT Node acts as an intermediary between the Sensing Control Node and the server. It receives the tire pressure and temperature data from the Sensing Control Node via SPI and combines it with the car's location information. Subsequently, the IoT Node transmits this consolidated data to the server for further processing and analysis.

The server, a crucial component of our software solution, receives the data from the IoT Node and stores it for future retrieval and analysis. It serves as a central repository for all the collected information, enabling seamless integration with the mobile app and web application. The server facilitates real-time updates and ensures that the latest tire pressure and temperature data, as well as the car's location, are accessible to users.

The mobile app and web application provide user-friendly interfaces for accessing and visualizing the data collected by the system. Users can easily monitor the tire pressure and temperature in real-time through these applications. Additionally, in case of extreme pressure or temperature values, the system triggers a signal to interrupt normal control and initiate appropriate actions to bring the car to a safe state.

Moving Control Node Hardware Design (Circuit Diagram)

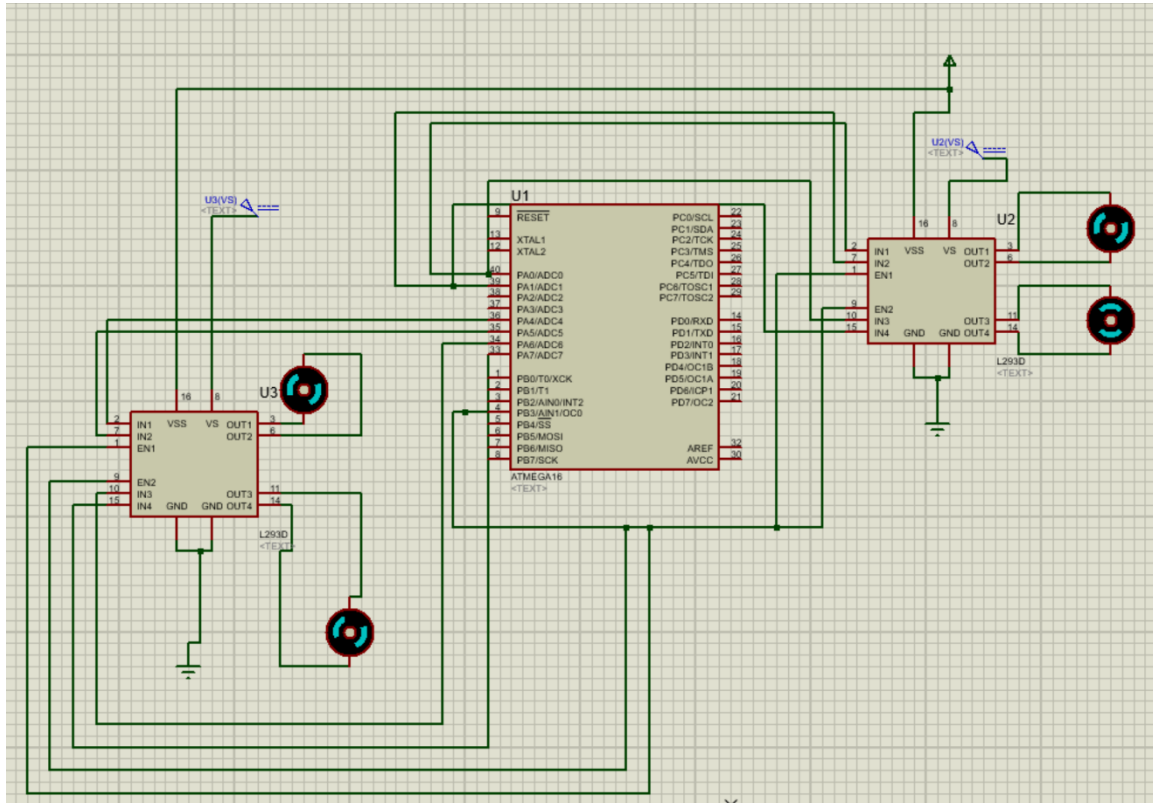


Figure 3: Moving Control Node Proteus Simulation

Components Description

Bluetooth Module

The Bluetooth module enables wireless communication between the RC car and a controlling device, such as a smartphone or a computer. Using HC-05 Bluetooth Module This module use the Bluetooth protocol to establish a connection with the controlling device and transmit/receive data wirelessly.

It has range up to 100m which depends upon transmitter and receiver, atmosphere, geographic & urban conditions. It is IEEE 802.15.1 standardized protocol, through which one can build wireless Personal Area Network ([PAN](#)). It uses frequency-hopping spread spectrum ([FHSS](#)).

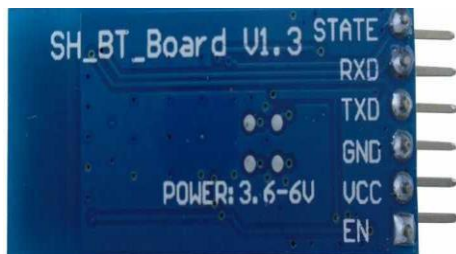


Figure 4: HC-05 Bluetooth Module 1

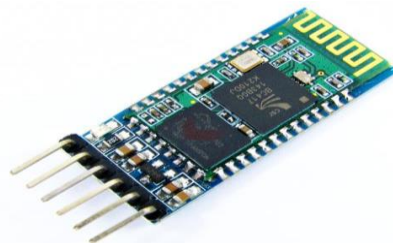


Figure 5: HC-05 Bluetooth Module 2

It uses serial communication to communicate with devices. It communicates with microcontroller using serial port (USART) at 2.4 GHz ISM band.

1. Key/EN: It is used to bring Bluetooth module in AT commands mode. If Key/EN pin is set to high, then this module will work in command mode. Otherwise by default it is in data mode. The default baud rate of HC-05 in command mode is 38400bps and 9600 in data mode.

HC-05 module has two modes,

1. Data mode: Exchange of data between devices.
2. Command mode: It uses AT commands which are used to change setting of HC-05. To send these commands to module serial (USART) port is used.
2. VCC: Connect 5 V or 3.3 V to this Pin.
3. GND: Ground Pin of module.
4. TXD: Transmit Serial data (wirelessly received data by Bluetooth module transmitted out serially on TXD pin)
5. RXD: Receive data serially (received data will be transmitted wirelessly by Bluetooth module).
6. State: It tells whether module is connected or not.

Microcontroller

The microcontroller serves as the brain of the RC car, receiving commands from the Bluetooth module and controlling the car's motors, and other functionalities. Using atmega 32 microcontroller.

Configure the I/O pins of the ATmega32 for motor control and Bluetooth connection. Initialize the necessary timers and PWM (Pulse Width Modulation) channels for motor control.



Figure 6: Atmega32 Mcicrocontroller

Hardware Connections

Connect the HC-05 module to the ATmega32 microcontroller. The module uses UART (Universal Asynchronous Receiver-Transmitter) communication, so you'll need to connect the HC-05's TX pin to the ATmega32's RX pin, and the HC-05's RX pin to the ATmega32's TX pin. Additionally, connect the module's VCC and GND pins to the appropriate power and ground pins of the microcontroller.

Bluetooth-controlled RC car system works

1. the application device (Bluetooth RC Controller) sends control commands (forward, backward, left, right and controlling its Speed) via a Bluetooth connection to the Bluetooth module on the RC car.

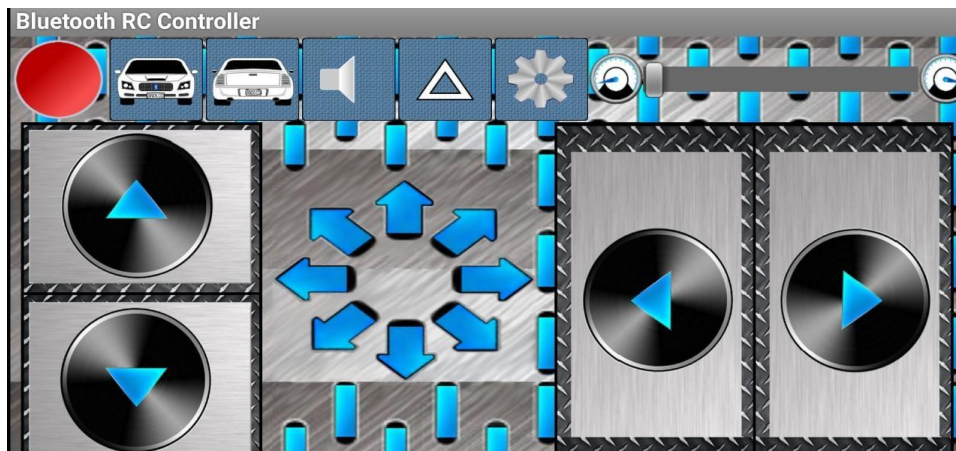


Figure 7: Controlling RC Car Mobile App

2. The Bluetooth module receives the commands and forwards them to the microcontroller:

- Initialize the UART (Universal Asynchronous Receiver-Transmitter) module of the ATmega32 to communicate with the HC-05 Bluetooth module.
- Set up the appropriate baud rate, parity, and other communication parameters to match the settings of the HC-05 module.
- Enable the interrupt-based UART receive feature to receive incoming Bluetooth commands.
- Enable Bluetooth on the controlling device (e.g., smartphone) and search for available devices.
- Pair the device with the HC-05 module by following the pairing procedure for the module. The procedure typically involves entering a passcode or using a default passcode (e.g. 1234) .

3. The microcontroller interprets the received commands and controls the various components of the RC car accordingly. For example, it may activate specific motors to move the car forward or backward, control the speed or its direction.

4. The RC car responds to the control commands, and the microcontroller continuously processes new commands received from the Bluetooth module to maintain real-time control over the car's movements.

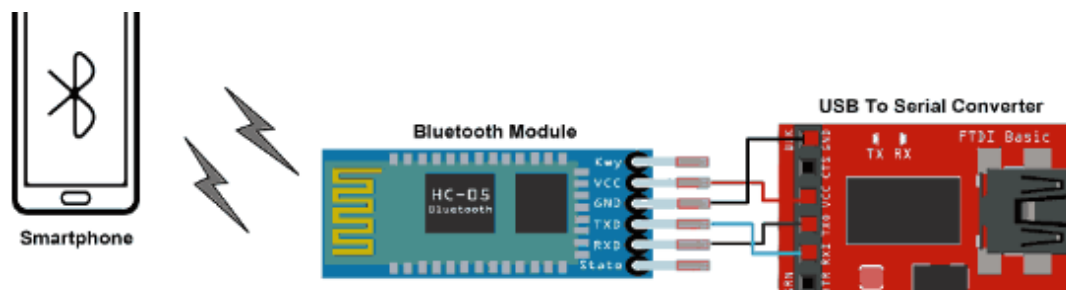


Figure 8: HC-05 Bluetooth Module 3

Motors

1. Use of 4 DC Motors:

The RC car utilizes four DC motors for its propulsion. These motors are responsible for providing the necessary torque and rotational motion to move the car in different directions.

2. PWM Channels for Speed Control:

To control the speed of the motors, the system employs PWM (Pulse Width Modulation) channels. PWM allows for precise control of the motor speed by generating variable duty cycle signals. By adjusting the duty cycle, the effective voltage and thus the speed of the motors can be regulated.

3. Motor Control Signals:

Based on the interpreted commands, the system generates motor control signals to direct the motors in the desired direction and at the desired speed. The control signals are sent to each motor independently.

4. Motor Control Algorithms:

The system implements motor control algorithms, particularly PWM-based speed control, to enable movement of the RC car in four directions:

a. Forward: To move the car forward, all four motors are activated in a clockwise direction. This coordinated rotation propels the car forward.

b. Backward: Moving the car backward involves activating all four motors in an anticlockwise direction. The opposite rotation causes the car to move in reverse.

c. Right: For a right turn, two motors in the back of the car are activated in a clockwise direction, while the left motor is also activated in a clockwise direction. However, the right motor rotates in an anticlockwise direction. This differential rotation allows the car to turn right.

d. Left: Similarly, for a left turn, two motors in the back of the car are activated in a clockwise direction, while the right motor is activated in a clockwise direction. However, the left motor rotates in an anticlockwise direction. This differential rotation facilitates a left turn.

e. Stop: To bring the car to a halt, all four motors are stopped by deactivating the motor control signals. The absence of rotation causes the car to stop moving.

By implementing these motor control algorithms and adjusting the PWM signals accordingly, the RC car can be maneuvered in different directions and at varying speeds. This allows for precise control and navigation of the vehicle.



Figure 9: DC Motor

Layered Architecture

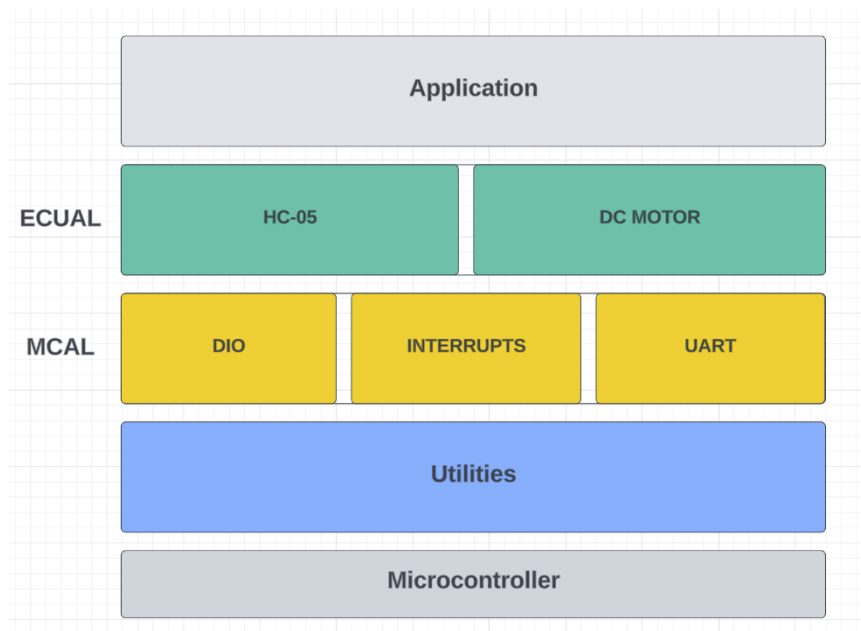


Figure 10: Layered Architecture for Moving Control Node

Application Code

app.h

```

#ifndef APPLICATION_APP_H_
#define APPLICATION_APP_H_

/*****
Application Dependency
*****/
#include "../ECUAL/MOTOR/motor_rc.h"
#include "../ECUAL/HC05/hc05.h"
/*****
Functions Prototypes
*****/
  
```

```
void app_Init(void);
void app_Start(void);
#endif /* APPLICATION_APP_H_ */
```

app.c

```
#include "app.h"

/*****
    Functions Definitions
*****/
void app_Init(void)
{
    delay(1);
    HC05_init(SLAVE);
    DC_Motor_Init();
}

void app_Start(void)
{
    if(u8_severeTiresStateFlag)
    {
        keep_Car_Safe();
    }
    else
    {
        t_carState = move_car();
    }
}
```

keep_Car_Safe function

```
void keep_Car_Safe(void)
{
    switch(t_carState.gearState)
    {
        case FORWARD:
        case LEFT:
        case RIGHT:
            // Set Gear: FORWARD
            t_carState.gearState = FORWARD;
            set_Car_State(t_carState);
            // Limit the speed to 50%.
            break;
        case BACKWARD:
        case LEFT_BACK:
        case RIGHT_BACK:
            t_carState.gearState = BACKWARD;
            set_Car_State(t_carState);
            // Limit the speed to 50%.
            break;
        case BRAKE:
        default:
            // Only Limit The Car Speed Until return to Safe or Moderate State.
            break;
    }

    switch(t_carState.carSpeed)
    {
        case SPEED_100:
            t_carState.carSpeed = SPEED_90;
            set_Car_State(t_carState);
            _delay_ms(250);
            _delay_ms(250);
        case SPEED_90:
            t_carState.carSpeed = SPEED_80;
            set_Car_State(t_carState);
            _delay_ms(250);
            _delay_ms(250);
        case SPEED_80:
            t_carState.carSpeed = SPEED_70;
            set_Car_State(t_carState);
            _delay_ms(250);
            _delay_ms(250);
        case SPEED_70:
            t_carState.carSpeed = SPEED_60;
            set_Car_State(t_carState);
            _delay_ms(250);
            _delay_ms(250);
    }
}
```

```

case SPEED_60:
    t_carState.carSpeed = SPEED_50;
    set_Car_State(t_carState);
    _delay_ms(250);
    _delay_ms(250);
case SPEED_50:
    t_carState.carSpeed = SPEED_40;
    set_Car_State(t_carState);
    _delay_ms(250);
    _delay_ms(250);
case SPEED_40:
    t_carState.carSpeed = SPEED_30;
    set_Car_State(t_carState);
    _delay_ms(250);
    _delay_ms(250);
case SPEED_30:
    t_carState.carSpeed = SPEED_20;
    set_Car_State(t_carState);
    _delay_ms(250);
    _delay_ms(250);
case SPEED_20:
    t_carState.carSpeed = SPEED_10;
    set_Car_State(t_carState);
    _delay_ms(250);
    _delay_ms(250);
case SPEED_10:
    t_carState.carSpeed = SPEED_0;
    set_Car_State(t_carState);
    _delay_ms(250);
    _delay_ms(250);
case SPEED_0:
    delay(10); /*Time estimated to give back the control to the driver*/
    sei();
default:
    // Do Nothing.
    break;
}
u8_speedIsLimited = SPEED_IS_LIMITED;
u8_severeTiresStateFlag = 0;
}

```

nove_Car function

```

carState move_car(void) {
    carState state;
    uint8 car_info = HC05_receiveByte();
    switch (car_info)
    {

```

```

case 'S':
case 'D':
    set_car_direction(BRAKE);
    state.gearState = BRAKE;
    break;

case 'F':
    set_car_direction(FORWARD);
    state.gearState = FORWARD;
    break;

case 'B':
    set_car_direction(BACKWARD);
    state.gearState = BACKWARD;
    break;

case 'H':
    set_car_direction(LEFT_BACK);
    state.gearState = LEFT_BACK;
    break;

case 'J':
    set_car_direction(RIGHT_BACK);
    state.gearState = RIGHT_BACK;
    break;

case 'L':
case 'G':
    set_car_direction(LEFT);
    state.gearState = LEFT;
    break;

case 'R':
case 'T':
    set_car_direction(RIGHT);
    state.gearState = RIGHT;
    break;

case '0':
    PWM_Timer0_Init(SPEED_0);
    state.carSpeed = SPEED_0;
    break;

case '1':
    PWM_Timer0_Init(SPEED_10);
    state.carSpeed = SPEED_10;
    break;
case '2':

```

```

    PWM_Timer0_Init(SPEED_20);
    state.carSpeed = SPEED_20;
    break;

case '3':
    PWM_Timer0_Init(SPEED_30);
    state.carSpeed = SPEED_30;
    break;

case '4':
    PWM_Timer0_Init(SPEED_40);
    state.carSpeed = SPEED_40;
    break;

case '5':
    PWM_Timer0_Init(SPEED_50);
    state.carSpeed = SPEED_50;
    break;

case '6':
    if(u8_speedIsLimited == SPEED_IS_LIMITED)
    {
        PWM_Timer0_Init(SPEED_SAFE_LIMIT);
        state.carSpeed = SPEED_SAFE_LIMIT;
    }
    else
    {
        PWM_Timer0_Init(SPEED_60);
        state.carSpeed = SPEED_60;
    }
    break;

case '7':
    if(u8_speedIsLimited == SPEED_IS_LIMITED)
    {
        PWM_Timer0_Init(SPEED_SAFE_LIMIT);
        state.carSpeed = SPEED_SAFE_LIMIT;
    }
    else
    {
        PWM_Timer0_Init(SPEED_70);
        state.carSpeed = SPEED_70;
    }
    break;

case '8':
    if(u8_speedIsLimited == SPEED_IS_LIMITED)
    {

```

```

        PWM_Timer0_Init(SPEED_SAFE_LIMIT);
        state.carSpeed = SPEED_SAFE_LIMIT;
    }
    else
    {
        PWM_Timer0_Init(SPEED_80);
        state.carSpeed = SPEED_80;
    }
    break;

case '9':
    if(u8_speedIsLimited == SPEED_IS_LIMITED)
    {
        PWM_Timer0_Init(SPEED_SAFE_LIMIT);
        state.carSpeed = SPEED_SAFE_LIMIT;
    }
    else
    {
        PWM_Timer0_Init(SPEED_90);
        state.carSpeed = SPEED_90;
    }
    break;

case 'q':
    if(u8_speedIsLimited == SPEED_IS_LIMITED)
    {
        PWM_Timer0_Init(SPEED_SAFE_LIMIT);
        state.carSpeed = SPEED_SAFE_LIMIT;
    }
    else
    {
        PWM_Timer0_Init(SPEED_100);
        state.carSpeed = SPEED_100;
    }
    break;

case 'X':
case 'x':
    u8_speedIsLimited = SPEED_IS_NOT_LIMITED;
    break;
case 'V':
    if (g_testCase >= 3)
    {
        g_testCase = 0;
    }
    else
    {
        g_testCase++;
    }

```

```

    }

    switch(g_testCase)
    {
    case 0:

        DIO_writePin(INTERRUPT_TESTCASE_PORT2, INTERRUPT_TESTCASE_PIN2,
        LOGIC_LOW);
        break;
    case 1:
        DIO_writePin(INTERRUPT_TESTCASE_PORT1, INTERRUPT_TESTCASE_PIN1,
        LOGIC_LOW);
        DIO_writePin(INTERRUPT_TESTCASE_PORT2, INTERRUPT_TESTCASE_PIN2,
        LOGIC_HIGH);
        break;
    case 2:
        DIO_writePin(INTERRUPT_TESTCASE_PORT1, INTERRUPT_TESTCASE_PIN1,
        LOGIC_HIGH);
        DIO_writePin(INTERRUPT_TESTCASE_PORT2, INTERRUPT_TESTCASE_PIN2,
        LOGIC_LOW);
        break;
    case 3:
        DIO_writePin(INTERRUPT_TESTCASE_PORT1, INTERRUPT_TESTCASE_PIN1,
        LOGIC_HIGH);
        DIO_writePin(INTERRUPT_TESTCASE_PORT2, INTERRUPT_TESTCASE_PIN2,
        LOGIC_HIGH);
        break;
    default:
        break;
    }
    break;
    case 'v':
        break;
    default:
        /*All other symbols are ignored.*/
        break;
    }
    return state;
}

```

[QR Code to the node file](#)



Figure 11: QR Code for the
Moving Control Node Folder

Sensing Control Node

Hardware Design (Circuit Diagram)

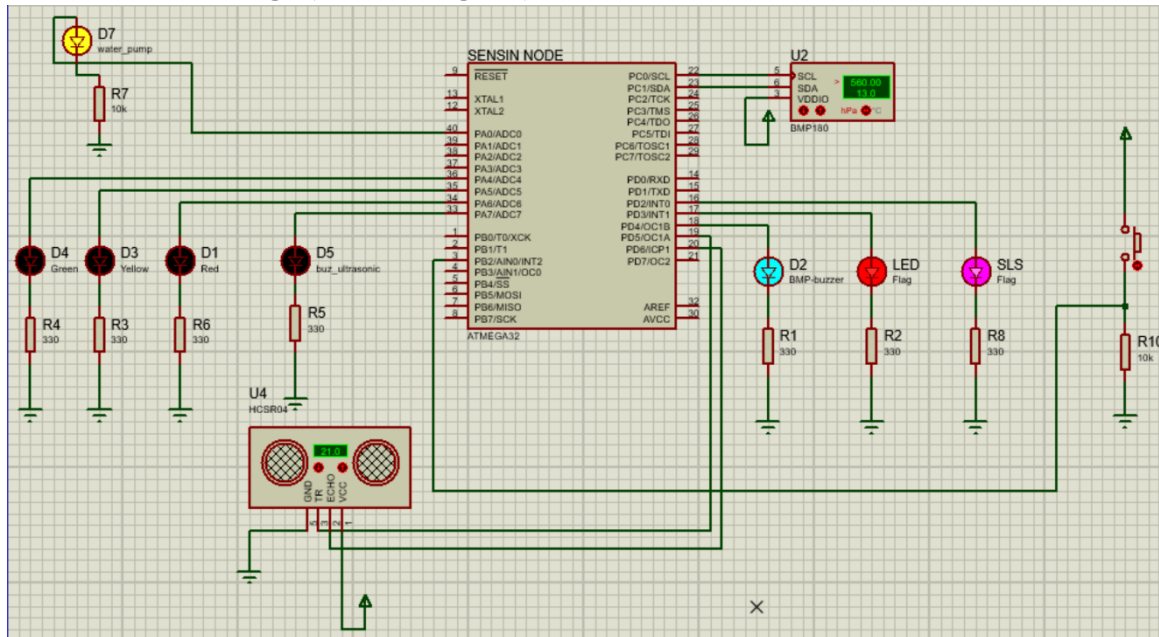


Figure 12: Sensing Control Node Proteus Simulation

Components Description

Systems

- Early Detection System (EDS)
- Steering Lock System (SLS)
- Water Cooling System (WCS)
- Parking Assistance System (PAS)

Overview

Early Detection System

Early Detection System (EDS) is the system responsible for giving a warning to the driver when the pressure and temperature reach a certain range. This range (moderate) is considered safe but close to the severe range. This warning is given to the driver in order to stop the vehicle before it enters the severe state.

Despite its simplicity, this is a very important system in the issue of keeping the passengers safe from the dangers of tire burst, as it gives warning to the driver before the dangers even occurs, so ideally, the passengers are never put in danger due to tire burst.

Steering Lock System

Steering Lock System (SLS) is the system responsible for handling the severe state of the vehicle, stopping it by limiting the driver's control and taking a specific set of actions to stop the vehicle safely.

This is done by the **Sensing Node** sending a signal to **Car Node** when it detects that the vehicle has entered the severe state.

The vehicle takes action when the pressure and temperature reach a severe value, indicating one of the tires is either burst or on the verge of bursting.

Ideally, this system wouldn't be needed, as the driver would change the tires when they are in danger of bursting, thanks to the Early Detection System. But in some fewer common cases, the tires are burst too suddenly, or the driver chose to ignore the warning.

In these cases, the Steering Lock System operates to keep the passengers safe. This is done by first giving the driver a warning that one of the tires burst and allowing them full control.

If the driver is unaware of how to handle this situation and takes an action that would put them in danger, control is taken from them, and the car starts taking a set of actions to keep the driver safe.

This set of action is, to put it simply, keeping the car from steering left or right, and gradually slowing down the car until it stops.

Water Cooling System

Water Cooling System (WCS) is the system responsible for cooling the tires using water to prevent tire burst due to overheating.

This is done by the system constantly sensing the temperature of the wheels, and when the temperature of one of the wheels exceeds a certain value, this system takes action.

The system **sprinkles water** on the wheel in question, until the temperature reading goes down to the range deemed safe. Once the temperature goes back to the safe range, the water sprinkling stops and the system goes back to the normal operation.

Parking Assistance System

Parking Assistance System (PAS) is a system that assists the driver in parking the vehicle more easily. The system measures the distance using sensors placed at multiple ends of the vehicle. If the vehicle is about to bump into another object, one of the sensors detects that and gives a warning to the driver.

The system gives different types of warnings to the driver to indicate how close the vehicle is to the object in question.

Working Principle: **PAS** works using distances measured by the **Ultrasonic** sensor. Excluding PAS, the working principle of all systems is based on the values of pressure and temperature measured by the **BMP180** sensor.

The system then classifies pressure and temperature **individually** into **three ranges for pressure**:

Severe: this range indicates that the wheel are either burst or about to burst, therefore action must be taken to keep the passengers safe.

Moderate: this range indicates that the wheel is still safe but is on the verge of getting into the severe range, hence a warning is given.

Safe: this is the expected range of pressure that indicates the wheel is operating normally.

And three ranges for temperature:

Severe: this range is very high and indicates the wheel is either burst, about to burst, melted or about to melt, in all cases action must be taken to keep the passengers safe.

Moderate: this range indicates the overheating of the wheel, hence action must be taken to rectify that.

Safe: this is the expected range of temperature that indicates the wheel is operating normally.

Water Cooling System operates based on temperature ranges only. However, the other two system work using both pressure and temperature ranges.

In order to handle Steering Lock System (SLS) and Early Detection System (EDS), these ranges are combined to make up **three system states**: Severe, Moderate and Safe.

Severe:

This state is the worst-case scenario. While in this state, the vehicle must be stopped carefully with specific steps or it will lead to accidents. While being in this state, one of the tires are either burst or about to burst. This state is entered when one of the tires reach either the severe range of temperature **or** the severe range of pressure. This state is handled by the **Steering Lock System (SLS)**.

Moderate:

This is the state in which the system gives a warning. While in this state, the vehicle is in danger of entering the severe state, and needs to be slowed down or stopped before it puts the driver in danger.

This state is entered when the tires reach either the moderate range of temperature **or** the moderate range of pressure, as long as **neither of them are in the severe range**. This state is handled by the **Early Detection System (EDS)**.

Safe:

This is the state in which the vehicle is in no danger and no action needs to be taken. This state is entered **if and only if** both temperature **and** pressure are in the safe range. No action is taken by either system in this state.

T State \ P State	Severe	Moderate	Safe
	Severe	Moderate	Safe
Severe	Severe	Severe	Severe
Moderate	Severe	Moderate	Moderate
Safe	Severe	Moderate	Safe

Figure 13: Table describing the states of the tire

Steering Lock System Operation

- Senses Pressure and Temperature.

- Identifies the System State.
- Executes other steps if the System State is **Severe**.
- Turns on LED.
- Turns on Buzzer.
- Sends a signal to the **Car Node** (raises SLS Flag).
- Early Detection System Operation
- Senses Pressure and Temperature.
- Identifies the System State.
- Executes other steps if the System State is **Moderate**.
- Turns on LED.
- Turns on Buzzer.
- Lowers SLS Flag if risen.

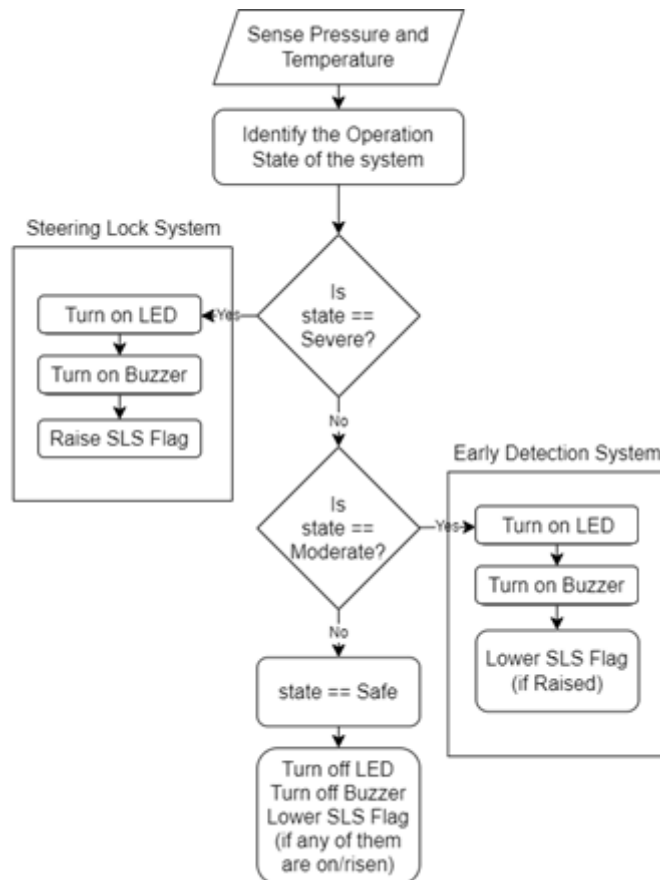


Figure 14: Flow chart showing the working sequence of SLS and EDS

Water Cooling System Operation

- Senses Temperature.
- Identifies the Temperature Range.
- Executes other steps if the Temperature Range is **Severe** or **Moderate**.
- Turns on **Water Pump**.
- Continues sensing Temperature.
- Turns off **Water Pump** when the temperature reaches the **Safe** range.

T State \ P State	Severe	Moderate	Safe
Severe	Severe (Pump on)	Severe (Pump on)	Severe (Pump off)
Moderate	Severe (Pump on)	Moderate (Pump on)	Moderate (Pump off)
Safe	Severe (Pump on)	Moderate (Pump on)	Safe (Pump off)

Figure 15: Table shows the cases in which water cooling system is activated

Parking Assistance System Operation

- Measure Distance.
- Identifies the Distance Range.
- **At Safe Distance**, the system does nothing.
- **At Low-Risk Distance**, the system flashes the **buzzer** and the **green LED** at a **low frequency**.
- **At Moderate-Risk Distance**, the system flashes the **buzzer** and the **yellow LED** at a **medium frequency**.
- **At High-Risk Distance**, the system flashes the **buzzer** and the **red LED** at a **high frequency**.
- **At Extreme Risk Distance**, the system flashes the **buzzer** and the **red LED** at a **very high frequency**.

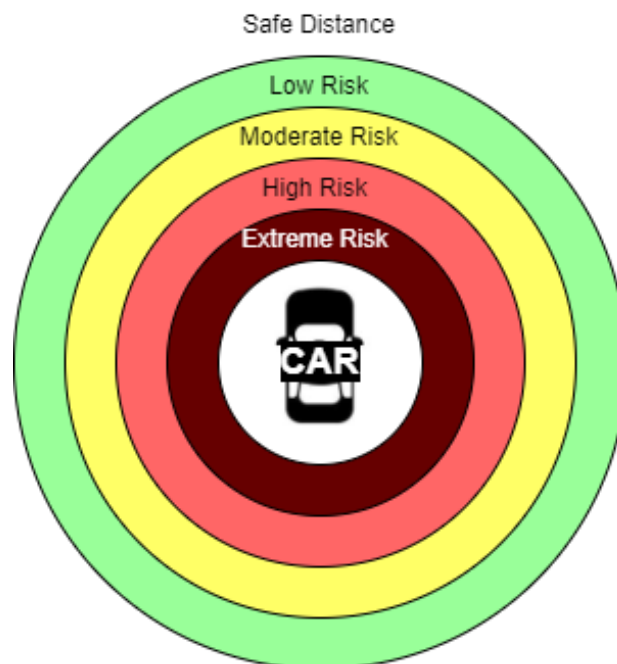


Figure 16: Different distance ranges of the PAS

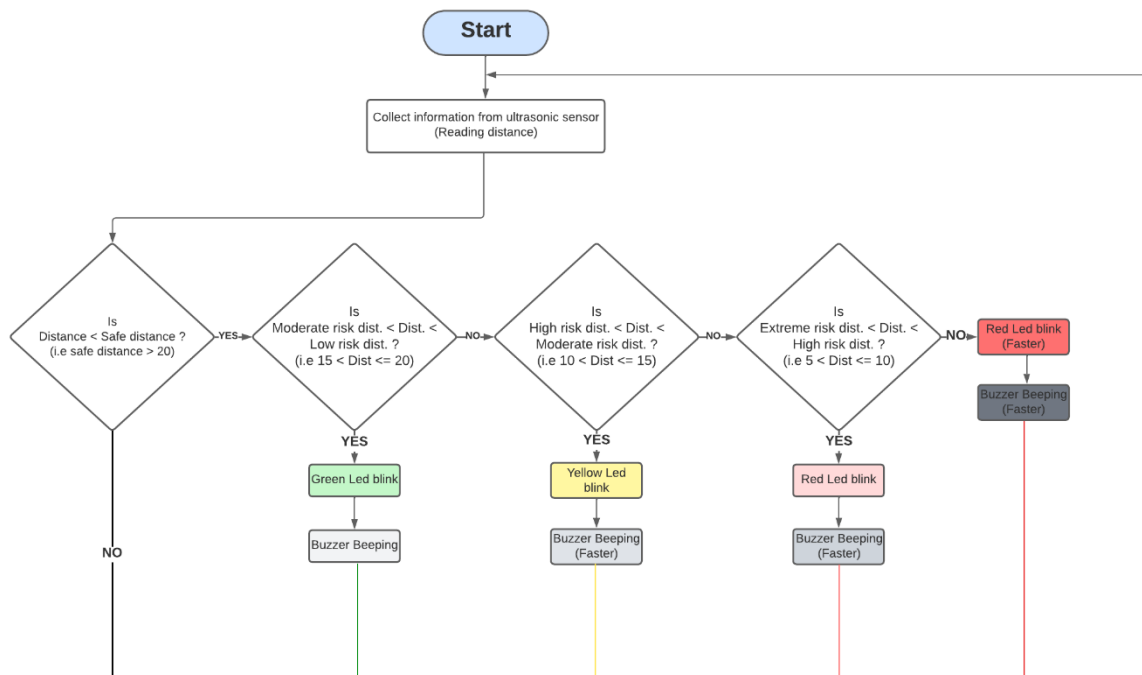


Figure 17: Flow Chart shows how the ultrasonic sensor works

Testing Mode

This is a functionality that will not be in the final product. It's simple enough to test and showcase the different modes of operations of the Parking Assistance System (PAS) as it is possible to physically change the distance between an Ultrasonic sensor and a nearby object. The same can't be said for the other systems, which depend on pressure and temperature.

Pressure and temperature are very difficult to manipulate, and although some difficult methods exist to increase or decrease the temperature, it is even more difficult to control them to specific values, that's why this feature has been implemented.

Testing Mode allows pressure and temperature values to be manipulated in order to test and showcase the operation of the system for all ranges of pressure and temperature.

This is done by pressing a button on the mobile application which moves the car. This button sends a signal to the car node, which sends it to the sensing node using DIO pins. This button to switch between **four different testing cases**:

Default Case

EDS Case

WCS Case

SLS Case

Default Case

This is the state which the final product should be in. System operates as desired, it takes readings from BMP180 sensor and decides what to do based on real readings, without the use of hard code.

EDS Case

This case tests the **Early Detection System** only. This is done by using hard code to give the system a **moderate pressure value** and a **safe temperature value**.

In this case, Water Cooling System is deactivated since the temperature is in the safe range.

WCS Case

This case tests the **Water-Cooling System**. This is done by using hard code to give the system a **safe pressure value** and a **moderate temperature value**.

In this case, Early Detection System also works since the temperature is in a moderate range.

SLS Case

This case tests the **Steering Lock System**. This is done by using hard code to give the system a **severe pressure value** and a **safe temperature value**.

In this case, Water Cooling System is deactivated since the temperature is in the safe range.

Testing Mode Operation

Sense two input pins and awaits output from car node. (2 pins = 4 possible values)

- **If value is 0** (binary 00), execute **Default Case** and take readings from BMP180 sensor.
- **If value is 1** (binary 01), execute **EDS Case** and give the system moderate pressure and safe temperature values.
- **If value is 2** (binary 10), execute **WCS Case** and give the system safe pressure and moderate temperature values.
- **If value is 3** (binary 11), execute **SLS Case** and give the system severe pressure and safe temperature values.

Continue the rest of the code.

Layered Architecture

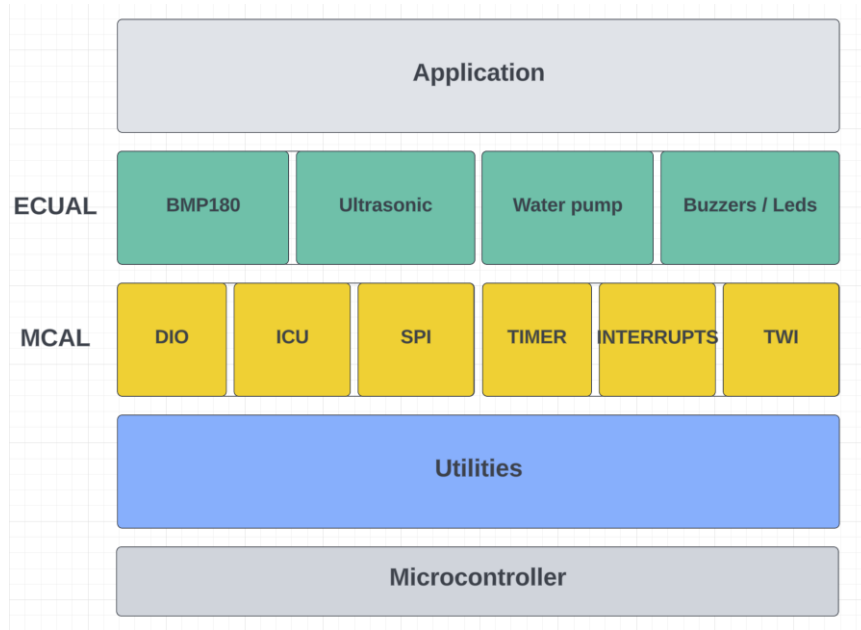


Figure 18: Layered Architecture of Sensing Control Node

Application Code

APP_Early detection

```
/*
*****

Includes

*****

#include <avr/io.h>
#include <avr/interrupt.h>
#include <util/delay.h>
#include "../MCAL/TWI/twi.h"
#include "../MCAL/DIO/dio.h"
#include "../MCAL/TIMER/timer.h"
#include "../ECUAL/BMP/bmp180.h"
#include "../ECUAL/WATERPUMP/waterpump.h"
#include "../ECUAL/ULTRASONIC/ultrasonic.h"
#include "../MCAL/SPI/spi.h"

```

```

#include "../ECUAL/LED/led.h"

#include "../ECUAL/BUZZER/buzzer.h"

#include "../UTILITIES/common_macros.h"

/*****

Enumerations

*****/

/* System states depending on Pressure and Temperature (BMP180) */
typedef enum
{
    SAFE,
    MODERATE,
    SEVERE
} STATE;

/* Ranges for distance (Ultrasonic) */
typedef enum
{
    SAFE_DISTANCE, /*More Than 20 CMs*/
    LOW_RISK_DISTANCE, /*Between 20 and 15 CMs*/
    MODERATE_RISK_DISTANCE, /*Between 15 and 10 CMs*/
    HIGH_RISK_DISTANCE, /*Between 10 and 5 CMs*/
    EXTREME_RISK_DISTANCE /*Less Than or Equal to 5 CMs*/
} t_ULTRASONIC_INTERVAL;

/* Test Cases */

```

```

typedef enum
{
    TEST_DEFAULT,
    TEST_EDS,
    TEST_WCS,
    TEST_SLS
} TEST_CASE;

/*****

Definitions

*****/

/* Pressure and Temperature Ranges (Minimum Value) */
#define PRESSURE_SEVERE      10000 /* Severe Pressure Range: 700 - MAX */
#define PRESSURE_MODERATE    2000 /* Moderate Pressure Range: 500 - 700 */
#define PRESSURE_SAFE        300 /* Safe Pressure Range: 300 - 500 */
#define TEMPERATURE_SEVERE   45 /* Severe Temperature Range: 40 - Max */
#define TEMPERATURE_MODERATE 38 /* Moderate Temperature Range: 20 - 40 */
#define TEMPERATURE_SAFE     0 /* Safe Temperature Range: 0 - 20 */

/* Ultrasonic Ranges */
#define LOW_RISK_DISTANCE_UPPER_LIMIT (20u)
#define LOW_RISK_DISTANCE_LOWER_LIMIT (15u)
#define MODERATE_RISK_DISTANCE_UPPER_LIMIT (15u)
#define MODERATE_RISK_DISTANCE_LOWER_LIMIT (10u)
#define HIGH_RISK_DISTANCE_UPPER_LIMIT (10u)
#define HIGH_RISK_DISTANCE_LOWER_LIMIT (5u)
#define EXTREME_RISK_DISTANCE_UPPER_LIMIT (5u)

*****/

```

Pin Configurations

```

*****/

/* Steering Lock System Flag which is sent to Car Node */

#define SLS_FLAG_PORT_ID      PORTD_ID
#define SLS_FLAG_PIN_ID      PIN2_ID

#define INTERRUPT_ACTIVATION_FLAG_PORT      PORTD_ID
#define INTERRUPT_ACTIVATION_FLAG_PIN      PIN7_ID

/* Early Detection System/Steering Lock System LED and Buzzer */

#define BMP_LED_PORT_ID      PORTD_ID
#define BMP_LED_PIN_ID      PIN3_ID
#define BMP_BUZZER_PORT_ID      PORTD_ID
#define BMP_BUZZER_PIN_ID      PIN5_ID

/* Automatic Parking System LED and Buzzer */

#define WARNING_FRONT_PORT_ID      PORTA_ID
#define WARNING_FRONT_LOW_RISK_LED_PIN_ID      PIN4_ID
#define WARNING_FRONT_MODERATE_RISK_LED_PIN_ID      PIN5_ID
#define WARNING_FRONT_HIGH_RISK_LED_PIN_ID      PIN6_ID
#define WARNING_FRONT_BUZZER_PIN_ID      PIN7_ID

/* Test Cases Pins */

#define INTERRUPT_TESTCASE_PORT1      PORTA_ID      //Will change later
#define INTERRUPT_TESTCASE_PIN1      PIN2_ID      //will change later
#define INTERRUPT_TESTCASE_PORT2      PORTC_ID      //Will change later
#define INTERRUPT_TESTCASE_PIN2      PIN3_ID      //will change later

```

```

/*****

Global variables

*****/

tyreState g_BMP180_readings = {0,0,0}; /* Wheel ID, Pressure and Temperature sent from BMP180 */
uint16 g_distance = 0; /* Measured by Ultrasonic */

STATE g_pressure_state = SAFE;

STATE g_temperature_state = SAFE;

t_ULTRASONIC_INTERVAL ultrasonicInterval = SAFE_DISTANCE;

TEST_CASE g_testCase = TEST_DEFAULT;

unsigned int counter_Bytes = 4;

/*****

Function Declarations

*****/

void timercallBckFunc(void);

void warning(uint16 Distance);

void Parking_LED_init(void);

void Parking_LED_default(void);

void Temp_state(void);

void Press_state(void);

void Action_severe(void);

void Action_moderate(void);

void Action_safe(void);

void Detection_LED_init(void);

```

```

int main()
{
    uint8 send_Byte=0;

    Timer_configuration
    Timer0_Configuration={Timer0,Normal,TIMER_INITIAL_VALUE,NO_COMPARE_VALUE,Prescaler_1024,timercallBckFunc};

    _delay_ms(250);

    TWI_init();

    BMP180_Start();

    Detection_LED_init();

    Parking_LED_init();

    Timer_init(&Timer0_Configuration);

    DIO_setupPinDirection(SLS_FLAG_PORT_ID, SLS_FLAG_PIN_ID, PIN_OUTPUT);

    DIO_writePin(SLS_FLAG_PORT_ID,SLS_FLAG_PIN_ID,LOGIC_HIGH);

    DIO_setupPinDirection(INTERRUPT_TESTCASE_PORT1, INTERRUPT_TESTCASE_PIN1, PIN_INPUT);

    DIO_setupPinDirection(INTERRUPT_TESTCASE_PORT2, INTERRUPT_TESTCASE_PIN2, PIN_INPUT);

    DIO_setupPinDirection(INTERRUPT_ACTIVATION_FLAG_PORT, INTERRUPT_ACTIVATION_FLAG_PIN, PIN_INPUT);

    DIO_writePin(INTERRUPT_ACTIVATION_FLAG_PORT, INTERRUPT_ACTIVATION_FLAG_PIN, LOGIC_HIGH); //pull up resistor

    DDRC |= (1<<2); /*Configure PC2 as Output pin*/

    PORTC |= (1<<2); /*Initiate it to be VCCed*/

```

```

/* Initialize the SPI driver as Master */

SPI_initMaster();

Ultrasonic_init(); /*initializing Ultrasonic*/

WATERPUMP_init();

// INT2_init();

sei();

while(1)
{
    if(!DIO_readPin(INTERRUPT_ACTIVATION_FLAG_PORT,INTERRUPT_ACTIVATION_FL
AG_PIN)){

        PORTC &= ~(1 << 2);

        SPI_sendReceiveByte(FRONT_LEFT); /*Wheel ID*/

        /**/

        for (uint8 i = 0; i < counter_Bytes; i++) {

            send_Byte = (g_BMP180_readings.temperature >> (i * 8));

            SPI_sendReceiveByte(send_Byte);

        }

        for (uint8 i = 0; i < counter_Bytes; i++) {

            send_Byte = (g_BMP180_readings.pressure >> (i * 8));

            SPI_sendReceiveByte(send_Byte);

        }

    }

    else

    {

```

```

PORTC |= (1 << 2);

Test_Case_Config();

switch(g_testCase)
{
case TEST_DEFAULT:

    BMP180_calculate(&g_BMP180_readings);

    break;

case TEST_EDS:

    g_BMP180_readings.temperature = 25;

    g_BMP180_readings.pressure = 3000;

    Parking_LED_default();

    BUZZER_OFF(WARNING_FRONT_PORT_ID,WARNING_FRONT_BUZZER_PIN_ID);

    break;

case TEST_WCS:

    g_BMP180_readings.temperature = 40;

    g_BMP180_readings.pressure = 900;

    Parking_LED_default();

    BUZZER_OFF(WARNING_FRONT_PORT_ID,WARNING_FRONT_BUZZER_PIN_ID);

    break;

case TEST_SLS:

    g_BMP180_readings.temperature = 25;

    g_BMP180_readings.pressure = 12000;

    Parking_LED_default();

    BUZZER_OFF(WARNING_FRONT_PORT_ID,WARNING_FRONT_BUZZER_PIN_ID);

    break;

}

```

```

    Temp_state();

    Press_state();

    if (g_pressure_state == SEVERE || g_temperature_state == SEVERE)
    {
        Action_severe();
    }

    else if (g_pressure_state == MODERATE || g_temperature_state == MODERATE)
    {
        Action_moderate();
    }

    else if (g_pressure_state == SAFE || g_temperature_state == SAFE)
    {
        Action_safe();
    }

    else
    {
        BUZZER_OFF(BMP_BUZZER_PORT_ID,BMP_BUZZER_PIN_ID);

        WATERPUMP_off();
    }

    g_distance = Ultrasonic_readDistance(); /*storing the measured distance*/

    warning(g_distance);

    //    _delay_ms(100);

}

}

```

```
return 0;

}
```

Timer callback function

```
void timercallBckFunc(void)
{
    static uint8 counter=0;

    switch(ultrasonicInterval)
    {
        case LOW_RISK_DISTANCE:
            //we need to make the buzzer on for 400ms and off for 100ms
            //to make it on for 100ms we need the timer to overflow 6 times
            //to make it off for 100ms we need the timer to overflow 6 times
            //so the buzzer will toggle when counter is 24

            Parking_LED_default();

            // Buzzer_on(WARNING_FRONT_PORT_ID,WARNING_FRONT_BUZZER_PIN_ID);
            // LED_ON(WARNING_FRONT_PORT_ID,WARNING_FRONT_LOW_RISK_LED_PIN_ID);

            // if(counter > 24)
            // {
            //     counter = 0;
            // }

            if(counter == 24)
            {
                BUZZER_TOGGLE(WARNING_FRONT_PORT_ID,WARNING_FRONT_BUZZER_PIN_ID
            );
            }
        }
    }
}
```

```

        LED_TOGGLE(WARNING_FRONT_PORT_ID,WARNING_FRONT_LOW_RISK_LED_PIN_ID);

        counter = 0;

    }

    else

    {

        counter++;

    }

    break;

case MODERATE_RISK_DISTANCE:

    //we need to make the buzzer toggle each 300ms

    //so it will toggle when counter is 18

    Parking_LED_default();

    // Buzzer_on(WARNING_FRONT_PORT_ID,WARNING_FRONT_BUZZER_PIN_ID);

    // LED_ON(WARNING_FRONT_PORT_ID,WARNING_FRONT_MODERATE_RISK_LED_PIN_ID);

    if(counter > 18)

    {

        counter = 0;

    }

    if(counter == 18)

    {

        BUZZER_TOGGLE(WARNING_FRONT_PORT_ID,WARNING_FRONT_BUZZER_PIN_ID);

        LED_TOGGLE(WARNING_FRONT_PORT_ID,WARNING_FRONT_MODERATE_RISK_LED_PIN_ID);

        counter = 0;

```

```

    }

    else

    {

        counter++;

    }

    break;

case HIGH_RISK_DISTANCE:

    //we need to make the buzzer toggle each 200ms

    //so it will toggle when counter is 12

    Parking_LED_default();

    // Buzzer_on(WARNING_FRONT_PORT_ID,WARNING_FRONT_BUZZER_PIN_ID);

    // LED_ON(WARNING_FRONT_PORT_ID,WARNING_FRONT_HIGH_RISK_LED_PIN_ID);

    if(counter > 12)

    {

        counter = 0;

    }

    if(counter == 12)

    {

        BUZZER_TOGGLE(WARNING_FRONT_PORT_ID,WARNING_FRONT_BUZZER_PIN_ID
);
        LED_TOGGLE(WARNING_FRONT_PORT_ID,WARNING_FRONT_HIGH_RISK_LED_PIN
_ID);

        counter = 0;

    }

    else

    {

        counter++;

    }

```

```

    }

    break;

case EXTREME_RISK_DISTANCE:

    //we need to make the buzzer toggle each 100ms

    //so it will toggle when counter is 6

    Parking_LED_default();

    // Buzzer_on(WARNING_FRONT_PORT_ID,WARNING_FRONT_BUZZER_PIN_ID);

    // LED_ON(WARNING_FRONT_PORT_ID,WARNING_FRONT_HIGH_RISK_LED_PIN_ID);


    if(counter > 6)

    {

        counter = 0;

    }

    if(counter == 6)

    {

        BUZZER_TOGGLE(WARNING_FRONT_PORT_ID,WARNING_FRONT_BUZZER_PIN_ID
);

        LED_TOGGLE(WARNING_FRONT_PORT_ID,WARNING_FRONT_HIGH_RISK_LED_PI
N_ID);

        counter = 0;

    }

    else

    {

        counter++;

    }

    break;

```

default:

```
Parking_LED_default();

BUZZER_OFF(WARNING_FRONT_PORT_ID,WARNING_FRONT_BUZZER_PIN_ID);

counter = 0;

break;

}

}
```

Warning function

```
void warning(uint16 Distance)

{

    if(Distance <= LOW_RISK_DISTANCE_UPPER_LIMIT && Distance >
LOW_RISK_DISTANCE_LOWER_LIMIT)

    {

        ultrasonicInterval = LOW_RISK_DISTANCE;

        Timer_reset(Timer0);

    }

    else if(Distance <= MODERATE_RISK_DISTANCE_UPPER_LIMIT && Distance >
MODERATE_RISK_DISTANCE_LOWER_LIMIT)

    {

        ultrasonicInterval = MODERATE_RISK_DISTANCE;

        Timer_reset(Timer0);

    }

    else if(Distance <= HIGH_RISK_DISTANCE_UPPER_LIMIT && Distance >
HIGH_RISK_DISTANCE_LOWER_LIMIT)

    {

        ultrasonicInterval = HIGH_RISK_DISTANCE;

        Timer_reset(Timer0);

    }

}
```

```

else if(Distance <= EXTREME_RISK_DISTANCE_UPPER_LIMIT)
{
    ultrasonicInterval = EXTREME_RISK_DISTANCE;
    Timer_reset(Timer0);
}
else
{
    ultrasonicInterval = SAFE_DISTANCE;
    Timer_reset(Timer0);
}
}

```

LEDs

```

void Parking_LED_init(void)
{
    LED_init(WARNING_FRONT_PORT_ID,WARNING_FRONT_LOW_RISK_LED_PIN_ID);
    LED_init(WARNING_FRONT_PORT_ID,WARNING_FRONT_MODERATE_RISK_LED_PIN_ID);
    LED_init(WARNING_FRONT_PORT_ID,WARNING_FRONT_HIGH_RISK_LED_PIN_ID);
    BUZZER_init(WARNING_FRONT_PORT_ID,WARNING_FRONT_BUZZER_PIN_ID);
}

void Parking_LED_default(void)
{
    LED_OFF(WARNING_FRONT_PORT_ID,WARNING_FRONT_LOW_RISK_LED_PIN_ID);
}

```

```

    LED_OFF(WARNING_FRONT_PORT_ID,WARNING_FRONT_MODERATE_RISK_LED_PIN_ID);

    LED_OFF(WARNING_FRONT_PORT_ID,WARNING_FRONT_HIGH_RISK_LED_PIN_ID);
}

```

Pressure and temperature state

```

void Temp_state(void){

    if (g_BMP180_readings.temperature >= TEMPERATURE_SEVERE)

    {

        g_temperature_state = SEVERE;

    }

    else if ((g_BMP180_readings.temperature >= TEMPERATURE_MODERATE) &&
(g_BMP180_readings.temperature < TEMPERATURE_SEVERE))

    {

        g_temperature_state = MODERATE;

    }

    else if ((g_BMP180_readings.temperature >= TEMPERATURE_SAFE) &&
(g_BMP180_readings.temperature < TEMPERATURE_MODERATE))

    {

        g_temperature_state = SAFE;

    }

}

void Press_state(void)

{

    if (g_BMP180_readings.pressure >= PRESSURE_SEVERE)

    {

        g_pressure_state = SEVERE;

    }

}

```

```
    else if ((g_BMP180_readings.pressure >= PRESSURE_MODERATE) &&
(g_BMP180_readings.pressure < PRESSURE_SEVERE))

    {

        g_pressure_state = MODERATE;

    }

    else if ((g_BMP180_readings.pressure >= PRESSURE_SAFE) && (g_BMP180_readings.pressure <
PRESSURE_MODERATE))

    {

        g_pressure_state = SAFE;

    }

}
```

Action types

```
void Action_severe(void)
{
    BUZZER_ON(BMP_BUZZER_PORT_ID,BMP_BUZZER_PIN_ID);
    LED_ON(BMP_LED_PORT_ID,BMP_LED_PIN_ID);
    if (g_temperature_state == SEVERE || g_temperature_state == MODERATE)
    {
        WATERPUMP_on();
    }
    else
    {
        WATERPUMP_off();
    }
    DIO_writePin(SLS_FLAG_PORT_ID, SLS_FLAG_PIN_ID, LOGIC_LOW);
    // _delay_ms(5);
    // DIO_writePin(SLS_FLAG_PORT_ID, SLS_FLAG_PIN_ID, LOGIC_LOW);
}

void Action_moderate(void)
{
    BUZZER_ON(BMP_BUZZER_PORT_ID,BMP_BUZZER_PIN_ID);
    LED_ON(BMP_LED_PORT_ID,BMP_LED_PIN_ID);
    if (g_temperature_state == SEVERE || g_temperature_state == MODERATE)
    {
        WATERPUMP_on();
    }
    else
    {
```

```
    WATERPUMP_off();

}

DIO_writePin(SLS_FLAG_PORT_ID, SLS_FLAG_PIN_ID, LOGIC_HIGH);
}

void Action_safe(void)
{
    BUZZER_OFF(BMP_BUZZER_PORT_ID,BMP_BUZZER_PIN_ID);
    LED_OFF(BMP_LED_PORT_ID,BMP_LED_PIN_ID);
    WATERPUMP_off();
    DIO_writePin(SLS_FLAG_PORT_ID, SLS_FLAG_PIN_ID, LOGIC_HIGH);
}
```

[QR Code to the node file](#)



*Figure 19: QR Code for the
Sensing Control Node Folder*

IoT Node Hardware Design (Circuit Diagram)

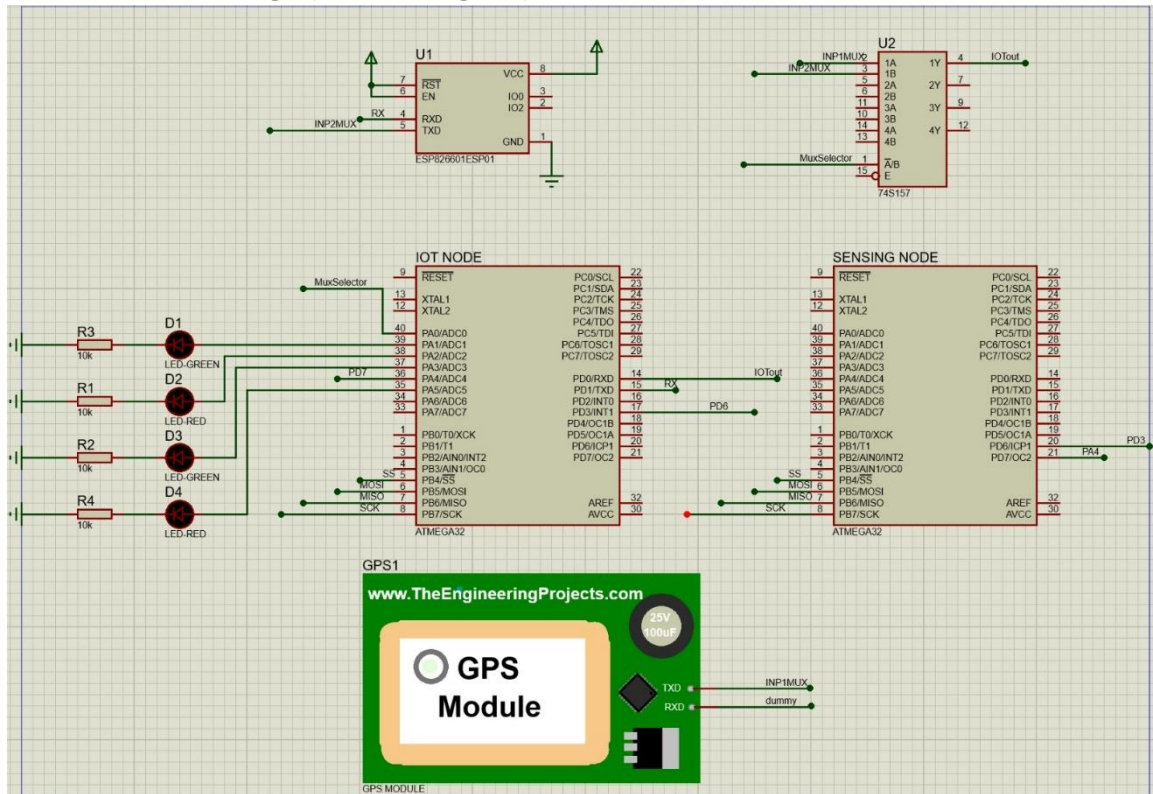


Figure 20: IoT Node Proteuos Simulation

Components Description

The IoT system described above integrates an ESP-01 WiFi module, a u-blox NEO-6 GPS module, a BMP sensor, and various communication protocols such as UART and SPI. It effectively collects GPS coordinates and BMP sensor readings, multiplexes UART communication, and establishes a connection with a server to transmit data.

An ESP-01 WiFi module to establish a connection with a server for transmitting data. The system collects and sends various types of data, including GPS coordinates and readings from a BMP sensor, to the server.

The system incorporates a GPS module, specifically the u-blox NEO-6, which is installed on the same node. The GPS module is responsible for acquiring the precise coordinates of the car's location. These GPS coordinates are then included in the data sent to the server.

Additionally, the system employs an SPI (Serial Peripheral Interface) to receive the BMP sensor readings from the sensor node. The BMP sensor provides information such as temperature and pressure. The SPI facilitates efficient and reliable communication between the microcontroller and the sensor node, enabling the retrieval of BMP sensor readings.

UART is also utilized in the system to facilitate bidirectional communication between the microcontroller and both the GPS module and the ESP module. The microcontroller, acting as a central control unit, exchanges data with the GPS module to receive location coordinates and sends commands or data to the ESP module for server communication.

To prevent conflicts between the GPS module and the ESP module, a 2x1 multiplexer (mux) is employed. This multiplexer selectively routes the UART (Universal Asynchronous Receiver-Transmitter) communication between the GPS module and the ESP module. By multiplexing the UART, the system ensures that both the GPS module and the ESP module can utilize the UART without interfering with each other's functionality.

GPS_sendingCoordinatesTask

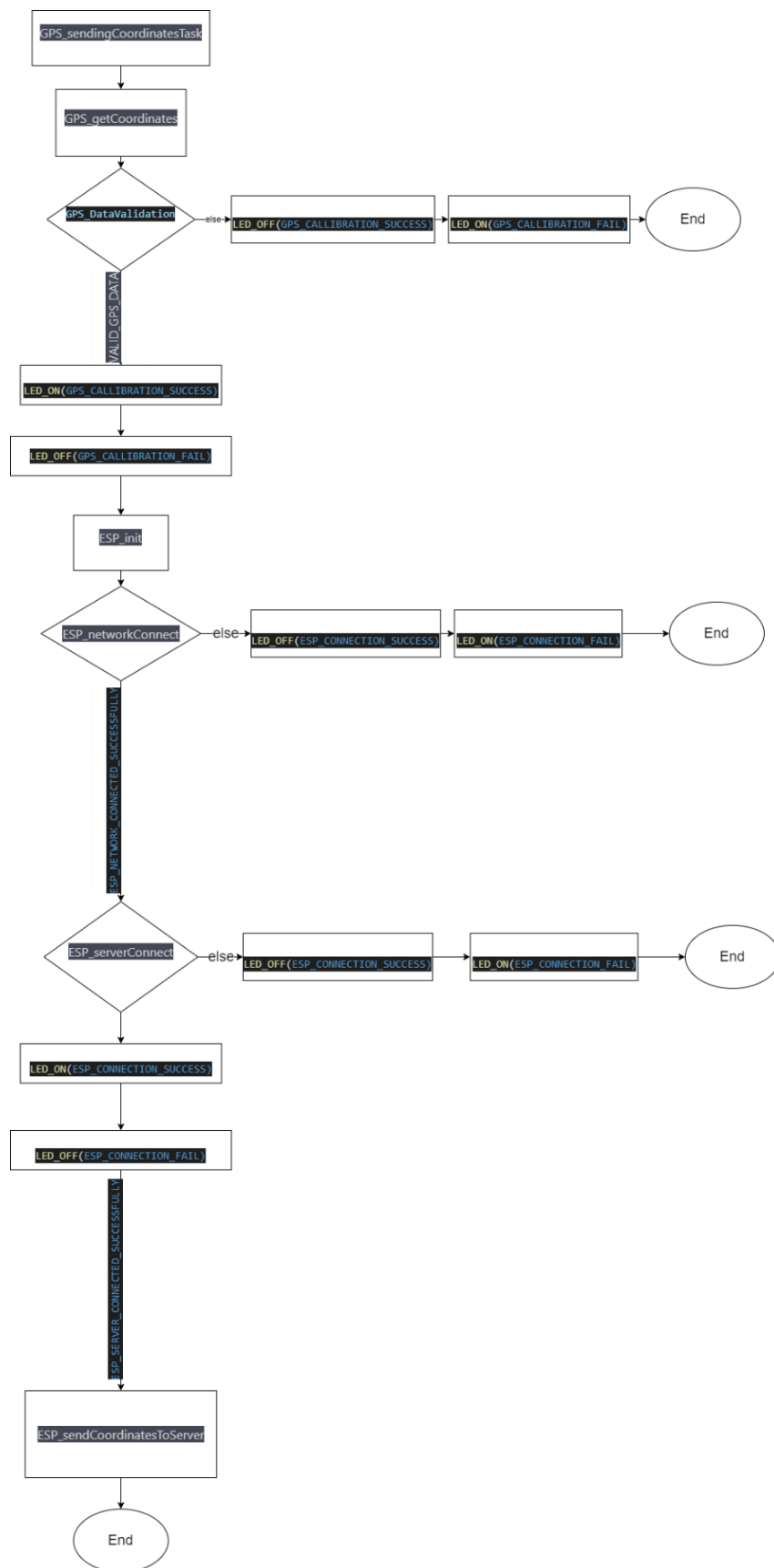


Figure 21: GPS_sendingCoordinatesTask Flow Chart

BMP180_sendingDataTask

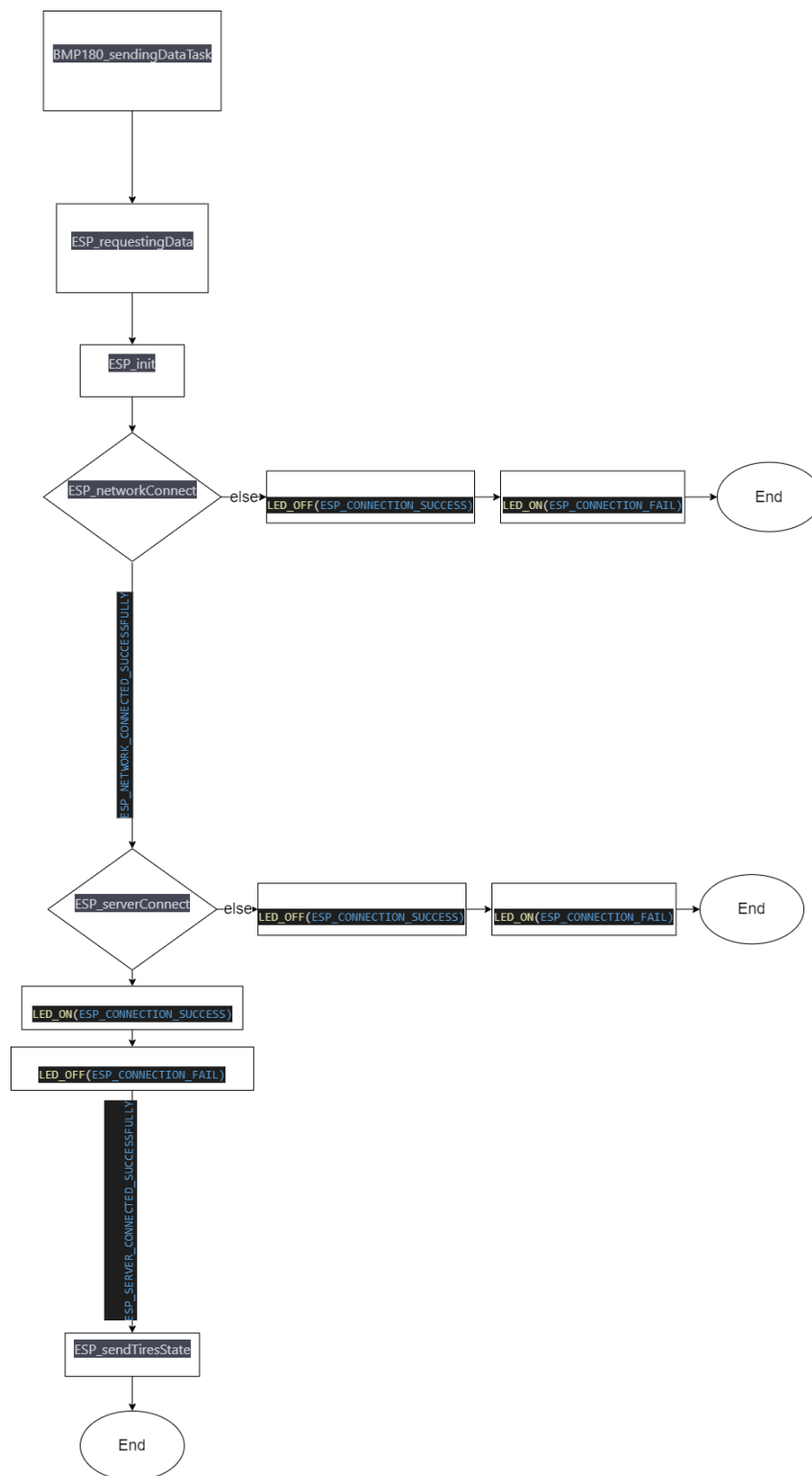


Figure 22: BMP180_sendingDataTask Flow Chart

Interaction with the other node

Initialization (app_Init)

The `app_Init` function is responsible for initializing the system components and setting up the necessary configurations. It includes delays to allow for proper initialization of the system. LED indicators for different tasks are initialized to visually indicate the state of each task. The GPS module and ESP module are initialized, and interrupts are enabled to handle interrupts during runtime.

Starting the Application (app_Start)

The `app_Start` function serves as the entry point of the application. It calls two tasks, `GPS_sendingCoordinatesTask` and `BMP180_sendingDataTask`, which are responsible for collecting and transmitting GPS coordinates and BMP sensor data, respectively.

GPS Sending Coordinates Task (GPS_sendingCoordinatesTask)

The `GPS_sendingCoordinatesTask` function retrieves GPS coordinates and transmits them to the server. It begins by reinitializing the GPS module to ensure accurate and fresh data. It then calls `GPS_getCoordinates` to obtain the GPS coordinates. If the data validation is successful (valid GPS data is received), the system proceeds to initialize the ESP module. The ESP module is used to establish a network connection with the specified SSID and password using `ESP_networkConnect`. If the network connection is successful, the system attempts to connect to the server using `ESP_serverConnect`. If the server connection is established, the GPS coordinates are sent to the server using `ESP_sendCoordinatesToServer`. LED indicators are updated accordingly to indicate the success or failure of each step.

BMP180 Sending Data Task (BMP180_sendingDataTask)

The `BMP180_sendingDataTask` function requests and transmits BMP sensor data to the server. It begins by requesting data from the ESP module using `ESP_requestingData`. It then initializes the ESP module and establishes a network connection similar to the GPS task. If the network connection and server connection are successful, the system sends the tire state data to the server using `ESP_sendTiresState`. LED indicators are updated accordingly to indicate the success or failure of each step.

In terms of behavior, the code follows a sequential execution flow. It first initializes the system components, including LEDs, GPS module, and ESP module. Then, it starts the application by calling the GPS and BMP sensor tasks. These tasks perform their respective functionalities, such as retrieving GPS coordinates and BMP sensor readings, establishing network connections, and transmitting data to the server. The LED indicators provide visual feedback on the status of each task and the overall system operation. If any step encounters an error or failure, the corresponding LED indicator is updated to reflect the failure, allowing for easy troubleshooting and monitoring.

Layered Architecture

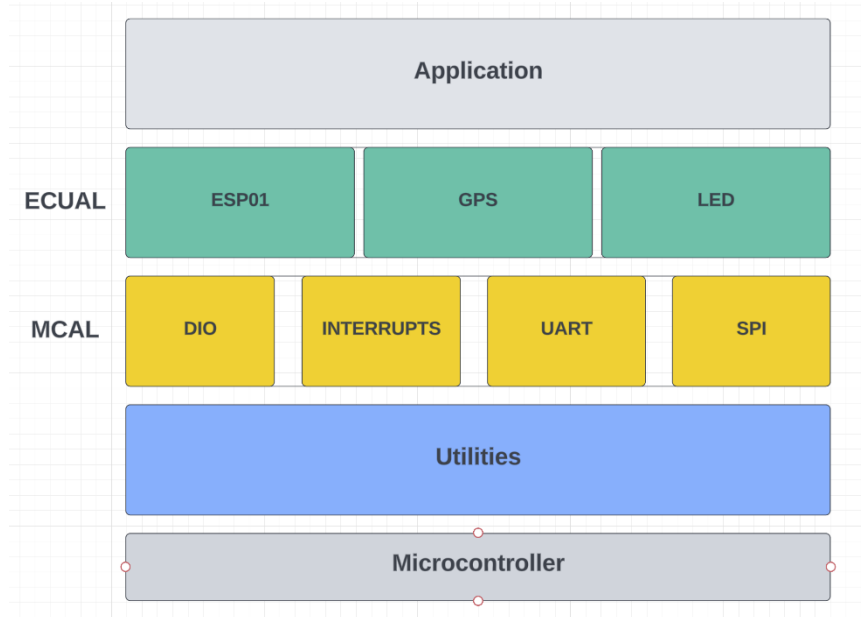


Figure 23: Layered Architecture for IoT Node

Application Code

app.h

```
#ifndef APPLICATION_APP_H_
#define APPLICATION_APP_H_

/*****

Includes

*****/

#include "../ECUAL/LED/led.h"
#include "../ECUAL/GPS/gps.h"
#include "../ECUAL/ESP01/esp.h"

/*****

Definitions

*****/

#define SSID (const uint8*) "Mahmoud"
#define PASSWORD (const uint8*) "12345678"
#define THINGSPEAK_SERVER_IP (const uint8*) "184.106.153.149"
#define THINGSPEAK_SERVER_URL (const uint8*) "api.thingspeak.com"
```

```

#define SW_TEAM_SERVER_IP (const uint8*) "34.122.187.191"

#define IP (const uint8*) "34.122.187.191"

#define PORT (const uint8*) "80"

#define API_KEY (const uint8*) "E3F314CY3VJGPB74"

#define CAR_ID (const uint8*) "63d5343990599fb5c8ce4c48"

#define GPS_CALLIBRATION_SUCCEED 0

#define GPS_CALLIBRATION_FAILED 1

#define ESP_CONNECTION_SUCCEED 0

#define ESP_CONNECTION_FAILED 1

#define ESP_CONNECTION_SUCCESS_INDICATION_PORT PORTA_ID

#define ESP_CONNECTION_SUCCESS_INDICATION_PIN PIN1_ID

#define ESP_CONNECTION_FAIL_INDICATION_PORT PORTA_ID

#define ESP_CONNECTION_FAIL_INDICATION_PIN PIN2_ID

#define GPS_CALLIBRATION_SUCCESS_INDICATION_PORT PORTA_ID

#define GPS_CALLIBRATION_SUCCESS_INDICATION_PIN PIN3_ID

#define GPS_CALLIBRATION_FAIL_INDICATION_PORT PORTA_ID

#define GPS_CALLIBRATION_FAIL_INDICATION_PIN PIN5_ID

/*****

        Functions Prototypes

*****/

void app_Init(void);

void app_Start(void);

void GPS_sendingCoordinatesTask(void);

void BMP180_sendingDataTask(void);

#endif /* APPLICATION_APP_H */

```

app.c

```

void app_Init(void)

```

```

{

    _delay_ms(250);

    _delay_ms(250);

    /*LEDs Initialization to Indicate the state of each task*/

    LED_init(ESP_CONNECTION_SUCCESS_INDICATION_PORT,ESP_CONNECTION_SUCCESS_INDICATION_PIN);

    LED_init(ESP_CONNECTION_FAIL_INDICATION_PORT,ESP_CONNECTION_FAIL_INDICATION_PIN);

    LED_init(GPS_CALLIBRATION_SUCCESS_INDICATION_PORT,
    GPS_CALLIBRATION_SUCCESS_INDICATION_PIN);

    LED_init(GPS_CALLIBRATION_FAIL_INDICATION_PORT,
    GPS_CALLIBRATION_FAIL_INDICATION_PIN);

    GPS_init();

    ESP_preInit();

    sei();

}

```

```

void GPS_sendingCoordinatesTask(void)

{

    GPS_reInit();

    GPS_DataValidation = GPS_getCoordinates(&t_GPS_Coordinates);

    if(GPS_DataValidation == VALID_GPS_DATA)

    {

        LED_ON(GPS_CALLIBRATION_SUCCESS_INDICATION_PORT,GPS_CALLIBRATION_SUCCESS_INDICATION_PIN);

        LED_OFF(GPS_CALLIBRATION_FAIL_INDICATION_PORT,GPS_CALLIBRATION_FAIL_INDICATION_PIN);

        ESP_init()

        if(ESP_networkConnect(SSID, PASSWORD))

        {

            if(ESP_serverConnect(SW_TEAM_SERVER_IP, PORT))

```

```

    {

ESP_sendCoordinatesToServer(CAR_ID,t_GPS_Coordinates.Longitude,t_GPS_Coordinates.Latitude);

LED_ON(ESP_CONNECTION_SUCCESS_INDICATION_PORT,ESP_CONNECTION_SUCCESS_INDICATION_PIN);

LED_OFF(ESP_CONNECTION_FAIL_INDICATION_PORT,ESP_CONNECTION_FAIL_INDICATION_PIN);

    }

    else

    {

        LED_OFF(ESP_CONNECTION_SUCCESS_INDICATION_PORT,ESP_CONNECTION_SUCCESS_INDICATION_PIN);

        LED_ON(ESP_CONNECTION_FAIL_INDICATION_PORT,ESP_CONNECTION_FAIL_INDICATION_PIN);

    }

    } else

    { LED_OFF(ESP_CONNECTION_SUCCESS_INDICATION_PORT,ESP_CONNECTION_SUCCESS_INDICATION_PIN);

        LED_ON(ESP_CONNECTION_FAIL_INDICATION_PORT,ESP_CONNECTION_FAIL_INDICATION_PIN); }

    }

    else

    {

        LED_OFF(GPS_CALLIBRATION_SUCCESS_INDICATION_PORT,GPS_CALLIBRATION_SUCCESS_INDICATION_PIN);

        LED_ON(GPS_CALLIBRATION_FAIL_INDICATION_PORT,GPS_CALLIBRATION_FAIL_INDICATION_PIN);

    }

    }}

```

```

void app_Start(void){

    GPS_sendingCoordinatesTask();

    BMP180_sendingDataTask();}

```

```

void BMP180_sendingDataTask(void){

```

```

ESP_requestingData();

ESP_init();

if(ESP_networkConnect(SSID, PASSWORD) {

    if(ESP_serverConnect(SW_TEAM_SERVER_IP, PORT)) {

        ESP_sendTiresState(CAR_ID);

        LED_ON(ESP_CONNECTION_SUCCESS_INDICATION_PORT,ESP_CONNECTION_SUCCESS_INDICATIO
N_PIN);

        LED_OFF(ESP_CONNECTION_FAIL_INDICATION_PORT,ESP_CONNECTION_FAIL_INDICATION_PIN);

    }

    else {

        LED_OFF(ESP_CONNECTION_SUCCESS_INDICATION_PORT,ESP_CONNECTION_SUCCESS_INDICATIO
N_PIN);

        LED_ON(ESP_CONNECTION_FAIL_INDICATION_PORT,ESP_CONNECTION_FAIL_INDICATION_PIN);

    }

}

else {

    LED_OFF(ESP_CONNECTION_SUCCESS_INDICATION_PORT,ESP_CONNECTION_SUCCESS_INDICATIO
N_PIN);

    LED_ON(ESP_CONNECTION_FAIL_INDICATION_PORT,ESP_CONNECTION_FAIL_INDICATION_PIN);

}

}

```

QR Code to the node file



*Figure 24: QR Code for the
IoT Node Folder*

Mobile Application

App.js

App.js is the most important file in the mobile application, it is the root of the program and what it returns is the thing that is shown on the screen of the mobile.

We will talk about what imports this file has, what it returns, the name of the component it creates, the functions it uses and the exports it will do in the end. We will go with this format through the application part of the book/documentation.

First, we will talk about imports:

- 1) It imports react from react and SafeAreaView, ScrollView, StyleSheet, Text from react native.
- 2) It imports Navigation which contain the flow of the whole app.
- 3) It imports Amplify and config to use AWS amplify authentication and configure it.
- 4) It imports Provider and store to use redux in the application to be able to store status and variables in it and use it in all the screens.

Secondly, the App component is created then it returns Provider component which has a store object, and it wraps the whole application. Then the Navigation component which has the order of screen in it and uses stack navigation and bottom tab navigation is returned after being wrapped by a SafeAreaView component.

Then lastly, we export the App component and before that amplify is configured and a style object is created that contains a root element used in styling the SafeAreaView component.

Src folder:

This file contains nearly all our written code in this application it contains 4 folders which we will talk about each of them later these folders are: (components, constants, navigation, and screens). The next diagram shows the structure of this folder.

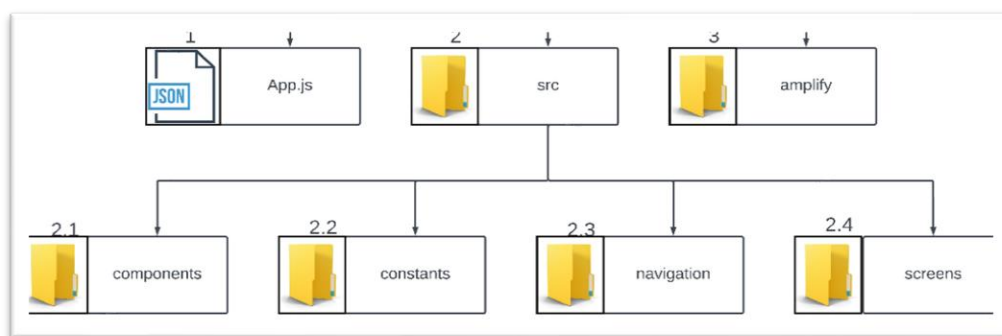


Figure 25: Src Folder

Components folder

Now we will talk about the sub folders of src folder which has the entirety of the code we had written, and we will start with the component folder.

This folder contains 9 sub folders, each of them is a component that is used in other components and screens in the application. These subfolders/components are: (Card, CustomButton, CustomGauge, CustomInput, ProgressBar, SocialSignInButtons, redux, socket).

Each sub folder of these folders contains one or two files of JavaScript which contains the code in it.

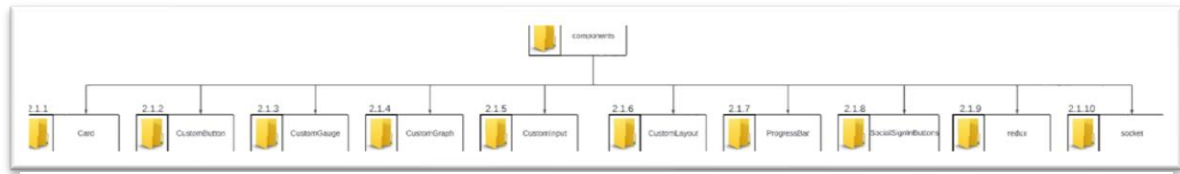


Figure 26: Components Folder

Card folder:

This folder contains a JavaScript file named Card.js which we used to make cards like custom buttons which take title, color and onPress as props to make this button customizable and easy to use in other components.

Card.js:

This file is not currently used in our application, but we used it in creating buttons with a certain style which we decided not to do but kept it in the code in case we use it again in the future.

First, we will talk about the imports:

- 1) React is imported from react.
- 2) TouchableOpacity, Text, StyleSheet is imported from react native.

Secondly, the Card component is created which takes in 3 props color, title and onPress which helps in making the component usable in many scenarios and helps to re-use it in different places in the code. It returns a text component which is wrapped in between the component TouchableOpacity.

Then lastly, the styles object is created, and it has card and title key-pairs or styles elements that are used in components used in the return mentioned above. The file exports Card component to be used in other files.

CustomButton folder:

This folder contains two files which are CustomButton.js and index.js the first is for the component code which is the CustomButton component and the other index.js which helps in the ease of importing the component to other files where we are not obligated to write the file name in the path when importing the component else whereas it goes directly to index.js and then to the component.

CustomButton.js:

This file creates a custom button which can be used in other files and components in the project. The component takes five props which are: (onPress, text, type, bgColor, fgColor) where onPress: is a function that tells the button what to do when pressed,

text: is the text on the button, type: there are three types a button could be primary secondary and tertiary where each type has its own styles for container and text, bgColor: background color and fgColor: foreground color.

First: we will talk about the imports for this file:

- 1) React from react.
- 2) View, Text, StyleSheet, Pressable from react-native.
- 3) Colors from react-native libraries.

Secondly, we will talk about the return of this file. It returns a text component which contains text and styles using the props of the component it sets the text and styles values. This text component is wrapped in between a Pressable component which contain onPress and styles, where their values are taken from the component props Aswell to increase the usability of the component.

Lastly, we have styles which are 3 types for each button type which are primary, secondary, and tertiary. Then the component is exported to be used in other files.

CustomGauge folder:

This folder contains a file called CustomGauge.js where it is used to create the gauges needed in the temperature and pressure screens in the application.

CustomGauge.js:

This file creates a custom gauge component which is used in different screens like the temperature and pressure screens, and it has two props which are (value, max_value) where the value is the current reading of temperature or pressure taken by the car and added as a prop to change it in the component. While the max_value is taken as a prop to change the maximum value of a tire. There is no min_value as this prop is manually set to zero in the code as a design decision.

First, we will talk about the imports:

- 1) React from react.
- 2) View, Text, StyleSheet, Pressable from react-native.
- 3) Colors from react-native libraries.

Secondly, we will talk about the return of this CustomGauge component. This component returns a RNSpeedometer which has six props 2 of them are taken from the main component props which are value and max_value. Meanwhile the six props that the RNSpeedometer has are: (value, size, min_value, max_value, allowedDecimals, and labels). This RNSpeedometer component is wrapped in between a view component which is further wrapped in between a SafeAreaView component.

Lastly, we have styles object created which is used in styling the CustomGauge component in the way we want it. Then the CustomGauge component is exported to be used in other files in the project.

CustomInput folder:

This folder contains two files the CustomInput.js and the index.js the custom input component is used as a data entry with some modifications according to the values of the props which can be changed in other components to customize the same component with different values that helps in the reuse of code and to avoid the redundancy of the code. While the index file is used to make importing to other files easier as mentioned above.

CustomInput.js:

This component is a custom input component which is used in screens like Sign in, sign up and new password screens. It is highly customizable and can be modified depending on its usage. This component has five props which are: (control, name, rules, placeholder, secureTextEntry,). The control prop is used to pass the form control instance from react-hook-form to the Controller component. The name prop is used as an identification for the value of the custom input component input. Rules prop is like an object which contains a lot of elements that define what can or cannot go in the custom input like the maximum length allowed, the minimum length allowed or simple validations. The placeholder prop is the name of element shown to the user. And secureTextEntry is used to change the words to star signs when writing the password.

First, we will talk about the imports:

- 1) Import React from react.
- 2) View, Text, TextInput, and StyleSheet are imported from react-native.
- 3) Controller is imported from react-hook-form.
- 4) Color is imported from react-native-reanimated.

Secondly, we will talk about the return of the CustomInput component. It returns a Controller component which contains most of the props that were taken in by the CustomInput component and the Controller component has view component inside of it and this view component has a text input component.

Lastly, the style object is created to style the component and then the CustomInput component is exported to be used in other files in the project.

CustomLayout folder:

The CustomLayout component is more like a screen as it shows the home screen in the application. It acts as a screen so it won't have many props and it would be used only in the home screen component which is the index file in the home screen folder and in the TyresTemperatureScreen and TyrePressureScreen.

CustomLayout.js:

This component is used as a home screen component and is called multiple times as shown above. It takes only one prop which is the menuOptions this menuOption prop as the name implies contains the names and icon names and other properties to show the buttons that can be pressed in the lower part of the home screen. At the start of the component code, we fetch the runningStatus and distance values from redux using the useSelector function we take them from their respective states. runBoolean Boolean variable is defined Aswell to change the color of the activity status dot.

First, we will talk about the imports of this file/component:

- 1) Import React from react.
- 2) Import View, Text, useEffect, FlatList, StyleSheet, and TouchableOpacity from react-native.
- 3) Import car from assets/images.
- 4) Import Image from react-native.
- 5) Import useSelector, useDispatch from react-redux.
- 6) Import ProgressBar from ProgressBar/ProgressBar.js.
- 7) Import FontAwesome, Entypo, MaterialCommunityIcons, FontAwesome5, Ionicons, MaterialIcons from @expo/vector-icons.

Secondly, this component returns the whole home page. So, it is divided into three parts. The first part is a view containing some text like the header and the status of the car, also has the user circle and it has the user activity dot which changes its color according to the changes in runBoolean variable. Then there is the car picture. The previous was the first part, the next part is a progressbar component which is a custom component that we will talk about a bit later. The last part is a flat list component which uses the prop from the CustomLayout component and add this menuOption prop as data value in the FlatList props. All the previous is then wrapped in between a view which takes the style container.

Lastly, the styles object is ceated to style the component as we please. Also, the component is exported to be used in other files/components in the project.

ProgressBar folder:

The ProgressBar folder contains the ProgressBar.js which is used in the CustomLayout component. It creates a progress bar which shows how many kilometers remaining till an oil change is needed. It also shows the total distance covered and a bar that changes colors when the state of the oil change changes.

ProgressBar.js:

The ProgressBar component takes in three props which are: (currentMiles, requiredMiles, oilChangeThreshold). The currentMiles from its name shows the current distance we are at right now, there is however a fault in the naming of this prop as it should be named currentKilometers as the measuring unit used is kilometers not miles. requiredMiles shows how many kilometers needed for the car to require an oil change. And oilChangeThreshold is a prop that shows the threshold at which the car should change the oil at just to be safe. There is also two if conditions, the first one compares the current miles to the required miles and shows the remaining distance if true and show a message that a change is required if false. The second one defines a barColor variable and compares between progress and oil change threshold and changes the color of the barColor variable according to the result of the compare.

First, we talk about the imports in this file/component:

- 1) Import React from react.
- 2) Import View, Text, StyleSheet from react-native.

Secondly, the ProgressBar component returns three Views components one of them shows the current distance moved and shows that number. The second View component shows the bar itself and how much progress it has. The third one shows the oil change status currently in the application all these three Views are in one View component which wraps them all.

Lastly, the styles object is created, and the component is exported to be used in other files/components in the project.

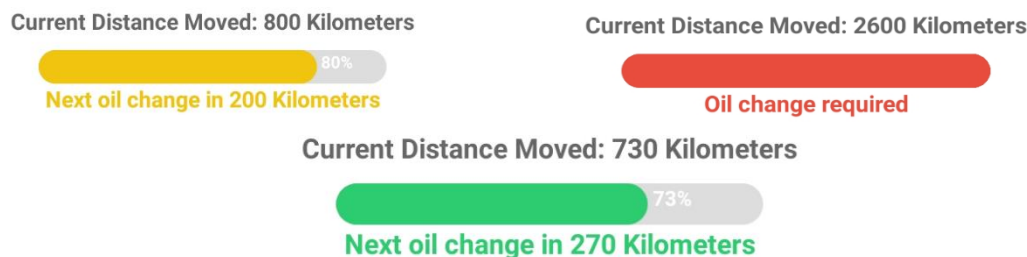


Figure 27: ProgressBar

SocialSignInButtons folder:

This folder has a file SocialSignInButtons that contains the buttons of Facebook and Google buttons. This feature is not currently available but when pressing these buttons, a message is given.

SocialSignInButtons.js:

The component SocialSignInButtons contain two functions onSignInFacebook and onSignInGoogle which currently only do one thing which sends a message for testing.

Firstly, we will talk about the imports of this file:

- 1) Import React from react.
- 2) Import View, Text from react-native.

3) Import CustomButton from /CustomButton.

Secondly, the return of these components is two CustomButton components, one for facebook and the other for google and both are wrapped in between an empty parent component.

Lastly, the component is exported to other components and files to be used in the project.

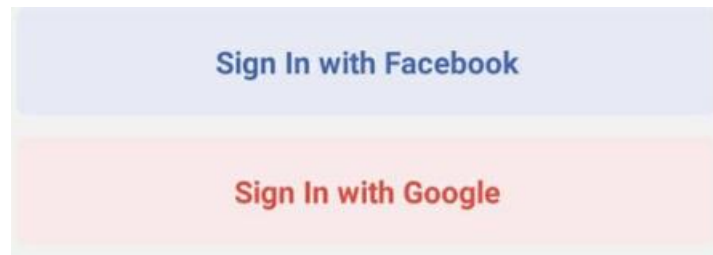


Figure 28: Sign in screen

Redux folder:

This folder contains the redux.js file that has saves the state of the whole application in it. This helps the application to save the state of their variables to be used in all screens of the application.

redux.js:

In this file the store component is created from the createStore function and takes the reducer function as argument. The redux file contains the reducer function that takes two arguments the state and the action. The state argument contains the initial state of initialState. This reducer function makes a switch case on the type of the action which are five states which contains one default state in them. The four states are SET_RUNNING_STATUS which changes the runningStatus variable, the SET_DISTANCE_STATUS state which changes the distance variable, the SET_SOCKET_STATUS state which changes the socket variable although we found a better way of using socket on all screens without using redux, the SET_USERNAME_STATUS state which changes the username variable/state, and the default state which just returns the state unchanged which is initially set as initialState.

Firstly, we will talk about the imports of this file:

1) Import createStore from redux.

Secondly, the component returns the state with the changed variable in it according to the value of the action and what the component wants to change as stated before. Also, the initialState object is created and initialized.

Lastly, the store object is created, and the store is exported to be used in other files/components in the project.

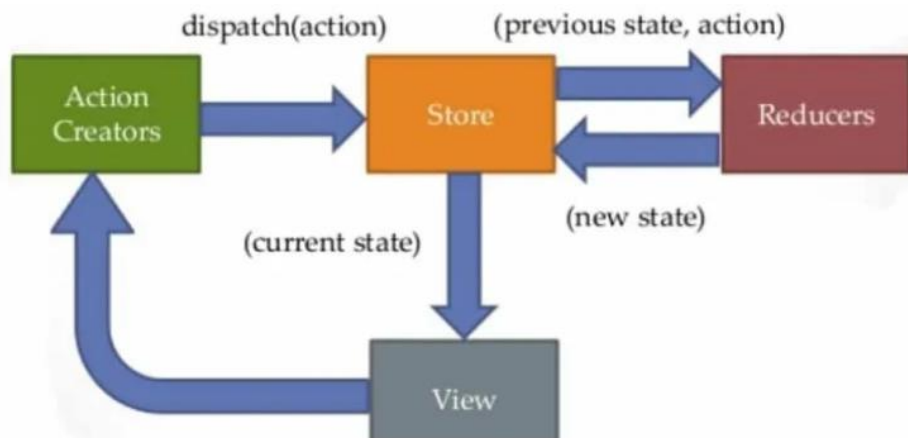


Figure 29: createStore

socket folder:

In this folder the file that is responsible for the communications with the server is created. this file which is called `socket.js`. We decided to use a socket connection with the application to the server in order to make the connection two ways connection to make the application to speak with the server and vice versa.

`socket.js`:

This file has mainly two functions defined in it one to make the socket connection and the other to disconnect it both are then exported to be used in other parts of the project. The `connectSocket` function takes the username as argument and makes an if condition that checks on the socket, if the socket is found and not null and if the `socket.connected` doesn't return false then we return the socket connection which is already been established. If this condition is not fulfilled, then we will initialize the socket object and then emit to the server the username to compare it then add the socket id to the database. While the `socketDisconnect` function makes an if condition to check that the socket is found, if so then the socket is disconnected by `socket.disconnect` function and changing the value of socket to null.

Firstly, we will talk about the imports of this file:

1) We import `io` from `socket.io-client`.

Secondly, the returns are mostly the socket object.

Lastly, the two functions `connectSocket`, `disconnectSocket` are exported to be used in other parts of the project.

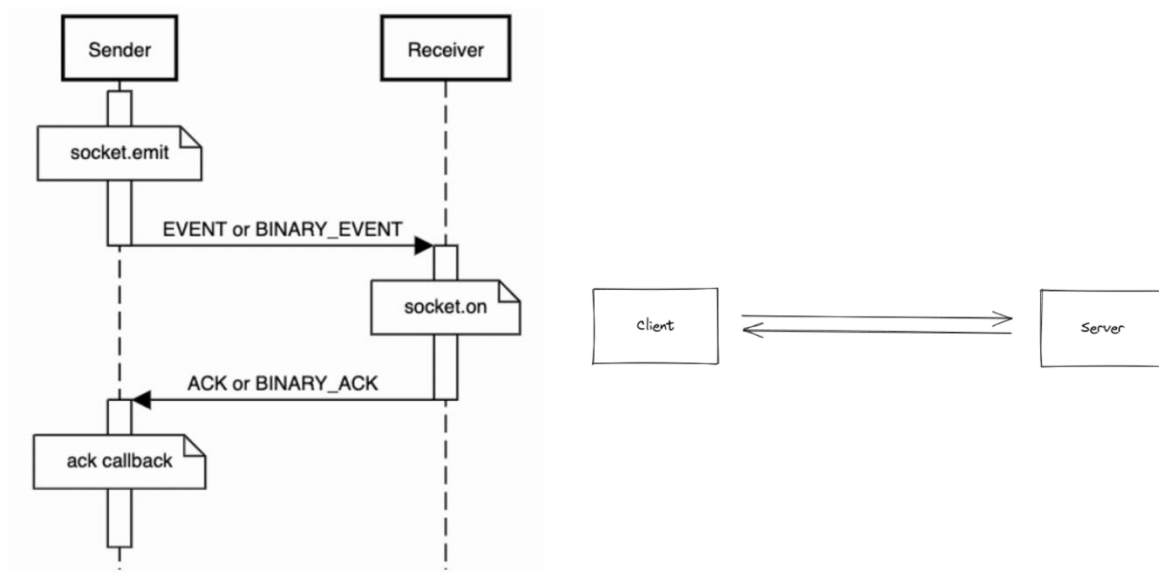


Figure 30: Socket Folder 1

constants folder

This folder contains the files that have constants values that are used in all the project instead of typing the values each time and make it redundant.

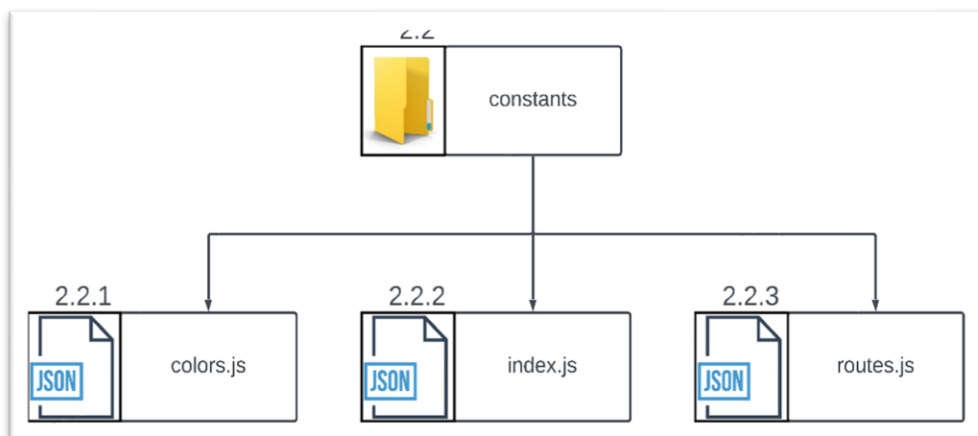


Figure 31: Constants Folder

colors.js:

This file exports some colors that are used in the project and gives them names that are easier to use than the hex color format.

index.js:

This file imports ROUTES and COLORS from their respective places in the project and exports them for ease of importing them in other places in the project.

routes.js:

This file defines an export default object that contains various key-value pairs representing different routes or screens in an application. Each key represents a unique identifier for a route, and its corresponding value represents the name or label of that route.

navigation folder

This folder contains the navigation of the whole application. It contains the bottom tab navigation and stack navigation. This folder contains two files BottomTabNav.js and index.js this index.js file contains the stack code the naming of it is not right.

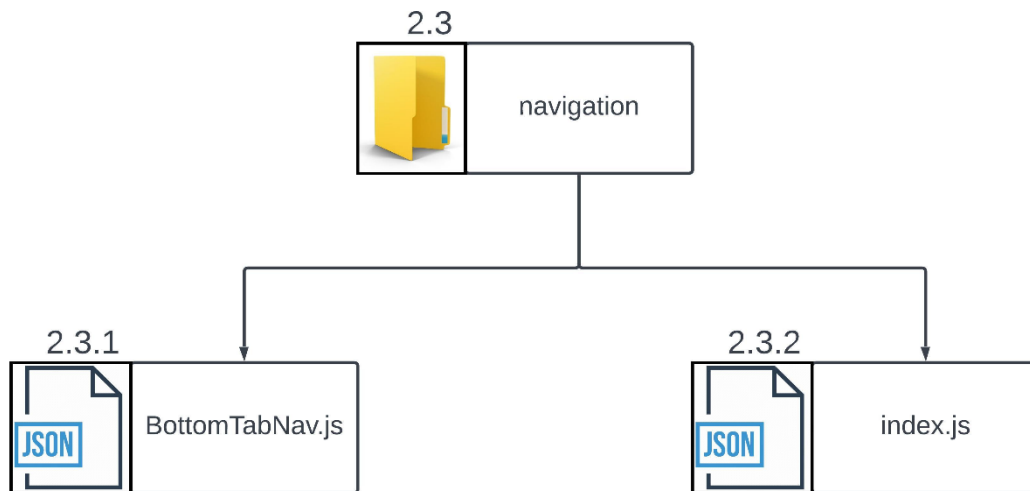


Figure 32: Navigation Folder

BottomTabNav.js:

This file contains the code for the navigation part of the bottom tab of the application. It navigates to four screens which are the home screen, GPS screen, logs screen and the settings screen.

The BottomTabNavigator component is responsible for rendering a bottom tab navigation in a React Native application using the createBottomTabNavigator function from @react-navigation/bottom-tabs library. It represents the main navigation structure for the app.

The component contains the following functions and variables:

BottomTabNavigator function: It receives the route as a prop and returns the bottom tab navigation. It utilizes the useNavigation hook to get the navigation object.

navigation: This variable is assigned the value returned by useNavigation() hook, which provides the navigation object for the component.

Then, we will talk about the imports:

The imports in the file are listed in the following order:

React: The core library for building UI components in React Native.

- 1) createBottomTabNavigator: The function from @react-navigation/bottom-tabs library used to create a bottom tab navigator.
- 2) StyleSheet, Platform, TouchableOpacity: Various utility components and functions from react-native for styling and handling user interactions.
- 3) COLORS, ROUTES: Constants imported from the constant's module.
- 4) HomeScreen, GPS, Logs, Settings: Component imports for the screens to be displayed in the tabs.

- 5) FontAwesome, Entypo, MaterialCommunityIcons, FontAwesome5, Ionicons, MaterialIcons: Icon components imported from @expo/vector-icons library.

Then, we will talk about the returns and the functions of this code.

The return of the BottomTabNavigator component consists of a Tab.Navigator component which configures the appearance of the tabs. Within the Tab.Navigator, there are multiple Tab.Screen components representing individual screens (tabs) within the bottom tab navigator.

Each Tab.Screen has the following attributes:

name: The name of the screen, derived from ROUTES constant.

component: The component to render when the tab is active.

The screenOptions prop of Tab.Navigator is used to customize the appearance of the tabs. It defines a function that receives the route and returns an object with configuration options. It sets headerShown to false, specifies the active tab color using tabBarActiveTintColor, and sets the icon for each tab using tabBarIcon. The icons are selected based on the route.name and the focused state.

Lastly, The BottomTabNavigator component is exported as the default export of the file, making it available for use in other modules.

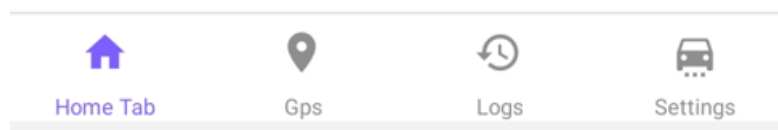


Figure 33:BottomTabNav

index.js (stack navigation):

The Navigation component is responsible for managing the navigation flow of the React Native application. It uses the NavigationContainer and createNativeStackNavigator components from the @react-navigation/native and @react-navigation/native-stack libraries, respectively. The component includes the following functions and variables:

user (state): A state variable that holds the user authentication status. It is initialized with the value undefined.

setUser (function): A function used to update the value of the user state variable.

dispatch (function): A function provided by the useDispatch hook from react-redux used for dispatching actions to the Redux store.

setUsernameStatusValue (function): A function that returns an object representing an action to set the username status in the Redux store.

setSocketStatusValue (function): A function that returns an object representing an action to set the socket status in the Redux store.

checkUser (function): An asynchronous function that checks the current authenticated user using the `Auth.currentAuthenticatedUser` method from `aws-amplify`. It updates the user state variable based on the authentication status and dispatches an action to set the username status if the user is authenticated.

useEffect (hook): A hook that runs the `checkUser` function when the component mounts to check the initial user authentication status. It also sets up a listener using the `Hub.listen` method from `aws-amplify` to listen for authentication events and update the user status accordingly. The hook is cleaned up using the return statement inside the `useEffect` function.

ActivityIndicator (component): A component from `react-native` that renders a loading indicator while the user authentication status is being determined.

Secondly, we will talk about the imports of this file:

- 1) `View`, `Text`, `ActivityIndicator` from `react-native`.
- 2) `React`, `useEffect`, `useState` from `react`.
- 3) `NavigationContainer` from `@react-navigation/native`.
- 4) `createNativeStackNavigator` from `@react-navigation/native-stack`.
- 5) `SignInScreen`, `SignUpScreen`, `ConfirmEmailScreen`, `ForgotPasswordScreen`, `NewPasswordScreen` from respective screen files.
- 6) `GPS` from `../screens/GpsScr/GPS`.
- 7) `HomeScreen` from `../screens/HomeScreen`.
- 8) `BottomTabNavigator` from `./BottomTabNav`.
- 9) `Auth`, `Hub` from `aws-amplify`.
- 10) `TyresTemperatureScreen`, `FLTTSscreen`, `TyresPressureScreen`, `PressureChartsScreen` from respective screen files.
- 11) `useDispatch`, `useSelector` from `react-redux`.
- 12) `io` from `socket.io-client`.

Thirdly, we will talk about the returns of this component: The `Navigation` component returns a `JSX` structure representing the navigation flow of the application. The return statement consists of the following components:

- 1) `NavigationContainer`: Wraps the stack navigator and provides the navigation context for the application.
- 2) `Stack.Navigator`: Represents the stack-based navigation container that holds the screens.
- 3) `Stack.Screen`: Defines individual screens within the stack navigator. The screens are conditionally rendered based on the user authentication status.
- 4) `Loading Indicator`: Rendered when the user state is undefined, indicating that the authentication status is being determined.
- 5) `Home and other Screens`: Rendered when the user is authenticated. The `BottomTabNavigator` component is used as the main screen, and other screens are included as separate stack screens.

Lastly, The `Navigation` component is exported as the default export of the file. Additionally, the `socket` variable, created using `io()` from `socket.io-client`, is exported. There are no explicit style declarations within the component file.

screens folder

This folder contains the files of the screens of the application. The screens are organized in a folder sub-folder then files format.

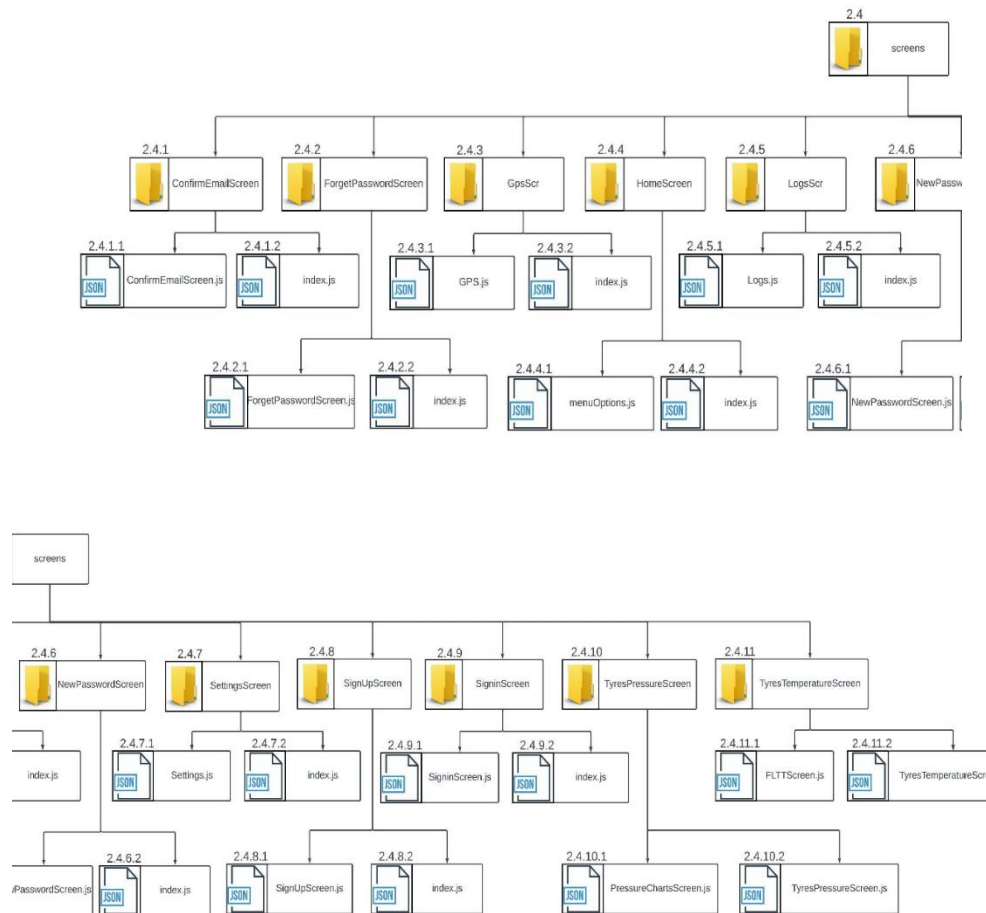


Figure 34: Screens Folder

ConfirmEmailScreen folder:

This folder contains the files that have the code for the confirm email screen. As the name implies, this screen is responsible for confirming the email after the user signs up.

ConfirmEmailScreen.js:

First, we will talk about the function `ConfirmEmailScreen`, and the function of the functions used in it.

The `ConfirmEmailScreen` component represents a screen in a React Native application that is used for confirming the user's email. It includes the following functions and variables:

`route`: A variable obtained using the `useRoute` hook from `@react-navigation/native` that provides access to the current route and its parameters.

`control`, `handleSubmit`, `watch`: Variables obtained using the `useForm` hook from `react-hook-form`. They are used for form control and handling form submissions. The

defaultValues option is used to set the default value of the username field from the route parameters.

username: The value of the username field extracted using the watch function from the control object.

navigation: A variable obtained using the useNavigation hook from @react-navigation/native. It provides access to the navigation object for navigating between screens.

onConfirmPressed: An asynchronous function triggered when the confirmation button is pressed. It attempts to confirm the sign-up using the Auth.confirmSignUp method from aws-amplify by passing the username and code values from the form data. If successful, it navigates to the 'SignIn' screen; otherwise, it displays an error message using Alert.alert.

onSignInPressed: A function triggered when the "back to Signin" button is pressed. It navigates to the 'SignIn' screen.

onResendPressed: An asynchronous function triggered when the "resend code" button is pressed. It attempts to resend the confirmation code to the user's email using the Auth.resendSignUp method from aws-amplify and displays a success message or an error message using Alert.alert.

Secondly, we will talk about the imports of the file:

- 1) React, useState from react.
- 2) View, Text, StyleSheet, ScrollView, Alert from react-native.
- 3) CustomInput from '../components/CustomInput'.
- 4) CustomButton from '../components/CustomButton'.
- 5) SocialSignInButtons from '../components/SocialSignInButtons'.
- 6) useNavigation from @react-navigation/native.
- 7) useForm from react-hook-form.
- 8) useRoute from @react-navigation/native.
- 9) Auth from aws-amplify.

Thirdly, we will talk about the returns of this component:

The ConfirmEmailScreen component returns a JSX structure representing the UI of the screen. The return statement consists of the following components:

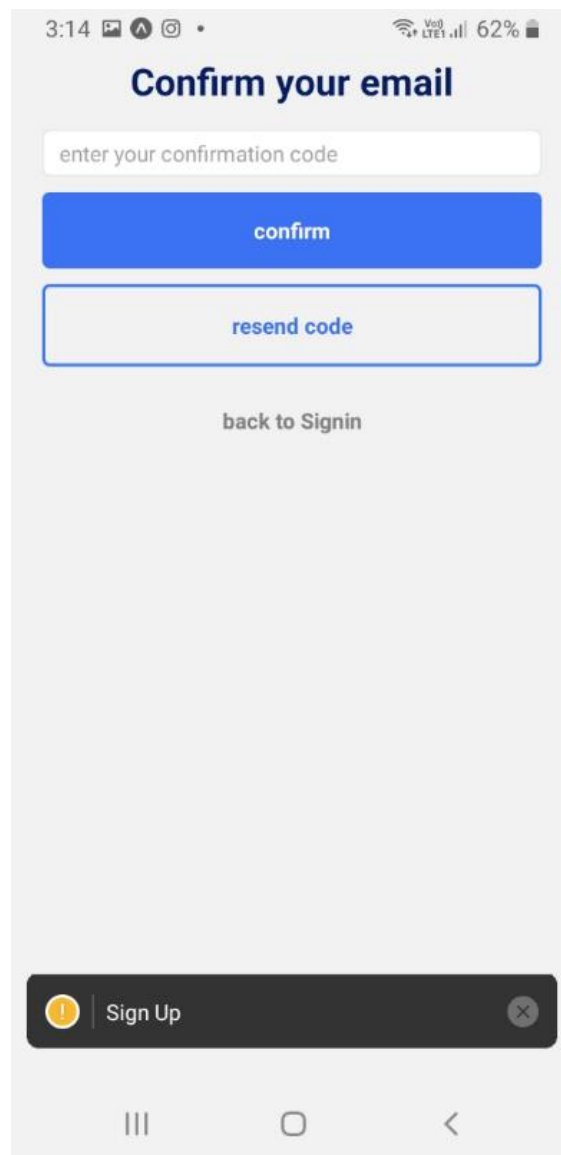
- 1) ScrollView: Provides a scrollable container for the screen content.
- 2) View (root): Represents the root container of the screen.
- 3) Text (title): Displays the title "Confirm your email".
- 4) CustomInput (username): Renders an input field for the username, controlled by the control object. It has a placeholder and validation rules.
- 5) CustomInput (code): Renders an input field for the confirmation code, controlled by the control object. It has a placeholder and validation rules.
- 6) CustomButton (Confirm): Displays a button with the text "Confirm". It triggers the onConfirmPressed function when pressed.

- 7) CustomButton (resend code): Displays a button with the text "resend code". It triggers the onResendPressed function when pressed and has a different visual style (type="SECONDARY").
- 8) CustomButton (back to Signin): Displays a button with the text "back to Signin". It triggers the onSignInPressed function when pressed and has a different visual style (type="TERTIARY").

Lastly, we will talk about the exports and the styles of this component:

The ConfirmEmailScreen component is exported as the default export of the file. Additionally, there is a style object created using StyleSheet.create. It includes the following styles:

- 1) root: Styles for the root container of the screen, including alignment and padding.
- 2) title: Styles for the title text, including font size, weight, color, and margin.
- 3) text: Styles for general text, including color and margin.
- 4) link: Styles for links, including color.



ForgetPasswordScreen folder:

This folder contains the files that have the code for the forgotten password screen. As the name implies, this screen is responsible for changing the password if the user forgets it. It does so by sending a serial code to the user email and when the user enters this code, he would be able to change the password to a new one.

ForgetPasswordScreen.js:

First, we will talk about the function ForgetPasswordScreen, and the function of the functions used in it.

- 1) The ForgotPasswordScreen component represents a screen in a React Native application used for resetting the user's password. It includes the following functions and variables:
- 2) control, handleSubmit: Variables obtained using the useForm hook from react-hook-form. They are used for form control and handling form submissions.
- 3) navigation: A variable obtained using the useNavigation hook from @react-navigation/native. It provides access to the navigation object for navigating between screens.
- 4) onSendPressed: An asynchronous function triggered when the "Send" button is pressed. It attempts to initiate the password reset process by calling Auth.forgotPassword from aws-amplify and navigates to the 'NewPassword' screen upon successful initiation. If an error occurs, it displays an error message using Alert.alert.
- 5) onSignInPressed: A function triggered when the "Back to Signin" button is pressed. It navigates to the 'SignIn' screen.

Secondly, we will talk about the imports of the file:

- 1) React, useState from react.
- 2) View, Text, StyleSheet, ScrollView from react-native.
- 3) CustomInput from '../components/CustomInput'.
- 4) CustomButton from '../components/CustomButton'.
- 5) SocialSignInButtons from '../components/SocialSignInButtons'.
- 6) useNavigation from @react-navigation/native.
- 7) useForm from react-hook-form.
- 8) Auth from aws-amplify.

Thirdly, we will talk about the returns of this component:

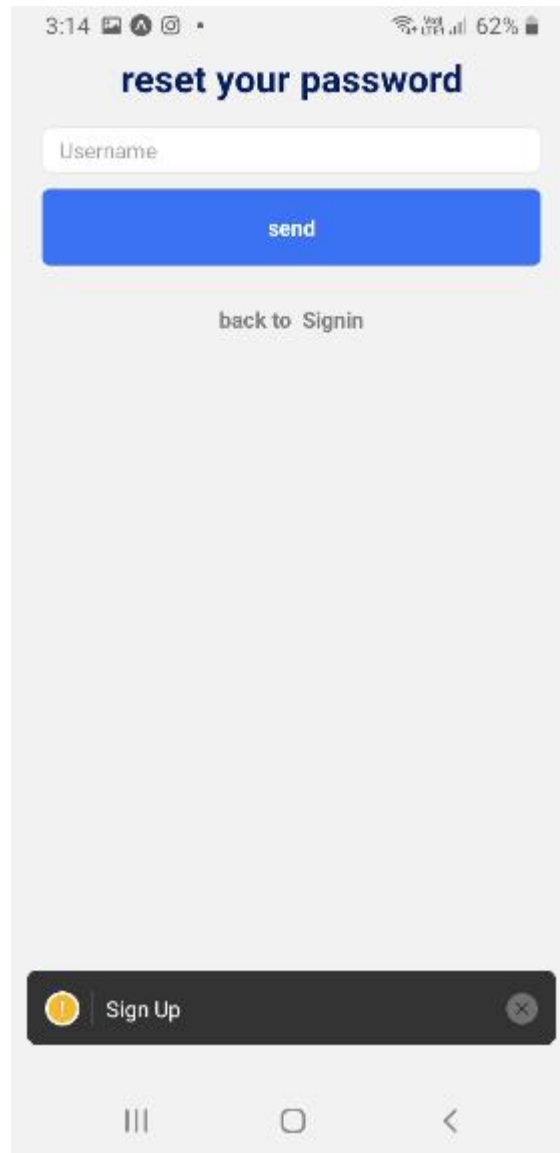
The ForgotPasswordScreen component returns a JSX structure representing the UI of the screen. The return statement consists of the following components:

- 1) ScrollView: Provides a scrollable container for the screen content.
- 2) View (root): Represents the root container of the screen.
- 3) Text (title): Displays the title "reset your password".
- 4) CustomInput (username): Renders an input field for the username, controlled by the control object. It has a placeholder and validation rules.
- 5) CustomButton (Send): Displays a button with the text "Send". It triggers the onSendPressed function when pressed.
- 6) CustomButton (Back to Signin): Displays a button with the text "Back to Signin". It triggers the onSignInPressed function when pressed and has a different visual style (type="TERTIARY").

Lastly, we will talk about the exports and the styles of this component:

The ForgotPasswordScreen component is exported as the default export of the file. Additionally, there is a style object created using StyleSheet.create. It includes the following styles:

- 1) root: Styles for the root container of the screen, including alignment and padding.
- 2) title: Styles for the title text, including font size, weight, color, and margin.
- 3) text: Styles for general text, including color and margin.
- 4) link: Styles for links, including color.



GpsScr folder:

This folder contains the files that have the code for the GPS screen. As the name implies, this screen is responsible for showing the location of the car using the longitude and latitude of the car sent from the embedded kit and show it on the map from google services.

First, we will talk about the function of the GPS screen, and the function of the functions used in it.

The GPS component is responsible for displaying a map view with markers and directions. It includes the following functions and variables:

- 1) navigator: A variable obtained using the useNavigation hook from @react-navigation/native. It provides access to the navigation object for navigating between screens.
- 2) checkLocationEnabled: An asynchronous function that checks if location services are enabled on the device.
- 3) getPosition: An asynchronous function that retrieves the stored longitude and latitude values from AsyncStorage and sets the position state.
- 4) handleDirectionsReady: A function that sets the directions state when directions are ready.
- 5) requestLocationPermission: An asynchronous function that requests permission to access the device's location.
- 6) setPosition: A state variable that holds the longitude and latitude values for the user's current position.
- 7) setMyLocation: A state variable that holds the longitude and latitude values for the user's location.
- 8) isLocationEnabled: A state variable that indicates whether location services are enabled on the device.
- 9) username: A variable obtained using the useSelector hook from react-redux. It retrieves the username from the Redux store.
- 10) getLocationAsync: An asynchronous function that retrieves the user's current location using the watchPositionAsync function from expo-location.
- 11) isMounted: A boolean variable that determines whether the component is mounted or not. It is used to prevent memory leaks when unsubscribing from the location updates.

Secondly, we will talk about the imports of the file:

- 1) React, useEffect, useState from react.
- 2) Pressable, View, Text, StyleSheet, Image from react-native.
- 3) NavigationContainer from @react-navigation/native.
- 4) useNavigation from @react-navigation/native.
- 5) io from socket.io-client
- 6) createStackNavigator, HeaderBackButton from @react-navigation/stack.
- 7) PermissionsAndroid from react-native.
- 8) myLoc from expo-location.
- 9) MapViewDirections from react-native-maps-directions.
- 10) AsyncStorage from @react-native-async-storage/async-storage.
- 11) socket from '../SignInScreen'.
- 12) connectSocket from '../components/socket/socket'.
- 13) useSelector from react-redux.
- 14) MapView, Marker from react-native-maps.

Thirdly, we will talk about the returns of this component:

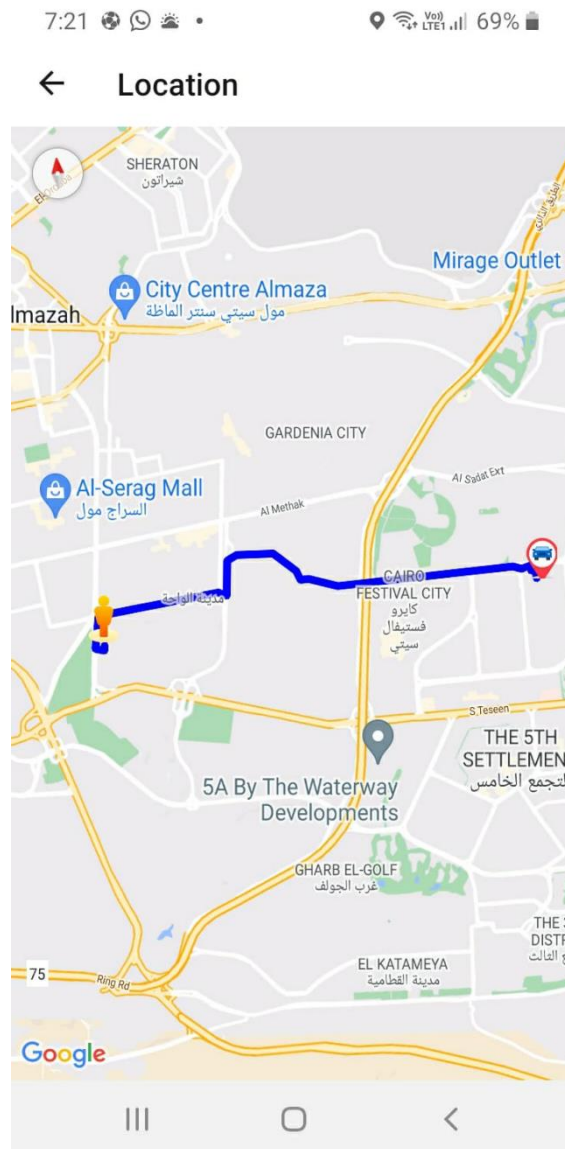
The GPS component returns a JSX structure representing the UI of the screen. The return statement consists of the following components:

- 1) View (container): Represents the root container of the component.
- 2) MapView (map): Renders the map view with a specified region and shows the user's location.
- 3) Marker (position): Displays a marker on the map for the user's current position.
- 4) MapViewDirections: Renders the directions between the user's location and the specified position.
- 5) The return statement also includes conditional rendering based on the `isLocationEnabled` state. If location services are disabled, only the map with the position marker is displayed. If location services are enabled, the directions are also displayed.

Lastly, we will talk about the exports and the styles of this component:

The GPS component is exported as the default export of the file. Additionally, there is a style object created using `StyleSheet.create`. It includes the following styles:

- 1) container: Styles for the container, setting flex to 1.
- 2) map: Styles for the map, setting width and height to '100%'.
- 3) buttonStyle: Styles for buttons, including color and margin.
- 4) marker: Styles for markers, including width, height, and resizeMode.
- 5) directionsContainer: Styles for the container of directions, including background color and padding.
- 6) directionsRow: Styles for the row containing directions, including flexDirection, alignItems, and marginBottom.
- 7) directionsText: Styles for the text of directions, including flex, fontSize, and marginRight.
- 8) directionsDistance: Styles for the distance of directions, including fontSize and color.



HomeScreen folder:

This folder contains the files that have the code for the home screen. As the name implies, this screen contains the code that makes the home screen look as it does and starts the connection with socket and redux.

index.js (homeScreen.js):

First, we will talk about the function of the home screen, and the function of the functions used in it. The Home component represents the home screen of the application. It includes the following functions and variables:

- 1) `setRunningStatusValue`: A function that returns an action object to set the running status value in the Redux store.
- 2) `setDistanceStatusValue`: A function that returns an action object to set the distance status value in the Redux store.
- 3) `runningStatus`: A variable that holds the current running status, initially set to 'Parked'.
- 4) `navigation`: A variable obtained using the `useNavigation` hook from `@react-navigation/native`. It provides access to the navigation object for navigating between screens.
- 5) `isFocused`: A variable obtained using the `useIsFocused` hook from `@react-navigation/native`. It indicates whether the screen is focused or not.
- 6) `socket`: A state variable that holds the socket object for establishing a connection with the server.
- 7) `isConnected`: A state variable that indicates whether the socket is connected or not.
- 8) `dispatch`: A variable obtained using the `useDispatch` hook from `react-redux`. It provides access to the Redux dispatch function.
- 9) `menuOptions`: An array of objects representing the available menu options on the home screen, including `name`, `iconName`, and `onPress/href` properties.

Secondly, we will talk about the imports of the file:

- 1) `View`, `Text`, `useEffect`, `useState`, `Linking`, `FlatList`, `StyleSheet`, `TouchableOpacity` from `react-native`.
- 2) `Auth` from `aws-amplify`.
- 3) `useState`, `useEffect`, `useRef` from `react`.
- 4) `React` from `react`.
- 5) `Image` from `react-native`.
- 6) `CustomLayout` from `../components/CustomLayout/CustomLayout`.
- 7) `useNavigation`, `useIsFocused` from `@react-navigation/native`.
- 8) `useDispatch`, `useSelector` from `react-redux`.
- 9) `Navigation` from `../navigation`.
- 10) `connectSocket`, `disconnectSocket` from `../components/socket/socket`.

Thirdly, we will talk about the returns of this component:

The Home component returns a JSX structure representing the UI of the screen. The return statement consists of the following component:

Lastly, we will talk about the exports and the styles of this component:

- 1) `CustomLayout`: Represents a custom layout component that displays the menu options on the home screen. It receives the `menuOptions` array as a prop.

The Home component is exported as the default export of the file. Additionally, there are no specific styles defined in the given code snippet.

`menuOptions.js`:

First, we will talk about the function of the `menuOptions` file, and the function of the functions used in it. This file is an array of menu options for the custom layout which is called in the home screen. Each menu option consists of the following properties:

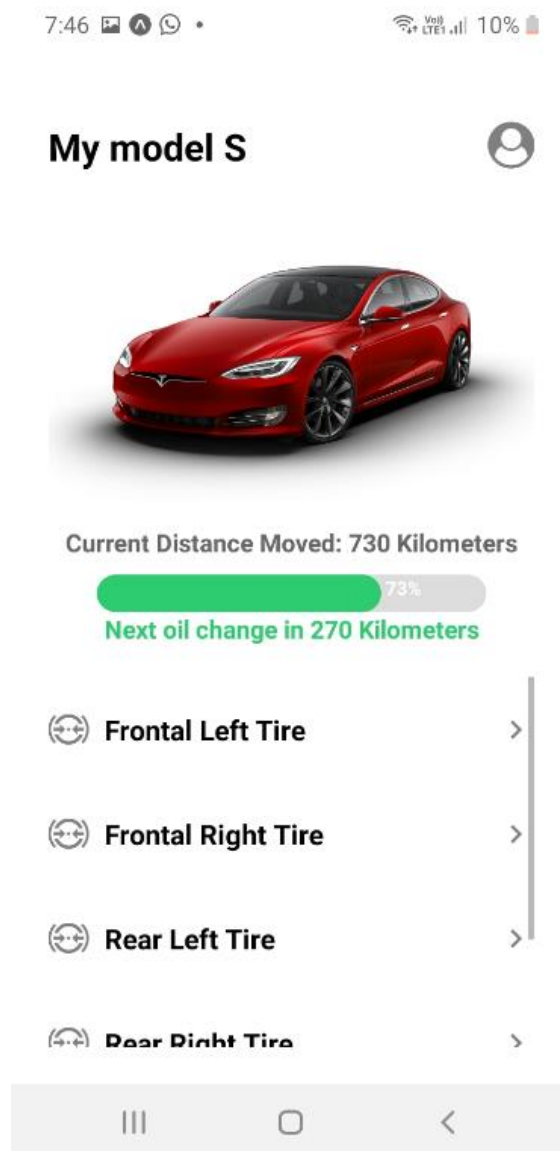
- 1) `name`: A string representing the name or label of the menu option.

- 2) `iconName`: A string representing the name of the icon associated with the menu option.
- 3) `href` (optional): A string representing the URL or path associated with the menu option.
- 4) `onPress` (optional): A function that is executed when the menu option is pressed.

Secondly, we will talk about the imports of the file:

- 1) `GPS` from `"../GpsScr/GPS"`.
- 2) `useNavigation` from `@react-navigation/native`.

Lastly, the export of this code is the whole menu options.



LogsScr folder:

This folder contains the files that have the code for the logs screen. As the name implies, this screen contains the code that makes the logs screen work as it does. It shows the history of all the data the embedded kit sends.

Logs.js:

First, we will talk about the function of the home screen, and the function of the functions used in it. The DataScreen component is a React Native component responsible for rendering a list of temperature and pressure data entries. It takes a data prop, which should contain two arrays: FLTT (temperature data) and FLTP (pressure data). The component merges the temperature and pressure entries based on their timestamps and displays them together in a formatted list.

Inside the component, the following functions and variables are declared:

- 1) combinedData: An array used to store the merged temperature and pressure entries with the same timestamp.
- 2) renderDataItem: A function used as the rendering template for each item in the list.
- 3) keyExtractor: A function used to extract a unique key for each item in the list.

Secondly, we will talk about the imports of the file:

- 1) React from the 'react' package.
- 2) View, Text, FlatList, and StyleSheet from the 'react-native' package.
- 3) FeatherIcon from the 'react-native-vector-icons/Feather' package.
- 4) moment for date and time formatting.

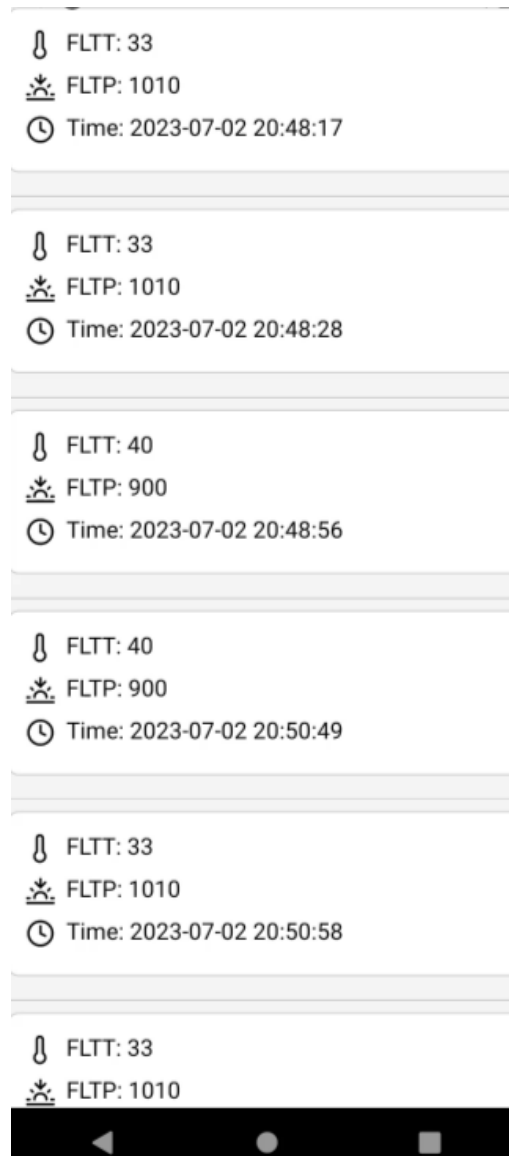
Thirdly, we will talk about the returns of this component:

- 1) A container view with the specified styles.
- 2) A FlatList component that renders the merged data entries.
- 3) The FlatList component receives the combinedData array as the data source.
- 4) The renderDataItem function is used to render each item in the list.
- 5) The keyExtractor function is used to extract unique keys for each item.
- 6) An ItemSeparatorComponent is provided to display a separator between list items.
- 7) The renderDataItem function defines the structure for each item in the list. Each item consists of:
 - 8) An item container view with the specified styles.
 - 9) A temperature icon and text displaying the temperature value.
 - 10) A pressure icon and text displaying the pressure value.
 - 11) A clock icon and text displaying the timestamp.

Lastly, we will talk about the exports and the styles of this component:

The DataScreen component is exported as the default export of the file.

The component also defines a StyleSheet object that contains styles for various elements in the component, including the container, item container, icons, and separator.



NewPasswordScreen folder:

This folder contains the files that have the code for the new password screen. As the name implies, this screen contains the code that makes the user changes the password after he enters the email in the forget password screen.

NewPasswordScreen.js:

First, we will talk about the function of the GPS screen, and the function of the functions used in it. It is responsible for rendering a screen where users can reset their password. The component includes the following functions, variables, and elements:

- 1) control and handleSubmit from the useForm hook provided by react-hook-form.

- 2) navigation obtained using the `useNavigation` hook from `@react-navigation/native`.
- 3) The component also defines a function `onSubmitPressed` which handles the submission of the new password form. It calls the `forgotPasswordSubmit` method from the `Auth` module provided by `aws-amplify` to reset the password. Additionally, there is an `onSignInPressed` function that navigates the user back to the sign-in screen.

Secondly, we will talk about the imports of the file:

- 1) `React` from `'react'`.
- 2) `useState` from `'react'`.
- 3) `View` from `'react-native'`.
- 4) `Text` from `'react-native'`.
- 5) `StyleSheet` from `'react-native'`.
- 6) `ScrollView` from `'react-native'`.
- 7) `CustomInput` from `'../components/CustomInput'`.
- 8) `CustomButton` from `'../components/CustomButton'`.
- 9) `SocialSignInButtons` from `'../components/SocialSignInButtons'`.
- 10) `useNavigation` from `@react-navigation/native`.
- 11) `useForm` from `'react-hook-form'`.
- 12) `Auth` from `'aws-amplify'`.

Thirdly, we will talk about the returns of this component:

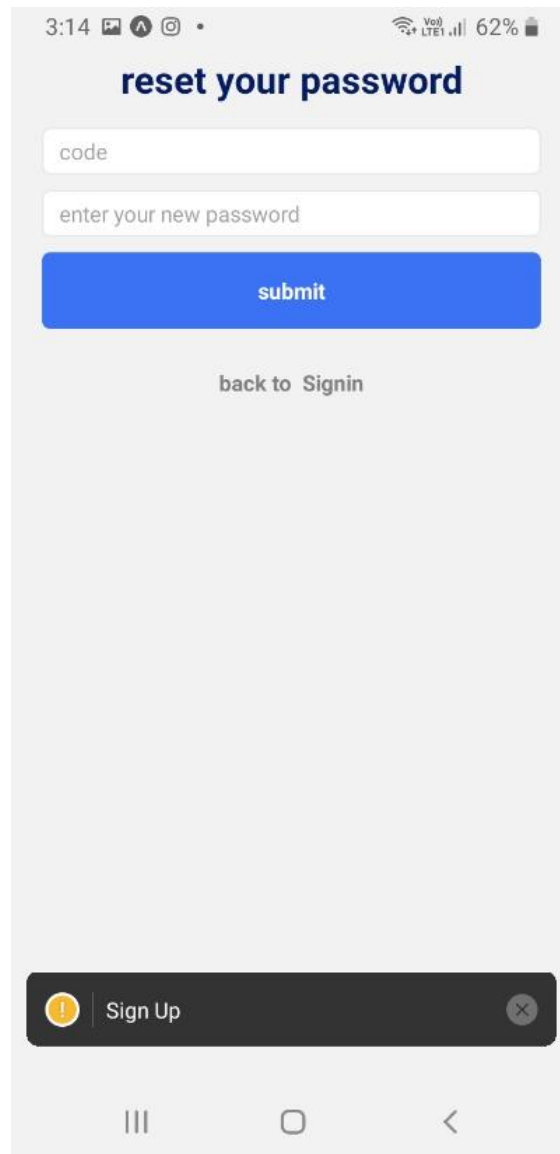
The component returns a `ScrollView` component wrapped around a `View` component. Inside the `View`, the following elements are rendered in the specified order:

- 1) `Text`: component displaying the title "reset your password" with the style defined in the `styles.title` object.
- 2) `CustomInput`: component for the "username" input field, controlled by the control object from `useForm`, and applying the specified validation rules.
- 3) `CustomInput`: component for the "code" input field, controlled by the control object from `useForm`, and applying the specified validation rules.
- 4) `CustomInput`: component for the "enter your new password" input field, controlled by the control object from `useForm`, and applying the specified validation rules. The `secureTextEntry` prop is set to `true` to hide the password.
- 5) `CustomButton`: component for submitting the form. It triggers the `handleSubmit` function from `useForm` when pressed, which calls the `onSubmitPressed` function.
- 6) `CustomButton`: component for navigating back to the sign-in screen. It triggers the `onSignInPressed` function when pressed.

Lastly, we will talk about the exports and the styles of this component:

The component is exported as the default export of the file. Additionally, the code defines a `styles` object using `StyleSheet.create`. The `styles` object includes the following style rules:

- 1) `root`: Styles for the root container view of the component, including alignment and padding.
- 2) `title`: Styles for the title text, including font size, font weight, color, and margin.
- 3) `text`: Styles for regular text elements, including color and margin.
- 4) `link`: Styles for links, including color.



SellCar folder:

This folder contains the files that have the code for the sell car screen. As the name implies, this screen contains the code that makes the user change the owner of the car in case he wants to change it to sell the car.

SellCar.js:

First, we will talk about the function of the GPS screen, and the function of the functions used in it. This component is responsible for rendering a screen where users can enter their email address to receive a one-time password for a car selling process. The component includes the following functions, variables, and elements:

- 1) `useState` hook is used to manage the state of the email and `isValidEmail` variables.
- 2) `useSelector` hook from `react-redux` is used to extract the username from the Redux store.
- 3) `handleSubmit` function is called when the user submits the form. It emits a "sellcar" event to a socket connection, passing the email and username as data, and then calls `Auth.deleteUser()` from `aws-amplify` to delete the user.

- 4) `handleEmailChange` function is called when the email text input value changes. It updates the email state and checks the validity of the email address using the `validateEmail` function.
- 5) `validateEmail` function validates the email address using a regular expression.

Secondly, we will talk about the imports of the file:

- 1) `React` from `'react'`.
- 2) `useState` from `'react'`.
- 3) `View` from `'react-native'`.
- 4) `Text` from `'react-native'`.
- 5) `TextInput` from `'react-native'`.
- 6) `TouchableOpacity` from `'react-native'`.
- 7) `StyleSheet` from `'react-native'`.
- 8) `connectSocket` from `'../components/socket/socket'`.
- 9) `useSelector` from `'react-redux'`.
- 10) `Auth` from `'aws-amplify'`.

Thirdly, we will talk about the returns of this component:

The component returns a `View` component that contains the following elements rendered in the specified order:

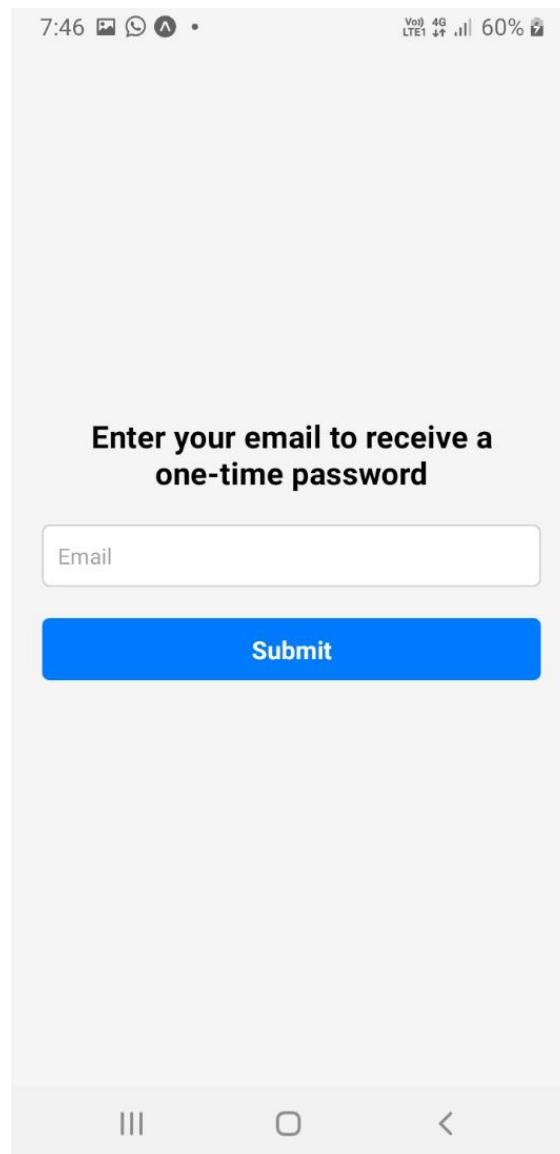
- 1) `Text`: component displaying the heading text "Enter your email to receive a one-time password" with the style defined in `styles.heading`.
- 2) `TextInput`: component for entering the email address. It is controlled by the email state and triggers the `handleEmailChange` function when the text changes. It also applies different styles (`styles.input` and `styles.inputError`) based on the validity of the email address.
- 3) Conditional rendering of an error message (`Text`): when `isValidEmail` is false. The error message is displayed if an invalid email address is entered.
- 4) `TouchableOpacity`: component representing a submit button. It triggers the `handleSubmit` function when pressed.
- 5) `Text`: component displaying the button text "Submit" with the style defined in `styles.buttonText`.

Lastly, we will talk about the exports and the styles of this component:

The component is exported as the default export of the file. Additionally, the code defines a `styles` object using `StyleSheet.create`. The `styles` object includes the following style rules:

- 1) `container`: Styles for the root container view of the component, including flex properties, padding, alignment, and background color.
- 2) `heading`: Styles for the heading text, including font size, font weight, margin, and text alignment.
- 3) `input`: Styles for the email text input, including width, height, padding, margin, border properties, border color, border radius, and background color.
- 4) `inputError`: Styles applied to the input when the email is invalid, including a red border color and increased border width.

- 5) button: Styles for the submit button, including width, height, alignment, background color, and border radius.
- 6) buttonText: Styles for the button text, including font size, font weight, and color.
- 7) errorText: Styles for the error message text, including color and margin.



SettingsScreen folder:

This folder contains the files that have the code for the settings screen. As the name implies, this screen contains the code that have the settings options.

Settings.js:

SignUpScreen folder:

This folder contains the files that have the code for the sign-up screen. As the name implies, this screen contains the code that makes the user sign up as a new user adding his own username email OTP and password.

SignUpScreen.js:

First, we will talk about the function of the Sign-up screen, and the function of the functions used in it. This component is responsible for rendering a screen where users can sign up by providing their name, username, email, one-time password (OTP), and password. The component includes the following functions, variables, and elements:

- 1) The component utilizes the useForm hook from the 'react-hook-form' library to handle form validation and submission.
- 2) The handleSubmit, watch, and control variables are extracted from the useForm hook to handle form submission and retrieve form values.
- 3) The pwd variable uses the watch function to retrieve the value of the 'password' field.
- 4) The navigation variable uses the useNavigation hook from '@react-navigation/native' to handle navigation within the app.
- 5) The onRegisterPressed function is called when the user presses the "Register" button. It makes a PATCH request to a specific endpoint using the axios library to update the user's information. It also calls Auth.signUp from 'aws-amplify' to initiate the user registration process with AWS Cognito.
- 6) The onSignInPressed function is called when the user presses the "have an account? Signin" button. It navigates the user to the sign-in screen.
- 7) The onTermsOfUsePressed function is a placeholder that can be implemented to handle user interaction with the "Terms of Use" link.
- 8) The onPrivacyPressed function is a placeholder that can be implemented to handle user interaction with the "Privacy Policy" link.

Secondly, we will talk about the imports of the file:

- 1) React from 'react'.
- 2) useState from 'react'.
- 3) axios from 'axios'.
- 4) View from 'react-native'.
- 5) Text from 'react-native'.
- 6) StyleSheet from 'react-native'.
- 7) ScrollView from 'react-native'.
- 8) Alert from 'react-native'.
- 9) CustomInput from '../components/CustomInput'.
- 10) CustomButton from '../components/CustomButton'.
- 11) SocialSignInButtons from
 '../components/SocialSignInButtons'.
- 12) NavigationContainer and useNavigation from '@react-navigation/native'.
- 13) useForm from 'react-hook-form'.
- 14) Auth from 'aws-amplify'.

Thirdly, we will talk about the returns of this component:

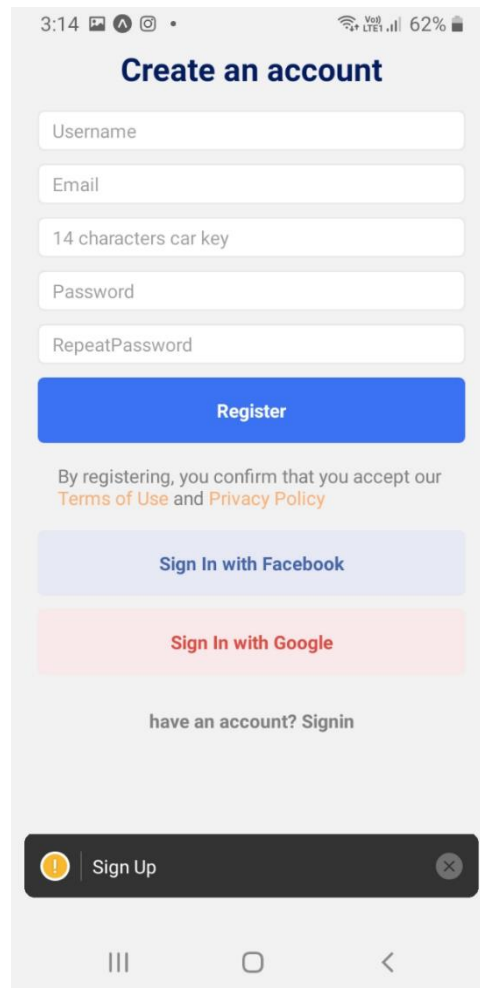
The component returns a ScrollView component that contains a View component. Inside the View, the elements are rendered in the following order:

- 1) Text: component displaying the title "Create an account" with the style defined in `styles.title`.
- 2) CustomInput: component for entering the user's name, controlled by the name field in the form and applying validation rules.
- 3) CustomInput: component for entering the user's username, controlled by the username field in the form and applying validation rules.
- 4) CustomInput: component for entering the user's email, controlled by the email field in the form and applying email validation.
- 5) CustomInput: component for entering the OTP, controlled by the OTP field in the form and applying validation rules.
- 6) CustomInput: component for entering the user's password, controlled by the password field in the form and applying validation rules.
- 7) CustomInput: component for re-entering the user's password for confirmation, controlled by the password-repeat field in the form and applying a validation rule to match the password field.
- 8) CustomButton: component representing the "Register" button, which triggers the `handleSubmit` function when pressed.
- 9) Text: component displaying a text with links to the "Terms of Use" and "Privacy Policy", styled accordingly.
- 10) SocialSignInButtons: component rendering social sign-in buttons.
- 11) CustomButton: component representing the "have an account? Signin" button, which triggers the `onSignInPressed` function when pressed.

Lastly, we will talk about the exports and the styles of this component:

The component is exported as the default export of the file. The styles object contains the following style definitions:

- 1) root: Styles applied to the root View component, including alignment and padding.
- 2) title: Styles applied to the title Text component, including font size, font weight, color, and margin.
- 3) text: Styles applied to the general text Text component, including color and margin vertical.
- 4) link: Styles applied to the link Text components within the general text, including color.



SigninScreen folder:

This folder contains the files that have the code for the sign-in screen. As the name implies, this screen contains the code that makes the user sign in to an existing account using the username and password.

SigninScreen.js:

First, we will talk about the function of the Sign-up screen, and the function of the functions used in it. This component is responsible for rendering a screen where users can sign in by providing their username and password. The component includes the following functions, variables, and elements:

- 1) The component utilizes the useForm hook from the 'react-hook-form' library to handle form validation and submission.
- 2) The handleSubmit, control, and formState variables are extracted from the useForm hook to handle form submission, form control, and form state.
- 3) The height variable uses the useWindowDimensions hook to retrieve the height of the window/screen.
- 4) The navigation variable uses the useNavigation hook from '@react-navigation/native' to handle navigation within the app.

- 5) The loading and setLoading variables use the useState hook to manage the loading state of the sign-in process.
- 6) The dispatch variable uses the useDispatch hook from 'react-redux' to dispatch actions.
- 7) The setSocketStatusValue function is defined and intended to dispatch an action to set the socket status value.

Secondly, we will talk about the imports of the file:

- 1) React from 'react'.
- 2) useState from 'react'.
- 3) useDispatch from 'react-redux'.
- 4) io from 'socket.io-client'.
- 5) View from 'react-native'.
- 6) Text from 'react-native'.
- 7) Image from 'react-native'.
- 8) StyleSheet from 'react-native'.
- 9) useWindowDimensions from 'react-native'.
- 10) ScrollView from 'react-native'.
- 11) TextInput from 'react-native'.
- 12) Alert from 'react-native'.
- 13) Logo from '../assets/images/ASU2.png'.
- 14) CustomInput from '../components/CustomInput'.
- 15) CustomButton from '../components/CustomButton'.
- 16) SocialSignInButtons from '../components/SocialSignInButtons'.
- 17) useNavigation from '@react-navigation/native'.
- 18) useForm and Controller from 'react-hook-form'.
- 19) Auth from 'aws-amplify'.

Thirdly, we will talk about the returns of this component:

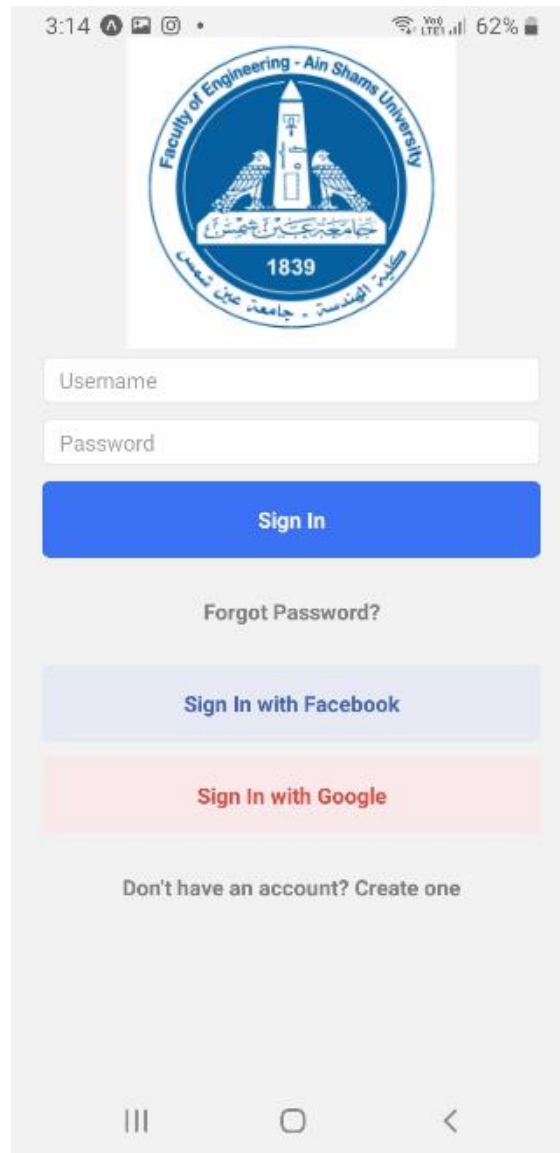
The component returns a ScrollView component that contains a View component. Inside the View, the elements are rendered in the following order:

- 1) Image: component displaying the logo image sourced from Logo variable, with its height adjusted based on the window height.
- 2) CustomInput: component for entering the user's username, controlled by the username field in the form and applying validation rules.
- 3) CustomInput: component for entering the user's password, controlled by the password field in the form and applying validation rules.
- 4) CustomButton: component representing the "Sign In" button, which triggers the handleSubmit function when pressed. The button text changes to "Loading..." when the loading state is true.
- 5) CustomButton: component for the "Forgot Password?" functionality, triggering the onForgotPasswordPressed function when pressed.
- 6) SocialSignInButtons: component rendering social sign-in buttons.
- 7) CustomButton: component for creating a new account, triggering the onSignUpPressed function when pressed.

Lastly, we will talk about the exports and the styles of this component:

The component is exported as the default export of the file. The styles object contains the following style definitions:

- 1) root: Styles applied to the root View component, including alignment, padding, and background color.



TyresPressureScreen folder:

This folder contains the files that have the code for the tires pressure screen. As the name implies, this screen contains the code that makes the see information about the car pressure whether currently or a graph of past measurements.

PressureChartsScreen.js:

First, we will talk about the function of the pressure charts screen, and the function of the functions used in it. The code includes a React functional component called PressureChartsScreen that receives the route prop. Within this component, the following functions are defined:

- 1) getTimeWithAmPm(date): Formats a given date object to display the time in 12-hour format.
- 2) getDateOnly(date): Formats a given date object to display only the date in YYYY-MM-DD format.

- 3) `filterData()`: Filters the chart data based on the selected time frame, returning a new data array and timestamps array.

The component also declares several state variables using the `useState` hook:

- 1) `selectedTimeFrame`: Stores the selected time frame for data filtering.
- 2) `chartData`: Holds the data and labels for the line chart.
- 3) `isLoading`: Tracks the loading state of the component.
- 4) Additionally, the code imports the `LineChart` component from the `react-native-chart-kit` library, various components and utilities from `react-native`, and the `useSelector` hook from `react-redux`. The `connectSocket` function is imported from a custom file called `socket/socket`.

Secondly, we will talk about the imports of the file:

- 1) `React`, `useEffect`, `useState` from `react`.
- 2) `LineChart` from `react-native-chart-kit`.
- 3) `View`, `Text`, `TouchableOpacity`, `StyleSheet`, `button`, `ActivityIndicator` from `react-native`.
- 4) `Dimensions` from `react-native`.
- 5) `CustomGauge` from a custom file called `../components/CustomGauge/CustomGauge`.
- 6) `useSelector` from `react-redux`.
- 7) `connectSocket` from a custom file called `../components/socket/socket`.

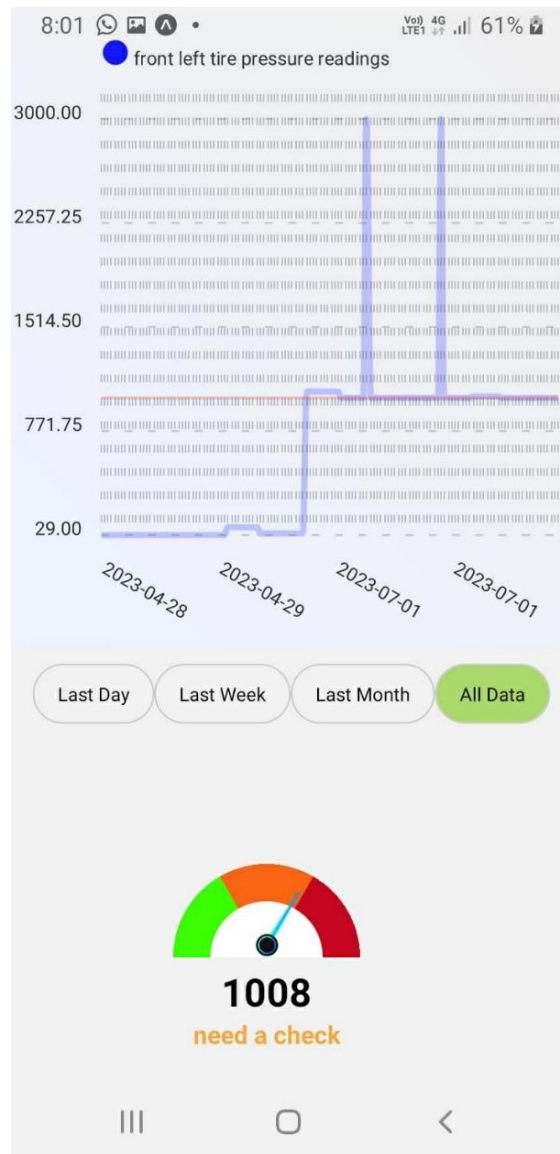
Thirdly, we will talk about the returns of this component:

The component returns a view container with conditional rendering based on the `isLoading` state. The return statement consists of two main sections:

- 1) When loading, an `ActivityIndicator` component is displayed.
- 2) When not loading:
- 3) A `LineChart` component is rendered, displaying the chart data based on the selected time frame.
- 4) Four buttons are displayed for selecting different time frames.
- 5) A `CustomGauge` component is shown, displaying the latest value from the chart data.

Lastly, we will talk about the exports and the styles of this component:

The `PressureChartsScreen` component is exported as the default export of the file. The component's styles are defined using the `StyleSheet.create` function, including styles for buttons, button containers, and the gauge.



TyresPressureScreen.js:

First, we will talk about the function of the pressure charts screen, and the function of the functions used in it. The TyresPressureScreen component represents a screen that displays a custom layout with menu options related to tire pressure readings. It uses the CustomLayout component to render the menu options. The screen allows users to navigate to different charts displaying tire pressure data for different tires. The main function in the component is TyresPressureScreen, which defines the behavior and structure of the screen. It includes the necessary imports, declares variables, and defines the return statement.

Secondly, we will talk about the imports of the file:

- 1) View and Text from 'react-native': These are React Native components used for rendering UI elements.
- 2) useEffect, useState, Linking, FlatList, StyleSheet, and TouchableOpacity from 'react-native': These are React Native hooks and components used for managing state, handling side effects, and creating interactive UI elements.

- 3) Auth from 'aws-amplify': This import is not used in the component.
- 4) socket from '../SignInScreen/SignInScreen': This import is used to establish a socket connection but is not used directly in this component.
- 5) car from '../assets/images/car.png': This import is not used in the component.
- 6) Image from 'react-native': This import is not used in the component.
- 7) CustomLayout from '../components/CustomLayout/CustomLayout': This import is used to render the custom layout for the screen.
- 8) useNavigation from '@react-navigation/native': This hook is used to access the navigation object for navigating between screens.
- 9) FLTTScreen from '../TyresTemperatureScreen/FLTTScreen': This import is not used in the component.
- 10) disconnectSocket from '../components/socket/socket': This import is used to disconnect the socket connection, but it is not used directly in this component.

Thirdly, we will talk about the returns of this component:

This returns the CustomLayout component with the menuOptions prop passed to it. It renders the custom layout on the screen.

Lastly, we will talk about the exports and the styles of this component:

The component is exported as the default export at the end of the file to be used in other files and components in the project.

TyresTempratureScreen folder:

This folder contains the files that have the code for the tires temperature screen. As the name implies, this screen contains the code that makes the see information about the car temperature whether currently or a graph of past measurements.

FLTTScreen.js:

First, we will talk about the function of the pressure charts screen, and the function of the functions used in it. The code represents a React component named FLTTScreen. Here is an overview of the component's functions and variables:

Functions:

- 1) getTimeWithAmPm(date): A function that takes a Date object as input and returns a formatted string representing the time in 12-hour format with AM/PM.
- 2) getDateOnly(date): A function that takes a Date object as input and returns a formatted string representing the date in the format "YYYY-MM-DD".
- 3) filterData(): A function that filters the chart data based on the selected time frame. It returns an object with filtered data and labels.
- 4) useEffect(): A hook function that runs side effects and is used to establish a socket connection and retrieve data.

Variables:

- 1) selectedTimeFrame: A state variable that holds the currently selected time frame for data filtering.
- 2) chartData: A state variable that stores the chart data and labels.
- 3) isLoading: A state variable that indicates whether the data is being loaded or not.

- 4) screenWidth: A constant variable that holds the width of the device screen.
- 5) values: An array that stores the data values for the chart.
- 6) timestamps: An array that stores the timestamps for the chart data.
- 7) username: A variable that stores the value retrieved from the Redux store.

Secondly, we will talk about the imports of the file:

- 1) React and other necessary components from the react and react-native libraries.
- 2) LineChart component from the react-native-chart-kit library.
- 3) Dimensions and ActivityIndicator components from the react-native library.
- 4) CustomGauge component from the `../../components/CustomGauge/CustomGauge` file.
- 5) useSelector function from the react-redux library.
- 6) connectSocket function from the `../../components/socket/socket` file.

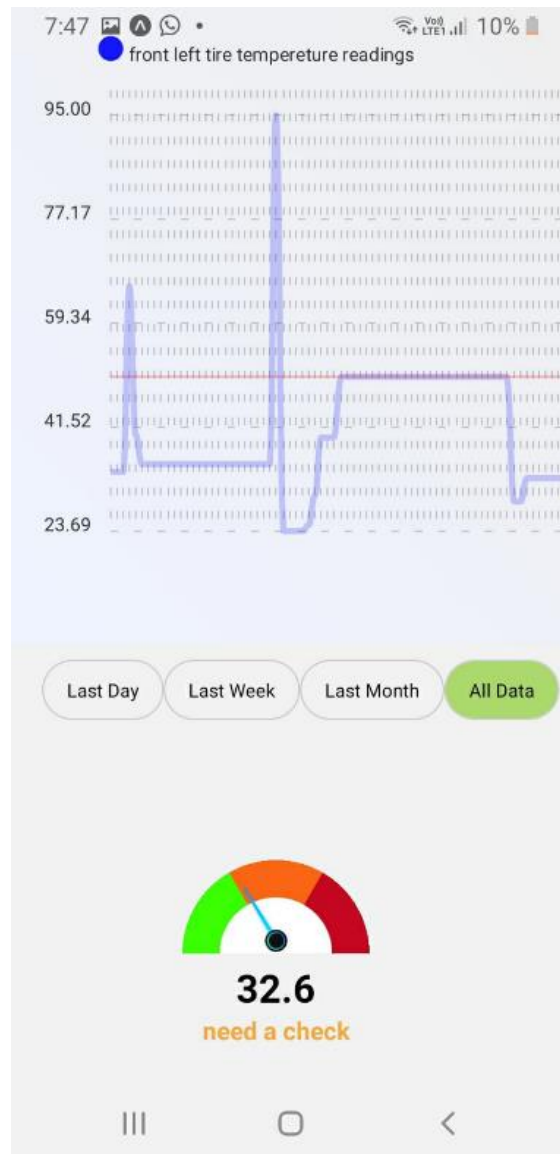
Thirdly, we will talk about the returns of this component:

The component's return statement consists of a conditional rendering based on the `isLoading` state. If `isLoading` is true, an `ActivityIndicator` is displayed to indicate that the data is being loaded. Otherwise, the chart and other UI elements are rendered. When `isLoading` is false, the following elements are rendered:

- 1) LineChart component: Displays a line chart with the filtered data based on the selected time frame.
- 2) View component with `buttonContainer` style: Contains buttons to select different time frames for data filtering.
- 3) TouchableOpacity components: Each button represents a time frame (last day, last week, last month, all data).
- 4) CustomGauge component: Renders a custom gauge component with the latest value from the chart data.

Lastly, we will talk about the exports and the styles of this component:

- 1) `const styles`: An object that contains style definitions for different UI elements used in the component.
- 2) `export default FLTTScreen`;: The default export statement for the `FLTTScreen` component.



TyresTemperatureScreen.js:

First, we will talk about the function of the pressure charts screen, and the function of the functions used in it. The TyresTemperatureScreen component represents a screen that displays a list of menu options related to tire temperature readings. It utilizes the CustomLayout component to render the menu options. The component defines an array of menu options, each containing a name, iconName, and onPress function to handle navigation to specific screens.

Functions and Variables:

- 1) navigation : This variable utilizes the useNavigation hook from the @react-navigation/native library to access the navigation object.
- 2) const menuOptions : This variable is an array of menu options that will be rendered in the CustomLayout component. Each menu option contains a name, iconName, and onPress function.

Secondly, we will talk about the imports of the file:

- 1) `import { View, Text, useEffect, useState, Linking, FlatList, StyleSheet, TouchableOpacity } from 'react-native':` This line imports various components from the react-native library for building the user interface.
- 2) `import { useState } from 'react':` This line imports the useState hook from the react library to manage state in the component.
- 3) `import React from 'react':` This line imports the React library, which is necessary for creating React components.
- 4) `import { Auth } from 'aws-amplify':` This line imports the Auth object from the aws-amplify library, which is used for authentication purposes.
- 5) `import car from '../assets/images/car.png':` This line imports an image file representing a car from the relative path.
- 6) `import { Image } from 'react-native':` This line imports the Image component from the react-native library for rendering images.
- 7) `import CustomLayout from '../components/CustomLayout/CustomLayout':` This line imports the CustomLayout component from the relative path, which is a custom component for rendering the menu options.
- 8) `import { useNavigation } from '@react-navigation/native':` This line imports the useNavigation hook from the @react-navigation/native library, which allows access to the navigation object for screen navigation.
- 9) `import FLTTScreen from './FLTTScreen':` This line imports the FLTTScreen component from the relative path, which represents a screen for displaying tire temperature readings.

Thirdly, we will talk about the returns of this component:

The component returns the CustomLayout component and passes the menuOptions array as a prop. This renders the menu options on the screen using the CustomLayout component.

Lastly, we will talk about the exports. The component is exported to the other files of the project to be used.

Welcome Test1299● parked

Current Distance Moved: 12034.61
Kilometers

Oil change required

 **Frontal Left Tire** >

 **Frontal Right Tire** >

 **Rear Left Tire** >



Amplify folder

This folder contains the files and folders necessary for Amplify to work in the application. It is installed automatically when installing and configuring amplify on the application from terminal.

Car information entry system (Web Application)

To make entries of new cars waiting to be sold or used cars which will be resold. There must be data entries of the information of these cars and linking them with emails to their respective owners when they get sold/resold so that they would register in the database. And then be used by the server.

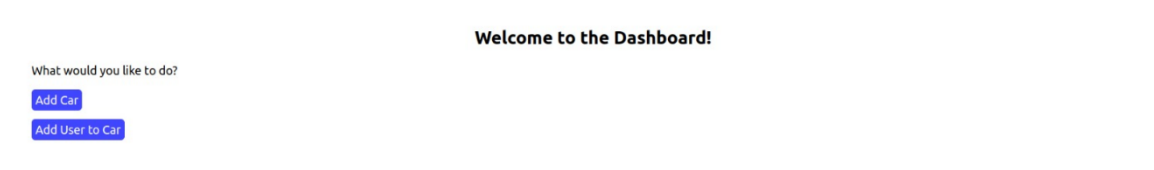
Login Page

This is a simple login page made so that the company employees can enter and access their accounts. If logged in successfully the user will be redirected to the dashboard screen.

A screenshot of a web browser showing a login page. The browser's address bar shows 'localhost:3000'. The page has a dark header with the word 'Login' in white. Below the header, there are two input fields: 'Username:' and 'Password:'. A blue button labeled 'Login' is positioned below the password field.

Dashboard Page

This is a simple dashboard page that can access two other pages through two buttons, the add car and add user to a car pages.

A screenshot of a web browser showing a dashboard page. The browser's address bar shows 'localhost:3000'. The page has a dark header with the text 'Welcome to the Dashboard!'. Below the header, there is a section titled 'What would you like to do?' with two blue buttons: 'Add Car' and 'Add User to Car'.

Add Car Page

This page lets the employee add new cars to the system to then be assigned a user to it when sold through the add user to a car page. The employee is required to select model year and car name for adding an entry.

Add Car

A screenshot of a web form titled 'Add Car'. It has two sections: 'Model Year:' with a dropdown menu showing 'Select Year', and 'Car Name:' with a dropdown menu showing 'Select Car Name'. At the bottom, there is a button labeled 'Add Car'.

When successfully added it gives:

Add Car

The car has been added!

Model Year:

2021

Car Name:

Hyundai Creta

Add Car

Add User to Car Page

In this page the employee assigns a user/buyer email to a car that is listed for sale in the system so that it can be added to the database and an OTP is then sent to the registered user email and then hashed and added to the database for extra security measurements.

Add User to Car

User added successfully

Select a car:

Honda Accord (2022)



Enter user email:

vnspec20@gmail.com

Add User

Server

This server contains the APIs needed to act as a mediator between the embedded system and mobile application. The receives readings from the embedded kit like the sensor's readings and the readings of the location. And then the server communicates with the mobile application and the web application to share information/data between them all. The server also communicates with the database which is a Mongodb database that stores all the necessary information needed by the system.

MongoDB database

The database we have created and been using for this application is a document for every user car and each document contains fields that have vital information for the car and the system. The document has many phases depending on the state of registration of the user and the car. An example for the documents is:

- 1) After adding a car using the web application:

```
_id: ObjectId('64a1b2badf48b394c0a2aca7')
cartype: "Hyundai Creta"
carmodel: 2021
latestTimeStamp: 2023-07-02T17:24:10.231+00:00
distance: 0
▶ FLTT: Array
▶ FRTT: Array
▶ RLTT: Array
▶ RRTT: Array
▶ FLTP: Array
▶ FRTP: Array
▶ RLTP: Array
▶ RRTP: Array
```

- 2) After adding a user to a car in the web application:

```
_id: ObjectId('64a02416c2dc84754a6ce29b')
cartype: "Honda Accord"
carmodel: 2022
distance: 0
▶ FLTT: Array
▶ FRTT: Array
▶ RLTT: Array
▶ RRTT: Array
▶ FLTP: Array
▶ FRTP: Array
▶ RLTP: Array
▶ RRTP: Array
createdAt: 2023-07-01T13:03:18.412+00:00
updatedAt: 2023-07-02T17:24:56.053+00:00
__v: 0
hashedOTP: "$2b$10$7fyYe/iPNJMqyo5TZRYpBubSbs5illwyXGgYTCjs7XjCUdt926X9."
userEmail: "vnspec20@gmail.com"
```

3) After user does the signup process on the mobile application:

```
_id: ObjectId('649de6d8e5e561f295a089df')
cartype: "Toyota Camry"
carmodel: 2021
distance: 111.19492664455873
▶ FLTT: Array
▶ FRTT: Array
▶ RLTT: Array
▶ RRTT: Array
▶ FLTP: Array
▶ FRTP: Array
▶ RLTP: Array
▶ RRTP: Array
createdAt: 2023-06-29T20:17:28.964+00:00
updatedAt: 2023-06-30T21:03:54.524+00:00
__v: 0
userEmail: "vnspec4@gmail.com"
username: "Vnspec4"
userSocket: "-ZfY1YqBpU5M4hQxAAAF"
latestTimeStamp: 2023-06-30T20:41:33.605+00:00
latitude: 31.114729
longitude: 31.778818
```

4) Sample of the FLTT data:

```
{
  _id: ObjectId('649de6d8e5e561f295a089df')
  cartype: "Toyota Camry"
  carmodel: 2021
  distance: 111.19492664455873
  FLTT: Array
    0: Object
      value: 21.5
      timestamp: 2023-06-30T20:40:42.211+00:00
      _id: ObjectId('649f3dca16ced641a6d8e334')
    1: Object
    2: Object
    3: Object
  FRTT: Array
  RLTT: Array
  RRTT: Array
  FLTP: Array
  FRTP: Array
  RLTP: Array
  RRTP: Array
  createdAt: 2023-06-29T20:17:28.964+00:00
  updatedAt: 2023-06-30T21:03:54.524+00:00
}
```

APIs that communicate with embedded kit

This API name is /api/carconnect and is used for connection with the embedded kit. The API receives data from embedded kit which are (FLTT, RLTT, FRTT, RRTT, FLTP, RLTP, FRTP, RRTP, Longitude, Latitude). Then it processes the data then stores the data in mongo collection and sends the data to user (Mobile application) if the user is connected (Socket).

API that adds new car to the system

This API name is /api/v1 and is used to add new cars to the system. The system receives car names and car model then adds them to the system.

API that adds user to car

This API name is /api/v1/adduser and is used to add user to car. The API takes the email from car information system (web application) and then sends an OTP to the user email and then hashes this OTP and adds it to the MongoDB database for security and authentication during user sign-up.

API getnewcars

The API name is /api/v1/getnewcars which sends cars which have no owner in the DB to the web application. It searches the database for cars that have no username and sends it to the web application.

Socket sellcar

The socket name is sellcar. It allows the owner to enter the email address of the buyer and then moves the ownership of the car from the current user to the buyer.

Socket carData

The socket's name is carData. It is responsible for sending the mobile application the new data which is received by the server from the embedded kit.

Socket connection

The socket's name is connection. It acknowledges the connection between the mobile application and the server.

Socket request tires temp and pressure

This is a collection of socket connections which sends the specific tires temperature and pressure when the client (mobile application) requests it.

Socket running status

The socket name is running status. It monitors when the last time the embedded kit has sent the server any value and if the time between the last sent data and now is more than 4 minutes it changes the car status in the client from on to parked.

Socket signinconnect

The socket's name is signinconnect. All it does is that when the user signs in the socket id is sent to the server then stored in the collection inside the MongoDB.

Socket disconnect

The socket's name is disconnect. All it does is remove the socket id from database and close the connection.

distance.js

This file contains a function that calculates the distance between two points. So after calculating the distance it accumulates to the distance in the database.

INTEGRATION AND TESTING

Unit Testing

BMP

Code

```
uint32 temperature = 0;

uint32 pressure = 0;


LCD_init();

TWI_init();

BMP180_Start();


LCD_displayStringRowColumn(0,0,"Press = ");
LCD_displayStringRowColumn(0,14,"hPa");
LCD_displayStringRowColumn(1,0,"Temp = ");
LCD_displayStringRowColumn(1,11,"C");


while(1)
{
    temperature = BMP180_calculateTemperature();
    pressure = BMP180_calculatePressure();
    if(pressure >= 65535)
    {
        LCD_moveCursor(1,14);
        int q = pressure/65536;
        LCD_intgerToString(q);
    }

    LCD_displayTemp(temperature);
    LCD_displayPress(pressure);
}
```

```

_delay_ms(1000);

}

return 0;

```

System diagram

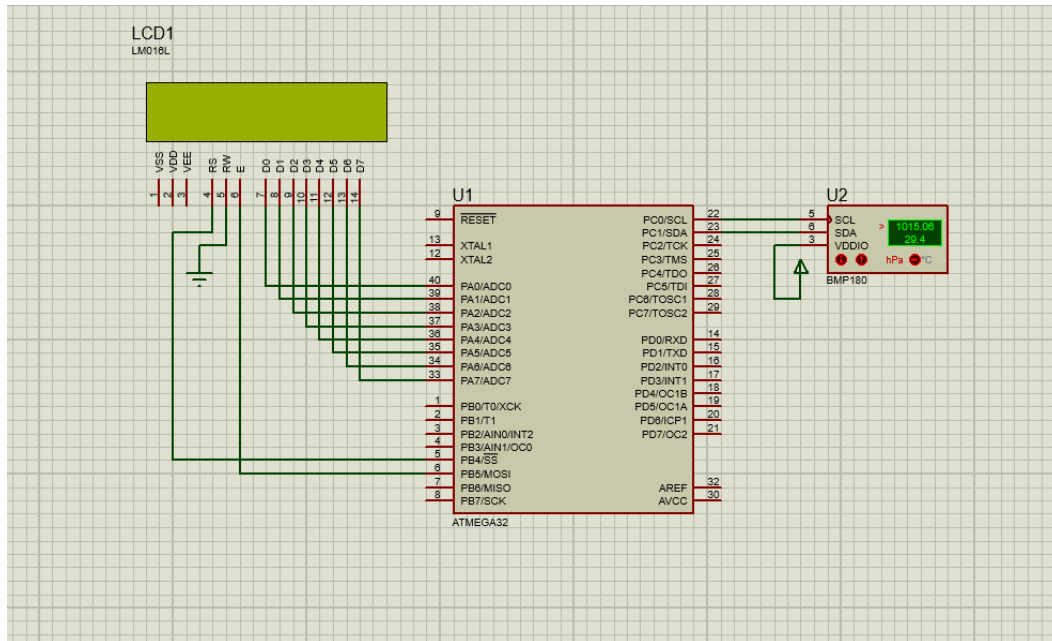


Figure 35: Test Case Diagram 1

Procedure

1. Initialize the LCD, TWI, and BMP180 sensor.
2. Set up the LCD display to show the pressure and temperature.
3. Read the temperature and pressure values from the BMP180 sensor.
4. Display the temperature and pressure values on the LCD screen.
5. Pause for 1 second.
6. Repeat steps 3-5 indefinitely.

Result:

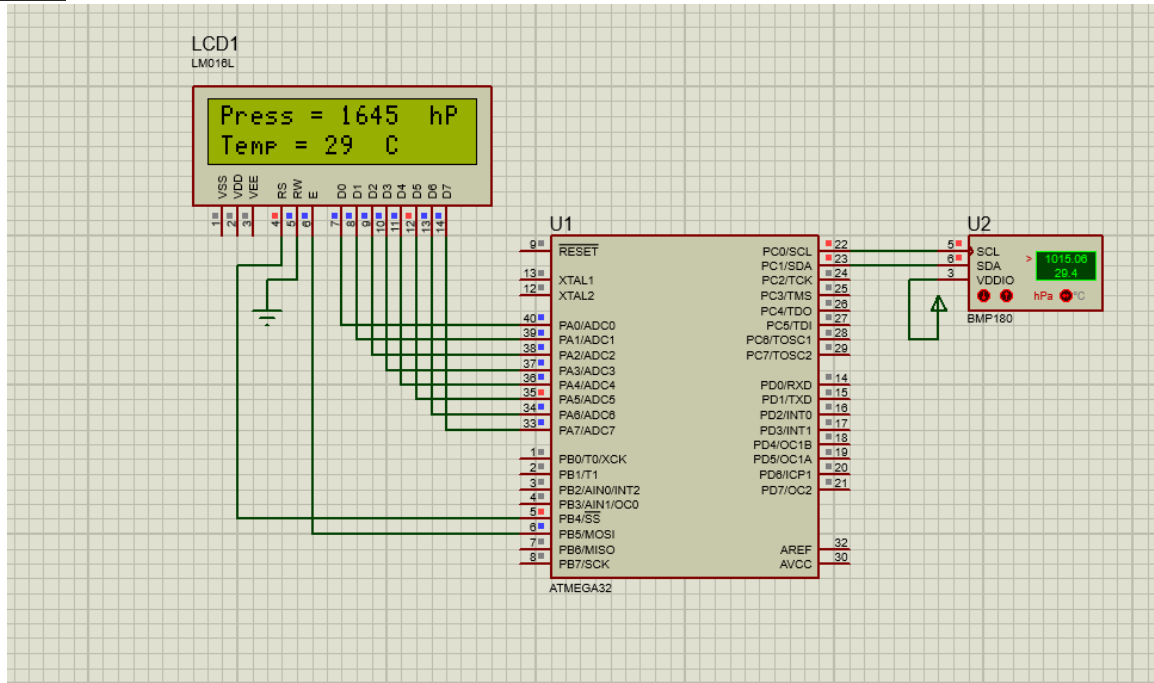


Figure 36: Test Case Diagram 2

From the picture, it appears that the calculated pressure value does not match the expected value. This indicates a potential error in either the code or the calculations being performed

Error detection

1st error:

```
//return ((Pressure_RAW[0] << 8) + Pressure_RAW[1]);
data = ((Pressure_RAW[0] << 16) + (Pressure_RAW[1] << 8) + Pressure_RAW[2]) >> (8 - OSS);

return data;
}
```

solution:

```
// combine the raw pressure data into a single uint32 value
data |= ((uint32_t) Pressure_RAW[0] << 16);
data |= ((uint32_t) Pressure_RAW[1] << 8);
data |= (uint32_t) Pressure_RAW[2];
data = data >> (8 - OSS);
//division by 2 power (8-oss which is in datasheet (0,1))

return data;
```

2nd error:

```
return ((Temperature_RAW[0] << 8) + Temperature_RAW[1]);
```

Solution:

```
data |= ((uint32_t) Temperature_RAW[0] << 8);  
data |= (uint32_t) Temperature_RAW[1];  
return data;
```

Conclusion

- The usage of simple addition causes some issues due to the usage of both signed and unsigned numbers, that is fixed by using bitwise operations instead.
- By using explicit casting, we ensure that the value is treated as an unsigned 32-bit integer during the bitwise operations.
- So, both the casting and the use of bitwise OR operators contribute to achieving accurate results.

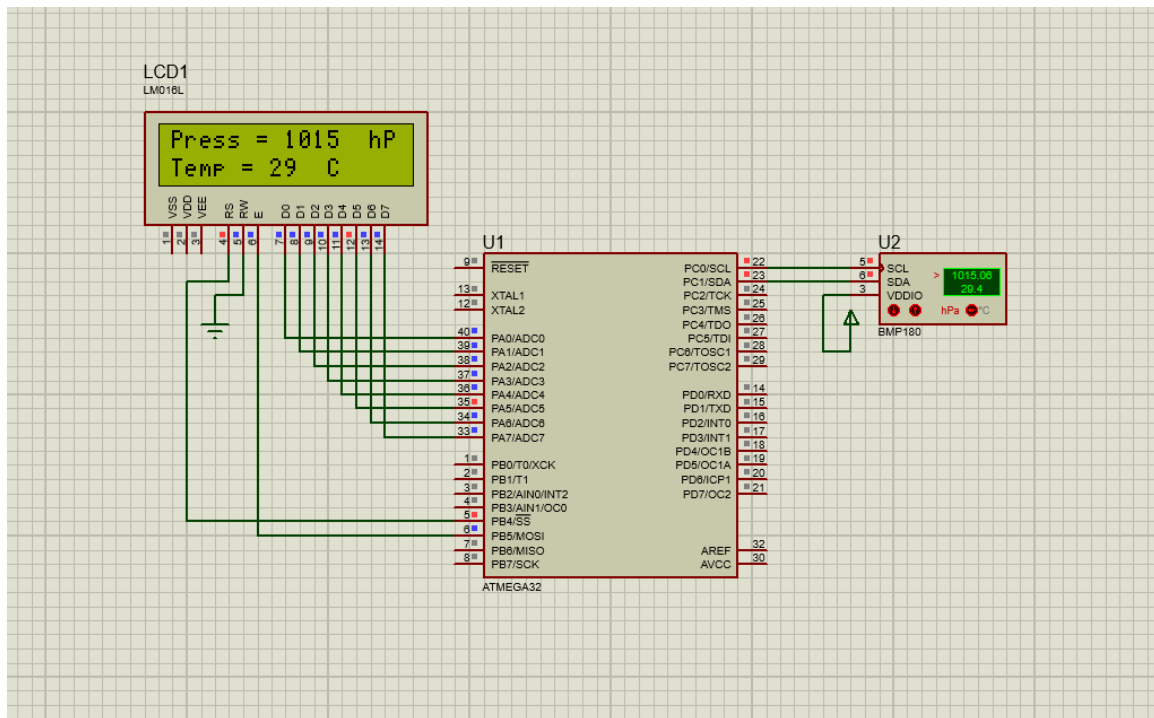


Figure 37: Test Case Diagram 3

GPS

Code

```
GPS_Coordinates t_GPS_Coordinates;

uint8 GPS_DataValidation;

LCD_init();

_delay_ms(260);

LCD_displayString("Test1");

GPS_init();

_delay_ms(260);

_delay_ms(260);

_delay_ms(260);

_delay_ms(260);

_delay_ms(260);
```

```

_delay_ms(260);
_delay_ms(260);
_delay_ms(260);
_delay_ms(260);
_delay_ms(260);
_delay_ms(260);
_delay_ms(260);
LCD_clearScreen();

while(1)
{
    LCD_clearScreen();
    GPS_DataValidation = GPS_getCoordinates(&t_GPS_Coordinates);
    if(GPS_DataValidation == VALID_GPS_DATA)
    {
        LCD_clearScreen();
        LCD_displayStringRowColumn(0,0,"Lat: ");
        LCD_displayStringRowColumn(1,0,t_GPS_Coordinates.Latitude);
        LCD_displayStringRowColumn(2,0,"Lon: ");
        LCD_displayStringRowColumn(3,0,t_GPS_Coordinates.Longitude);
        _delay_ms(260);
        _delay_ms(260);

    }
    else if(GPS_DataValidation == VOID_GPS_DATA)
    {
        LCD_displayString("Void Reading");
        _delay_ms(260);
    }
}

```

```

        _delay_ms(260);

        _delay_ms(260);

        _delay_ms(260);

        _delay_ms(260);

        LCD_clearScreen();

    }

    else

    {

        LCD_displayString("Incorrect Reading");

        _delay_ms(260);

        _delay_ms(260);

        _delay_ms(260);

        _delay_ms(260);

        _delay_ms(260);

        LCD_clearScreen();

    }

}

```

procedure

1. Initialize the LCD and display "Test1".
2. Initialize the GPS module.
3. Clear the LCD screen.
4. Read GPS coordinates and check their validity.
5. If the data is valid, display the latitude and longitude on the LCD screen.
6. If the data is void, display "Void Reading" on the LCD screen.
7. If the data is incorrect, display "Incorrect Reading" on the LCD screen.
8. Repeat steps 4-7 indefinitely.

conclusion

In a closed area, the GPS did not function properly, but it worked correctly in an open area when provided with sufficient delay to synchronize with satellites.

Bluetooth

Code

```
#define AT_COMMAND_CHECK_MODE (const uint8*)"AT\r\n"

#define AT_COMMAND_RESET_MODULE (const uint8*)"AT+RESET\r\n"

#define AT_COMMAND_SET_BAUD_RATE (const uint8*)"AT+UART=115200,1,0\r\n"

#define AT_COMMAND_CONNECT_FIXED_ADDRESS (const uint8*)"AT+CMODE=0\r\n"

#define AT_COMMAND_TO_SET_ROLE_AS_SLAVE (const uint8*)"AT+ROLE=0\r\n"

#define AT_COMMAND_TO_SET_ROLE_AS_MASTER (const uint8*)"AT+ROLE=1\r\n"

#define AT_COMMAND_CONNECT_TO_SLAVE_1 (const uint8*)"AT+BIND=IP\r\n"

#define AT_COMMAND_BIND_TO_SLAVE_2 (const uint8*)"AT+BIND=0022:06:019208\r\n"

#define AT_COMMAND_LINK_TO_SLAVE_2 (const uint8*)"AT+LINK=0022:06:019208\r\n"

#define AT_COMMAND_CONNECT_TO_SLAVE_3 (const uint8*)"AT+BIND=IP\r\n"

#define AT_COMMAND_CONNECT_TO_SLAVE_4 (const uint8*)"AT+BIND=IP\r\n"

void HC05_init(role Role)
{
    UART_init(HC05_COMMANDS_BAUD_RATE);

    GPIO_setupPinDirection(EN_PORT,EN_PIN,PIN_OUTPUT);

    GPIO_setupPinDirection(VCC_PORT,VCC_PIN,PIN_OUTPUT);

    GPIO_writePin(EN_PORT,EN_PIN,LOGIC_LOW);

    GPIO_writePin(VCC_PORT,VCC_PIN,LOGIC_LOW);

    /*To start Command Mode:

    * Initially: VCC -> LOW, EN -> LOW

    * 1- SWITCH OFF VCC

    * 2- WRITE 1 ON EN PIN

    * 3- SET VCC

    * 4- SEND COMMANDS BY UART

    * 5- SWITCH OFF EN PIN
```

```

* 6- SWITCH OFF VCC

* 7- SWITCH ON VCC

* OR 5,6,7- RESET MODULE

* */

/*OPEN COMMAND MODE*/

_delay_ms(3);

GPIO_writePin(EN_PORT,EN_PIN,LOGIC_HIGH);

_delay_ms(5);

GPIO_writePin(VCC_PORT,VCC_PIN,LOGIC_HIGH);

delay(1); /*Optimized.*/

GPIO_writePin(EN_PORT,EN_PIN,LOGIC_LOW);

delay(2); /*Optimized.*/

/*"AT+UART=115200,1,0\r\n"*/

UART_sendString(AT_COMMAND_SET_BAUD_RATE);

/*AT+PSWD=9999\r\n*/

UART_sendString("AT+PSWD=1234\r\n");

/*AT+CMODE=0*/

UART_sendString(AT_COMMAND_CONNECT_FIXED_ADDRESS);

/*AT+ROLE=Role\r\n*/

switch(Role)

{

case SLAVE:

    UART_sendString(AT_COMMAND_TO_SET_ROLE_AS_SLAVE);

```

```

        break;

    case MASTER:

        UART_sendString(AT_COMMAND_TO_SET_ROLE_AS_MASTER);

        UART_sendString(AT_COMMAND_BIND_TO_SLAVE_2);

        UART_sendString(AT_COMMAND_LINK_TO_SLAVE_2);

        break;

    default:

        /*Default is Slave*/

        UART_sendString(AT_COMMAND_TO_SET_ROLE_AS_SLAVE);

    }

    /*Case of Master:*/

    /*AT+BIND=IP*/

    /*AT+ADDR? -> return your address*/

    /*AT+BIND? -> returns IP that you are connected to*/


    while(1);

    /*CLOSE COMMAND MODE*/

    UART_sendString(AT_COMMAND_RESET_MODULE);

    while(1)
    {
        if(UART_receiveByte() == 'O')
        {
            if(UART_receiveByte() == 'K')
            {
                break;
            }
        }
    }

```

```
}  
  
UART_init(DEFAULT_BAUD_RATE);  
  
}
```

Procedure

After studying the datasheet of HC-05 and utilizing serial TTL communication, along with a terminal on the PC, we familiarized ourselves with the usage of AT commands and the process to enter the command mode of the Bluetooth module. We manually experimented with different AT commands until we successfully developed the code mentioned above. This code allows us to receive and send data via Bluetooth according to our specific requirements.

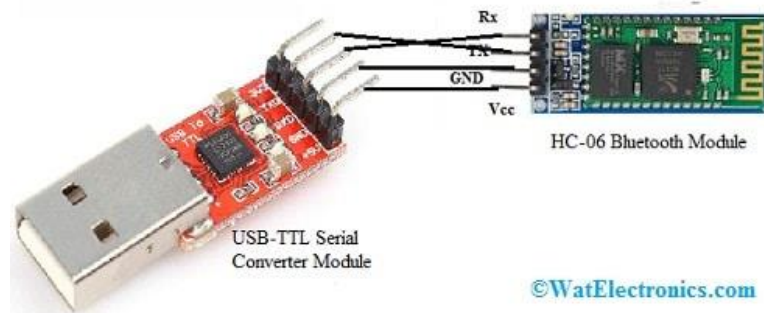


Figure 38: HC-05 Testing Procedure

Conclusion

After successfully implementing the Bluetooth driver, we expanded its functionality by adding additional features. These new features were built upon the existing functions. Ultimately, we utilized the Bluetooth module and its driver to establish remote control of the car.

Integration Testing

Sensing node testing modes:

Overview

- This is a functionality that will not be in the final product.
- It's simple enough to test and showcase the different modes of operations of the Parking Assistance System (PAS) as it is possible to physically change the distance between an Ultrasonic sensor and a nearby object.
- The same can't be said for the other systems, which depend on pressure and temperature.
- Pressure and temperature are very difficult to manipulate, and although some difficult methods exist to increase or decrease the temperature, it is even more difficult to control them to specific values, that's why this feature has been implemented.
- Testing Mode allows pressure and temperature values to be manipulated in order to test and showcase the operation of the system for all ranges of pressure and temperature.

Working Principle

- This is done by pressing a button on the mobile application which moves the car.
- This button sends a signal to the car node, which sends it to the sensing node using DIO pins.
- This button to switch between **four different testing cases**:
 1. Default Case
 2. EDS Case
 3. WCS Case
 4. SLS Case

Default Case

- This is the state which the final product should be in.
- System operates as desired, it takes readings from BMP180 sensor and decides what to do based on real readings, without the use of hard code.

EDS Case

- This case tests the **Early Detection System** only.
- This is done by using hard code to give the system a **moderate pressure value** and a **safe temperature value**.
- In this case, Water Cooling System is deactivated since the temperature is in the safe range.

WCS Case

- This case tests the **Water Cooling System**.
- This is done by using hard code to give the system a **safe pressure value** and a **moderate temperature value**.
- In this case, Early Detection System also works since the temperature is in a moderate range.

SLS Case

- This case tests the **Steering Lock System**.
- This is done by using hard code to give the system a **severe pressure value** and a **safe temperature value**.
- In this case, Water Cooling System is deactivated since the temperature is in the safe range.

Code

```
ISR(INT2_vect)
{
    if (g_testCase == TEST_SEVERE)
    {
        g_testCase = TEST_DEFAULT;
    }
    else
    {
        g_testCase++;
    }
}
```

Procedures

1. Initializing whole system:
 - 1.1. initializing TWI for bmp180
 - 1.2. bmp180 to read temperature and pressure.
 - 1.3. initializing buzzers& LEDs
 - 1.4. initializing SPI to send reading of bmp180.
 - 1.5. initializing timer for ultrasonic& initializing ultrasonic and water pump
2. start looping to check whether exceeded limit for each test case or not to activate interrupt that raises flag.

Errors

- Since the whole system utilizes timer, ICU, I2C, and SPI, the usage of an additional interrupt disrupts the system flow.
- Interrupt Handlers' Execution Time: If the interrupt handlers take too long to execute, it can disrupt the timing and overall operation of the system. Long interrupt execution times can delay the execution of other critical tasks and lead to system instability.
- Interrupt Priority: If the interrupt priorities are not set correctly, it can lead to incorrect or inconsistent behavior. Make sure to assign appropriate priorities to different interrupts to ensure they are handled in the desired order.
- System in case of using interrupt to put high value on a pin collapse and it stuck once the flag is raised.

Solution:

- It's easier to put 1 on pin manually(using button same behavior as push-button) to avoid such an overloading for simple function to be performed.

New Code:

```
/* Test Cases */

typedef enum
{
    TEST_DEFAULT,
    TEST_EDS,
    TEST_WCS,
    TEST_SLS
} TEST_CASE;

/* Test Cases Pins */

#define INTERRUPT_TESTCASE_PORT1    PORTA_ID
#define INTERRUPT_TESTCASE_PIN1    PIN2_ID
#define INTERRUPT_TESTCASE_PORT2    PORTC_ID
#define INTERRUPT_TESTCASE_PIN2    PIN3_ID
..
..
..
while(1)
```

```

{

    if(!DIO_readPin(INTERRUPT_ACTIVATION_FLAG_PORT,INTERRUPT_ACTIVATI
ON_FLAG_PIN)){

        .

        .

    }

    else

    {

        PORTC |= (1 << 2);

        Test_Case_Config();

        switch(g_testCase)
        {

            case TEST_DEFAULT:

                BMP180_calculate(&g_BMP180_readings);

                break;

            case TEST_EDS:

                g_BMP180_readings.temperature = 25;

                g_BMP180_readings.pressure = 3000;

                Parking_LED_default();

                BUZZER_OFF(WARNING_FRONT_PORT_ID,WARNING_FRONT_BUZZER_PIN_ID
);

                break;

            case TEST_WCS:

                g_BMP180_readings.temperature = 40;

                g_BMP180_readings.pressure = 900;

                Parking_LED_default();

```

```

    BUZZER_OFF(WARNING_FRONT_PORT_ID,WARNING_FRONT_BUZZER_PIN_ID
);

        break;

    case TEST_SLS:

        g_BMP180_readings.temperature = 25;

        g_BMP180_readings.pressure = 12000;

        Parking_LED_default();

    BUZZER_OFF(WARNING_FRONT_PORT_ID,WARNING_FRONT_BUZZER_PIN_ID
);

        break;

    }

```

Procedure

1. Sense two input pins and awaits output from car node. (2 pins = 4 possible values)
2. **If value is 0** (binary 00), execute **Default Case** and take readings from BMP180 sensor.
3. **If value is 1** (binary 01), execute **EDS Case** and give the system moderate pressure and safe temperature values. In that case, **Early Detection System** works (Warning LED and Buzzer turn on).
4. **If value is 2** (binary 10), execute **WCS Case** and give the system safe pressure and moderate temperature values. In that case, **Water Cooling System** works (Water Pump turns on, LED connected to same pin as Water Pump for testing also turns on).
5. **If value is 3** (binary 11), execute **SLS Case** and give the system severe pressure and safe temperature values. In that case, **Steering Lock System** works (SLS Flag raised and car stops, LED connected to same pin as SLS Flag for testing also turns on).
6. Continue the rest of the code.

Conclusion

- Now that we use GPIO pins instead of Interrupts, the system flow isn't disrupted, and the system works as intended with no issues.

Testing modes allow us to manipulate the pressure and temperature that the system sees, thus testing different states of the vehicle and how the system handles each

TOOLS AND PLATFORMS EMPLOYED

GCC compiler



Figure 39: GCC Compiler

The GCC (GNU Compiler Collection) is a renowned open-source compiler suite that supports multiple programming languages, including C. In this section, we explore the specific usage of the GCC compiler in the context of C language development for our graduation project. The GCC compiler played a vital role in facilitating the compilation and execution of our C code.

Compilation Process

To develop our C codebase, we chose to employ the GCC compiler due to its widespread adoption and compatibility with various operating systems. The GCC compiler enabled us to compile our C code into executable binaries, which were crucial for testing and evaluating the functionality of our project.

Benefits of GCC Compiler

While our project did not extensively explore the full range of capabilities offered by the GCC compiler, we recognized several key benefits it provided:

Language Support and Portability

The GCC compiler supports multiple programming languages, making it a versatile tool for developers. Its cross-platform compatibility ensured that our C code could be compiled and executed on different operating systems without any significant modifications, thus enhancing the portability of our project.

Error Checking and Diagnostic Messages

During the development process, the GCC compiler proved to be a valuable tool for identifying and rectifying errors in our C code. It performed rigorous error checking and provided detailed diagnostic messages, helping us debug and resolve issues efficiently.

Optimization Options (Not Explored)

While we did not specifically utilize code optimization features for our project, it is important to note that the GCC compiler offers a range of optimization options. These options allow developers to enhance the performance of their code by optimizing execution speed or reducing code size.

Integration with Debuggers (Not Utilized)

The GCC compiler integrates well with popular debugging tools like GDB (GNU Debugger), providing developers with extensive debugging capabilities. Although we did not leverage this functionality in our project, it is worth mentioning as a valuable resource for future projects that require in-depth debugging.

Conclusion

The GCC compiler played an integral role in enabling the development and compilation of our C codebase. Its support for multiple programming languages, error checking capabilities, portability, and integration with debugging tools make it a versatile and powerful compiler suite. Although we did not fully explore its optimization and debugging features, we acknowledge their significance in improving code performance and facilitating efficient debugging. The GCC compiler remains a valuable tool for developers working with the C programming language, contributing to the success and reliability of software projects.

Eclipse IDE



Figure 40: Eclipse IDE

The Eclipse IDE (Integrated Development Environment) is a widely used software development platform that offers comprehensive features and tools for efficient coding, debugging, and project management. In this section, we explore the specific usage of the Eclipse IDE in the context of C language development for our graduation project. While the report primarily focuses on other aspects, the Eclipse IDE played a vital role in facilitating our development workflow and enhancing productivity.

Project Setup and Configuration

We chose the Eclipse IDE as our development environment for its user-friendly interface and extensive support for multiple programming languages, including C. The IDE provided a seamless setup process, enabling us to quickly create and configure our C project with the necessary build settings, libraries, and dependencies.

Code Editing and Navigation

The Eclipse IDE offered a rich set of features for code editing and navigation, improving our productivity during the development process. The IDE provided syntax highlighting, code completion, and intelligent code suggestions, making it easier to write and maintain our C codebase. Additionally, powerful search and navigation tools allowed us to quickly locate functions, variables, and definitions within our project.

Build and Compilation

The Eclipse IDE integrated with the GCC compiler, allowing us to compile our C code within the IDE itself. The build process was streamlined, and the IDE provided real-time error checking, highlighting potential issues as we typed. This significantly reduced the time spent on identifying and resolving compilation errors, ensuring the generation of error-free executable binaries.

Project Management and Version Control

The Eclipse IDE offered convenient project management features, allowing us to organize and manage our codebase efficiently. We could create and manage multiple projects, add, and remove files,

and utilize a built-in version control system or integrate with external version control systems like Git. These capabilities facilitated collaboration and version control throughout the development lifecycle.

Conclusion

The Eclipse IDE played a significant role in streamlining our C language development process. Its intuitive interface, code editing and navigation features, seamless integration with the GCC compiler, and robust debugging support greatly enhanced our productivity and facilitated the creation of high-quality C code. The Eclipse IDE's project management and version control capabilities further contributed to efficient collaboration and code management. Overall, the Eclipse IDE proved to be a valuable tool for C language development, empowering developers with a comprehensive and feature-rich environment for software development.

Visual studio Code with GitHub copilot extension and GitHub version control



Figure 41: VS



GitHub Copilot

Figure 42: GitHub Copilot



Figure 43: GitHub

Visual Studio Code (VS Code) acted as a helpful assistant for code suggestion and completion, enhanced by the GitHub Copilot extension. VS Code provides a versatile and user-friendly coding environment, while the GitHub Copilot extension, powered by ChatGPT, offers AI-powered code suggestions tailored to our coding style. The combined use of VS Code and GitHub Copilot greatly influenced our development process and productivity.

Visual Studio Code Setup and Configuration

Visual Studio Code was selected as our helper IDE due to its popularity, extensive features, and cross-platform compatibility. The setup process was straightforward, and the IDE provided various customization options, allowing us to tailor it to our preferences. We configured the necessary extensions and settings to enhance our C language development experience.

Code Editing and GitHub Copilot Extension

The GitHub Copilot extension for Visual Studio Code introduced an innovative approach to code suggestion. Powered by ChatGPT, this extension analyzed our coding style and context, providing us with AI-generated code suggestions as we typed. It recognized patterns in our code and offered intelligent completions, saving us valuable time when writing boilerplate code or complex conditional statements.

Benefits and Limitations of GitHub Copilot Extension

The GitHub Copilot extension proved highly beneficial in accelerating our coding workflow by generating code snippets and suggesting logical constructs based on our coding style. It expedited the implementation of repetitive code segments, such as if-else conditions with multiple conditions,

allowing us to write complex code structures with minimal effort. However, it is important to note that while GitHub Copilot significantly improved our productivity, the suggested code should be reviewed carefully, as it is not infallible and may not always align perfectly with our desired logic or intent.

Integration with Git and Version Control

Visual Studio Code seamlessly integrated with Git, allowing us to manage version control and collaborate effectively. We could commit changes, switch branches, and resolve conflicts directly within the IDE. This integration enhanced code collaboration and facilitated efficient tracking and management of code revisions.

Conclusion

The combination of Visual Studio Code and the GitHub Copilot extension powered by ChatGPT provided an enhanced coding experience for our C language development. Visual Studio Code offered a comprehensive and customizable IDE, while GitHub Copilot intelligently suggested code snippets based on our coding style, saving us considerable time and effort. The integration with Git further streamlined version control and collaboration. However, it is important to exercise caution and review the suggestions provided by GitHub Copilot, as they are AI-generated and may require human validation to ensure correctness and alignment with our intended logic. Overall, the integration of Visual Studio Code with the GitHub Copilot extension proved to be an asset, improving our productivity and efficiency in C language development.

CONCLUSION

In conclusion, the tire pressure management system and early bursting detection are two important technologies that have a significant impact on vehicle safety and performance.

Tire pressure management system is a system that monitors tire pressure levels in real-time, alerting drivers of any potential issues that may arise. It monitors tire pressure levels on a regular basis to ensure that they are within recommended levels, so it reduces the risk of tire failure, improves fuel efficiency, enhances vehicle stability and handling, and increases road safety.

Early bursting detection system is a technology that detects potential problems with tires before they become critical as it is monitoring tire health on a regular basis and alerting drivers to potential issues, so it allows drivers to address potential issues with their tires before they lead to a tire bursting, reducing the risk of accidents and improving road safety.

Water cooling system for tires is a system that uses water to cool down tires and prevent them from overheating as it cools tires down to reduce the risk of overheating and tire failure so it decreases the risk of tire failure, improves the longevity of tires, and enhances road safety.

Steering lock system is a system that helps to prevent the vehicle from losing control and veering off the road in the event of a tire bursting so engaging the steering lock system in the event of a tire bursting to keep the vehicle moving in a straight line, so it helps to mitigate the consequences of a tire bursting, reducing the risk of accidents and improving road safety.

The tire pressure management system, early bursting detection, water cooling system for tires, ABS control, and steering lock system are all essential technologies for ensuring the safety and reliability of vehicles on the road. By incorporating these technologies into our vehicles.

By implementing these systems, we can reduce the risk of tire failure and improve vehicle safety, helping to make our roads safer for all.

Undoubtedly, the existence of mobile application will play an important role in facilitating the user's daily life in getting his car's status information. As the user can get his car's location information, he can get his car's engine temperature and his car's tires readings. Our future work will add more tools like sending notifications to the user if the server detected dangerous car readings. It will notify the user when the engine is turned on, the user will have option to request monthly reports about his car.

Some advice for drivers to avoid tire bursting in roads:

- Regularly check tire pressure levels: Ensure that tires are inflated to the correct pressure level, as recommended by the vehicle manufacturer.
- Maintain tires well: Regularly inspect tires for signs of wear and tear and replace them when necessary.
- Avoid overloading: Overloading a vehicle can put excessive pressure on the tires, leading to tire failure.
- Drive at a safe speed: Excessive speed can increase the risk of tire failure, especially on rough or uneven roads.
- Avoid sudden braking or accelerating: Sudden changes in speed can cause tires to fail, particularly if they are already in poor condition.
- Replace the tire: If the tire has burst, it will need to be replaced. Do not attempt to drive on a flat tire as this can cause further damage to the vehicle and put you at risk.

In the event of a tire bursting, drivers should follow these steps:

- Remain calm: Try to stay calm and maintain control over the vehicle beside actions of the full system.
- Gradually slow down: Ease off the accelerator and gradually reduce your speed.
- Steer in a straight line: Try to keep the vehicle moving in a straight line, avoiding sudden turns or lane changes.
- Pull over: Once you have slowed down and regained control, pull over to the side of the road as soon as possible.

By following these tips, drivers can help to reduce the risk of tire failure and maintain the safety and reliability of their vehicles on the road. Finally

Stay Safe

APPENDIX

DIO Peripheral

Definition

DIO stands for Digital Input/Output. In a microcontroller, DIO refers to the ability for the microcontroller to read digital signals from external devices (input) and to control external devices by sending digital signals (output).

Operation

DIO in microcontrollers works by using dedicated pins on the microcontroller that can be configured as either inputs or outputs. These pins are connected to external devices, such as sensors or actuators, and the microcontroller can read or write digital signals to these devices through these pins.

When a DIO pin is configured as an input, the microcontroller can read the digital signal present on that pin, which is typically connected to an external device such as a sensor. The microcontroller reads the state of the pin, either high (1) or low (0) and can use that value to make decisions or take action in the program.

When a DIO pin is configured as an output, the microcontroller can write digital signals to that pin, which is typically connected to an external device such as an actuator, like a LED or motor. The microcontroller can set the state of the pin, either high (1) or low (0), thus turning on or off the external device.

The specific instructions and registers used to configure and read/write digital input/output pins can vary depending on the microcontroller, but the basic concept is the same. The microcontroller has dedicated registers and instructions to configure the pin as input or output, read or write to the pin, and check the status of the pin.

Interrupts

Definition

Interrupts are a mechanism in microcontrollers that allows the microcontroller to temporarily halt the execution of its normal program flow and jump to a specific routine, called an interrupt service routine (ISR), when a specific event or condition occurs.

Interrupts are useful in situations where the microcontroller needs to respond to an external event or condition quickly, without waiting for the normal program flow to reach the appropriate section of code. This allows the microcontroller to perform other tasks while waiting for an interrupt to occur.

Interrupts can be triggered by a variety of events or conditions, such as a timer overflow, a button press, or an external signal on an input pin. Each type of interrupt has a corresponding ISR that is executed when the interrupt occurs.

Interrupts are typically managed using special registers and instructions in the microcontroller. These registers and instructions allow the programmer to enable or disable specific interrupts, set the priority of interrupts, and clear interrupt flags.

Operation

When an interrupt event or condition occurs, the microcontroller's interrupt controller is notified. The interrupt controller then checks the priority of the interrupt and the status of other interrupts. If the interrupt has a higher priority than the current instruction or if other interrupts are not enabled, the interrupt controller sends a signal to the microcontroller to stop executing its current instruction and jump to the corresponding ISR.

The microcontroller then saves the current program counter and status register on the stack, and loads the program counter with the starting address of the ISR. The microcontroller then starts executing the instructions in the ISR.

Once the ISR has finished executing, the microcontroller retrieves the saved program counter and status register from the stack and resumes executing the main program flow from where it left off.

It's important to note that the microcontroller can only handle one interrupt at a time, if multiple interrupts occur at the same time, the microcontroller will only handle the highest priority interrupt first, and then move on to the next one.

Timer Peripheral

Definition

In microcontrollers, a timer is a peripheral that generates timed events or pulses, which can be used for various purposes such as counting, measuring, or generating a specific frequency. Timers can also be used to create delays, or to control the speed of various operations.

There are different types of timers available in microcontrollers, but they all generally have the following components:

- A counter: This is the register that keeps track of the current value of the timer. The counter increments or decrements based on the clock signal.
- A clock source: This is the signal that is used to increment or decrement the counter. The clock source can be an internal or external oscillator, or it can be a signal from a pin on the microcontroller.
- A compare or match register: This register stores a value that the counter is compared to. When the counter reaches the value stored in the compare register, an event or an interrupt is generated.
- Control registers: These registers allow the user to configure the timer, such as setting the clock source, the prescaler, the compare value, and the mode of operation.
- Interrupts: Timers can generate interrupts when the counter reaches a specific value or when a specific event occurs.

The most common use of timers in microcontrollers is generating periodic interrupts. For example, a timer can be configured to generate an interrupt every 1ms, and in the interrupt service routine, a counter can be incremented, and certain operations can be performed every 1ms.

Operation

Timers in microcontrollers work by counting the clock cycles and generating timed events or pulses based on the value stored in a compare or match register. The basic working principle of a timer is as follows:

- **Initialization:** The timer is configured by setting the clock source, the prescaler, the compare value, and the mode of operation using the control registers.
- **Counting:** The timer starts counting clock cycles, and the counter increments or decrements based on the clock signal. The counter value is continuously compared to the value stored in the compare or match register.
- **Event generation:** When the counter reaches the value stored in the compare or match register, an event or an interrupt is generated. Depending on the mode of operation, the counter may be reset to zero, or it may continue counting.
- **Interrupt handling:** If the timer is configured to generate an interrupt, the interrupt service routine (ISR) is executed. The ISR can perform specific tasks, such as incrementing a counter, toggling a pin, or updating a variable.
- **Repeat:** The timer continues counting, and the process repeats until the timer is stopped or reset.

There are different types of timers available in microcontrollers, such as:

- **General-purpose timers:** These are the most common type of timers, they can be used for various purposes such as counting, measuring, or generating a specific frequency.
- **Watchdog timers:** These timers are used to reset the microcontroller if it gets stuck in an infinite loop or if it stops responding. **Real-time clocks (RTCs):** These timers are used to keep track of the time and date, even when the microcontroller is in sleep mode.
- It's worth mentioning that timers can be used in different ways, for example, it can be used as a counter, a timer, a PWM generator, a capture, or a comparator.

ICU

Definition

ICU stands for Input Capture Unit. It is a feature found in some microcontrollers that allows the microcontroller to measure the duration of an external signal.

An ICU can capture the state and/or the value of a specified input pin at a specific moment in time, triggered by a specific event or conditions. The ICU then records the time at which this event occurred, allowing the microcontroller to calculate the duration of the external signal. This can be useful in various applications such as measuring the frequency of a signal, measuring the time between two events, or measuring the width of a pulse. ICUs typically include a timer and a capture register. The timer is used to measure the time between the trigger event and the capture event, while the capture register stores the value of the input at the time of capture.

Operation

When the ICU is activated, it starts monitoring a specified input pin. When a specific event or condition occurs on the input pin (such as a rising or falling edge of a signal, a specific value, etc.), the ICU captures the value of the input and the current timer value. This value is then stored in the capture register.

The microcontroller can then use the value stored in the capture register and the current timer value to calculate the duration of the external signal, allowing the microcontroller to measure the frequency of a signal, measure the time between two events, or measure the width of a pulse.

The specific instructions and registers used to configure and read the ICU, as well as the specific events and conditions that can trigger the ICU, can vary depending on the microcontroller.

The microcontroller may have specific registers and instructions to configure the ICU and read the captured value, and to calculate the duration of the external signal.

UART Communication Protocol

Definition

UART, or universal asynchronous receiver-transmitter, is one of the most used device-to-device communication protocols. When properly configured, UART can work with many different types of serial protocols that involve transmitting and receiving serial data.

UART is a type of communication interface commonly used in computers and microcontrollers to communicate with external devices such as modems, sensors, and other microcontrollers.

UARTs can be used for a variety of applications including serial communication with a computer, sending and receiving data over a network, and communicating with other devices such as sensors, displays, and other microcontrollers.

UART is a standard communication interface and can be found in many microcontrollers, computers, and other electronic devices.

Operation

In serial communication, data is transferred bit by bit using a single line or wire. In two-way communication, we use two wires for successful serial data transfer.

A UART typically consists of two main components: a transmitter and a receiver. The transmitter is responsible for sending data out of the device, while the receiver is responsible for receiving data into the device. A UART typically uses two signal lines, one for sending data (transmit) and one for receiving data (receive). Data is sent and received over these signal lines in the form of asynchronous serial data.

Asynchronous means that the data is not sent in a synchronous manner, where timing is controlled by a clock signal. Instead, the data is sent using start and stop bits that are added to the data to indicate

the start and end of a data packet. This allows the receiver to synchronize with the transmitter and properly interpret the received data.

When sending data, the UART converts parallel data from the microcontroller's internal memory into a serial data stream and sends it out over the transmit line. The serial data stream consists of a series of bits, each one representing a piece of the original parallel data. The UART also adds start and stop bits to the data, which are used to indicate the start and end of the data packet.

When receiving data, the UART receives the serial data stream over the receive line and converts it back into parallel data that can be understood by the microcontroller. The UART uses the start and stop bits to determine when the data packet begins and ends, and it uses the data bits to construct the original parallel data.

Additionally, the UART uses a baud rate, which is the number of times the data is sent or received per second. This baud rate is set by the user and should match the baud rate of the device that the UART is communicating with.

It's important to mention that the UART can be configured to have different data width (number of data bits), parity bit, and stop bits, this can be done using the UCSRC register in the case of AVR UART, this configuration is called the frame format.

I2C Communication Protocol

Definition

I2C (Inter-Integrated Circuit) is a communication protocol that allows multiple devices to communicate with one another over a two-wire bus. It is a widely used communication protocol in embedded systems and is commonly used to connect sensors, displays, and other peripherals to a microcontroller or a computer.

The I2C bus consists of two lines: SDA (Serial Data) and SCL (Serial Clock) where the SDA line carries the data and the SCL line carries the clock signal. Each device on the bus has a unique address, and the devices communicate by sending and receiving data to and from each other's address.

In I2C, there are two types of devices: the master device and the slave device. The master device initiates and controls the communication on the bus, while the slave devices respond to the master's requests. The master device generates the clock signal and controls the data flow on the bus, and the slave devices respond to the master's requests by sending or receiving data.

One of the advantages of I2C is that it allows multiple devices to share the same bus and communicate with each other, which reduces the number of wires and pins required to connect the devices. Additionally, I2C is a relatively simple protocol, which makes it easy to implement and use in embedded systems.

Operation

- The I2C communication is controlled by a master device, which generates the clock signal and controls the data flow on the bus. The slave devices respond to the master's requests by sending or receiving data.
- The communication process starts with the master device sending a start condition on the bus. The start condition is a change in the level of the SDA line while the SCL line is high. This signals to all devices on the bus that a communication is about to start.
- After the start condition, the master device sends the address of the slave device it wants to communicate with. The address is 7-bits long and is followed by a read/write bit, which indicates whether the master wants to read from or write to the slave device.
- If the master wants to write data to the slave device, it sends the data on the SDA line, while the SCL line is high. The slave device acknowledges the receipt of the data by sending an ACK bit on the SDA line. The master can then continue to send more data.
- If the master wants to read data from the slave device, it sends a repeated start condition on the bus and then sends the address of the slave device again, but with the read/write bit set to read. The slave device then sends the requested data on the SDA line. The master device acknowledges the receipt of the data by sending an ACK bit on the SDA line. The process repeats until the master has received all the data it needs.
- When the communication is finished, the master device sends a stop condition on the bus. The stop condition is a change in the level of the SDA line while the SCL line is high. This signals to all devices on the bus that the communication is finished.

SPI Communication Protocol

Definition

The Serial Peripheral Interface (SPI) is a synchronous communication protocol commonly used for exchanging data between microcontrollers, sensors, and other peripheral devices. It allows for full-duplex, serial communication, where data is transferred simultaneously in both directions between a master device and one or more slave devices.

Operation

The SPI protocol operates using a master-slave configuration, where the master device initiates and controls the data transfer. Here is a step-by-step overview of how SPI communication typically takes place:

1. Configuration:
 - Mode: The master and slave devices must agree on a common communication mode, which includes parameters such as clock polarity (CPOL) and clock phase (CPHA). These parameters determine the timing and synchronization of data transfer.
 - Bit Order: The devices must also agree on the bit order, specifying whether the Most Significant Bit (MSB) or Least Significant Bit (LSB) is transmitted first.

2. Chip Select (CS) Line:

- The master device selects the specific slave device it wants to communicate with by asserting the Chip Select (CS) line associated with that slave. This line indicates that the selected slave should pay attention to the data transfer.

3. Clock Generation:

- The master device generates a clock signal (SCLK) that synchronizes the data transfer. The clock frequency and timing parameters are determined by the master device and agreed upon by all devices in the system.

4. Data Transfer:

- Data is transferred in a series of full-duplex transactions, where one bit is transmitted per clock cycle.
- Each device has a shift register (usually a shift register with a shift register with one register for transmission and another for reception) where the transmitted and received bits are shifted in and out simultaneously.
- The master device initiates the data transfer by simultaneously shifting out a bit while also shifting in a bit from the slave device.
- Data is transmitted in a sequential manner, with the most significant bit (MSB) or least significant bit (LSB) being transmitted first, depending on the agreed bit order.
- The number of bits transmitted per transaction is typically fixed and determined by the devices' capabilities.

5. Synchronization:

- The clock signal ensures synchronization between the master and slave devices. The data is sampled by the receiver on the rising or falling edge of the clock signal, depending on the agreed clock polarity and phase.

6. Chip Deselect:

- After the desired amount of data has been transmitted, the master deasserts the Chip Select (CS) line associated with the slave device, indicating the end of the transaction.
- The CS line can then be asserted for another slave device if required, initiating communication with a different slave.

SPI is a versatile protocol that offers high data transfer rates and can support multiple slave devices using separate Chip Select lines. It is commonly used in applications that require fast and reliable communication, such as memory devices, displays, sensors, and other peripheral devices in embedded systems.

Wi-Fi

Definition

A Wi-Fi module is a device that allows communication over Wi-Fi wireless network technology. It can be integrated into other devices such as laptops, smartphones, IoT devices, etc. to enable wireless

internet connectivity. The module typically includes a Wi-Fi transceiver, antenna, and other required components to implement the Wi-Fi protocol stack.

Operation

A Wi-Fi module works by transmitting and receiving data over radio waves in the 2.4GHz or 5GHz frequency band. It uses a Wi-Fi transceiver and antenna to send and receive signals to and from Wi-Fi routers or access points.

The Wi-Fi module implements the Wi-Fi protocol stack, which includes the physical (PHY) and media access control (MAC) layers, to transmit and receive data packets over the air. The module also uses network protocols such as TCP/IP to communicate with other devices on the network and to access the internet.

The module can be configured to connect to a specific Wi-Fi network using a unique identifier (SSID) and security key (password), allowing it to securely exchange data with other devices on the network.

Bluetooth

Definition

A Bluetooth module is a device that enables communication between two or more devices using Bluetooth technology. It includes a Bluetooth transceiver, antenna, and other necessary components to implement the Bluetooth protocol stack.

Bluetooth modules are commonly used in a wide range of consumer electronics such as smartphones, smart speakers, wireless headphones, and more to enable short-range wireless connectivity. Bluetooth modules use radio waves in the 2.4GHz frequency band to transmit and receive data, and can be configured to connect to other Bluetooth-enabled devices using a unique identifier.

Operation

- A Bluetooth module works by transmitting and receiving data over short-range radio waves in the 2.4GHz frequency band.
- It uses a Bluetooth transceiver and antenna to send and receive signals to and from other Bluetooth-enabled devices.
- The Bluetooth module implements the Bluetooth protocol stack, which includes the physical (PHY) and link (LINK) layers, to transmit and receive data packets over the air.
- The module also uses various Bluetooth profiles to define the specific capabilities and services that it provides, such as audio streaming, file transfer, or remote control.
- When two Bluetooth devices want to communicate, they first discover each other using the Bluetooth inquiry process.
- Once they are discovered, they can establish a connection using a unique Bluetooth address.
- The connected devices can then exchange data using the Bluetooth protocol, which provides a secure and efficient way to transmit data over short distances.
- The range of a Bluetooth connection is typically around 10 meters, but can vary depending on the environment and the power of the Bluetooth module.

Ultrasonic Sensor



Figure 44: Ultrasonic Sensor

An ultrasonic sensor generates the high-frequency sound (ultrasound) waves. When this ultrasound hits the object, it reflects as echo which is sensed by the receiver as shown in below figure

In ultrasonic module HCSR04, we have to give trigger pulse, so that it will generate ultrasound of frequency 40 kHz. After generating ultrasound i.e., 8 pulses of 40 kHz, it makes echo pin high. Echo pin remains high until it does not get the echo sound back. So, the width of echo pin will be the time for sound to travel to the object and return back. Once we get the time we can calculate distance, as we know the speed of sound. HC-SR04 can measure up to range from 2 cm - 400 cm. The module has only 4 pins, VCC, GND, Trig, and Echo.

When a pulse of 10μsec or more is given to the Trigger pin, 8 pulses of 40 kHz are generated. After this, the Echo pin is made high by the control circuit in the module. The echo pin remains high till it gets an echo signal of the transmitted pulses back. The time for which the echo pin remains high, i.e., the width of the Echo pin gives the time taken for generated ultrasonic sound to travel towards the object and return. Using this time and the speed of sound in air, we can find the distance of the object using a simple formula for distance using speed and time.

HC-SR04 Pin Description

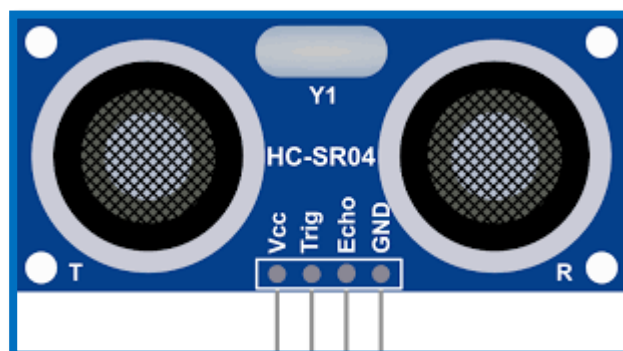


Figure 45: Ultrasonic Pinout

VCC: +5 V supply

TRIG: Trigger input of sensor. Microcontroller applies 10 us trigger pulse to the HCSR04 ultrasonic module.

ECHO: Echo output of sensor. Microcontroller reads/monitors this pin to detect the obstacle or to find the distance.

GND: Ground

HC-SR04 Working Principle

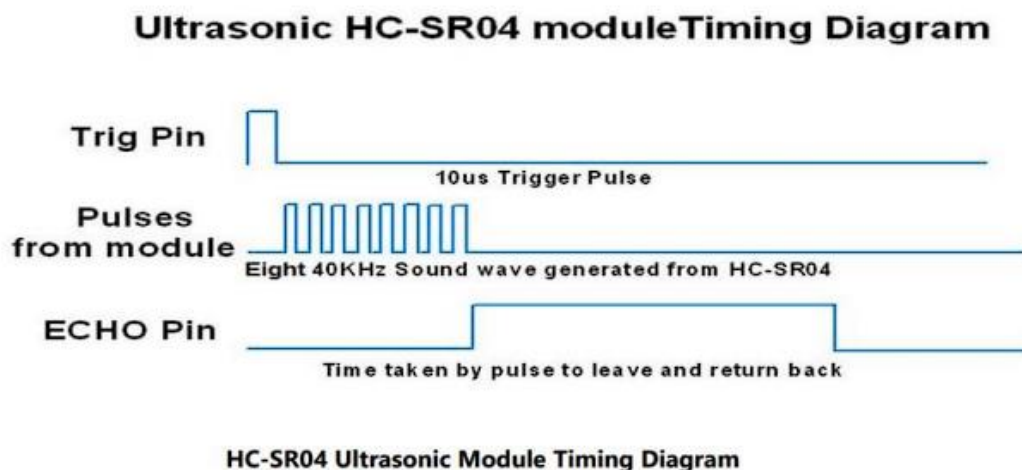


Figure 46: Ultrasonic Timing Diagram

- 1- We need to transmit trigger pulse of at least 10 us to the HC-SR04 Trig Pin.
- 2- Then the HC-SR04 automatically sends Eight 40 kHz sound wave and wait for rising edge output at Echo pin.
- 3- When the rising edge capture occurs at Echo pin, start the Timer and wait for falling edge on Echo pin.
- 4- As soon as the falling edge is captured at the Echo pin, read the count of the Timer. This time count is the time required by the sensor to detect an object and return from an object.

Distance calculations

Distance = Time * Speed. The speed of sound waves is 343 m/s, then

$$\text{Distance} = \frac{343 * \text{Time of High(ECHO) Pulse}}{2}$$

Where total distance is divided by 2 because signal travels from HC-SR04 to object and returns to the module HC-SR-04.

Steps of calculations

1. ATmega32 microcontroller needs to transmit at least 10 us trigger pulse to the HC-SR04 Trig Pin.
2. After getting a trigger pulse, HC-SR04 automatically sends eight 40 kHz sound waves and the microcontroller waits for rising edge output at the Echo pin.

3. When the rising edge capture occurs at the Echo pin which is connected to an input of ATmega32, start Timer of ATmega32 and again wait for a falling edge on the Echo pin.
4. As soon as the falling edge is captured at the Echo pin, the microcontroller reads the count of the Timer. This time count is used to calculate the distance to an object
5. Here we are using the Input Capture unit on PD6(ICP1) pin.

From configuration we can adjust both prescaler and clock of Microcontroller which by default will be 16MHZ.

Sound velocity = 343.00 m/s = 34300 cm/s.

So, we get

$$\frac{34300 \times 16 \times 10^{-6}}{2 \times \text{prescaler}} + 1, \text{ where 1 is for tolerance}$$

GPS Module



Figure 47: GPS module

The Ublox NEO-6M GPS module is a high-performance GPS receiver module made by u-blox, a Swiss company. It has a built-in ceramic patch antenna, a 50-channel receiver, and supports GPS, GLONASS, Galileo, and QZSS satellite systems. It is capable of providing position, velocity, and time information with high accuracy and low power consumption. It also has a built-in EEPROM for storing configuration settings and can be controlled and configured using NMEA and UBX protocol commands. It can be used in a variety of applications such as drones, robots, and navigation systems.

Working Principle of Ublox NEO-6M

The Ublox NEO-6M GPS module works on the principle of trilateration. Trilateration is a method of determining the location of an object or point by measuring the distance to it from at least three known reference points.

The GPS system consists of a network of satellites that orbit the Earth, and a GPS receiver, such as the Ublox NEO-6M, that is used to determine the location of an object on the Earth's surface. The GPS receiver uses signals from multiple satellites to determine the location of the receiver.

When the GPS receiver is turned on, it begins searching for signals from GPS satellites. Once it receives signals from at least three satellites, it can calculate its position using trilateration. The GPS receiver uses the time of arrival of the signals from each satellite to determine the distance to each satellite, and then calculates its position using the known positions of the satellites and the measured distances.

REFERENCES



Figure 48: QR Code for the references